دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)
دانشکده‌ی مهندسی کامپیوتر

عنوان:

تمرین سری اول

آشنایی با تنظیم دقیق با پارامترهای کارآمد (PEFT)

درس:

پردازش زبان طبیعی پیشرفته

دانشجو:

امید طرب آور (۴۰۴۱۳۱۰۹۷)

استاد:

آقای دکتر زینلی

فهرست مطالب

عنوان **صفحه**

۱- آماده‌سازی داده‌ها و ارزیابی مدل پایه ۶

۱-۱- آماده‌سازی داده‌ها و تنظیمات آزمایش ۶

۱-۱-۱- معرفی مجموعه‌داده‌ها و تفکیک دامنه‌ها ۶

۱-۱-۲- بارگذاری، پاک‌سازی و آماده‌سازی متن‌ها ۸

۱-۱-۳- مدل پایه و پیکربندی محاسباتی ۹

۱-۲- روش محاسبه پرپلکسیتی ۱۱

۱-۲-۱- تعریف ریاضی پرپلکسیتی ۱۲

۱-۲-۲- حذف توکن‌های Padding و تجمیع صحیح زیان ۱۳

۱-۳- نتایج ارزیابی مدل پایه ۱۴

۱-۳-۱- نتایج کمی ارزیابی ۱۵

۱-۳-۲- تفسیر و تحلیل نتایج ۱۵

۱-۴- نکات پیاده‌سازی و ملاحظات عملی ۱۷

۱-۴-۱- توکن‌سازی، Padding پویا و ساخت Batchها ۱۷

۱-۴-۲- پیاده‌سازی دستی محاسبه پرپلکسیتی ۱۹

۱-۵- جمع‌بندی فصل ۲۲

۲- پیاده‌سازی و آموزش مدل با روش LoRA ۲۳

۲-۱- مبانی و پیکربندی روش LoRA ۲۳

۲-۱-۱- سازوکار تنظیم کم‌رتبه ۲۳

۲-۱-۲- لایه‌های هدف و ابرپارامترهای LoRA ۲۴

۲-۱-۳- انجماد مدل پایه و پارامترهای قابل‌آموزش ۲۶

۲-۲- فرایند تنظیم دقیق روی اخبار فارسی ۲۷

۲-۲-۱- داده‌ها و شرایط آموزش ۲۸

۲-۲-۲- بهینه‌سازی و ثبت روند آموزش ۲۹

۲-۲-۳- ذخیره مدل و خروجی‌های آموزش ۳۱
۲-۳- ارزیابی مدل تنظیم‌شده با LoRA ۳۱
۲-۳-۱- نتایج کمی روی مجموعه‌های آزمون ۳۲
۲-۳-۲- تحلیل عملکرد درون‌دامنه و برون‌دامنه ۳۲
۲-۴- نکات پیاده‌سازی و ملاحظات عملی ۳۵
۲-۴-۱- شناسایی ماژول‌های هدف LoRA ۳۵
۲-۴-۲- پیاده‌سازی حلقه آموزش با تابع train_model ۳۶
۲-۴-۳- اجرای اعتبارسنجی دوره‌ای ۳۹
۲-۵- جمع‌بندی فصل ۳۹
۳- پیاده‌سازی دستی و آموزش مدل با روش Bottleneck Adapter ۴۱
۳-۱- مبانی و طراحی Bottleneck Adapter ۴۱
۳-۱-۱- مفهوم تنظیم دقیق با Adapter ۴۱
۳-۱-۲- معماری Bottleneck Adapter ۴۲
۳-۱-۳- محل قرارگیری و اتصال Residual ۴۴
۳-۱-۴- انجماد مدل پایه و پارامترهای قابل‌آموزش ۴۶
۳-۲- تزریق دستی و فرایند آموزش Bottleneck Adapter ۴۷
۳-۲-۱- شناسایی لایه‌های متنی و تزریق Adapter ۴۷
۳-۲-۲- داده‌ها و شرایط آموزش ۴۹
۳-۲-۳- بهینه‌سازی و ثبت روند آموزش ۵۱
۳-۳- ارزیابی مدل تنظیم‌شده با Bottleneck Adapter ۵۳
۳-۳-۱- نتایج کمی روی مجموعه‌های آزمون ۵۳
۳-۳-۲- تحلیل عملکرد درون‌دامنه و برون‌دامنه ۵۴
۳-۴- نکات پیاده‌سازی و ملاحظات عملی ۵۶
۳-۴-۱- پیاده‌سازی Bottleneck Adapter و اتصال آن به MLP ۵۷
۳-۴-۲- شناسایی لایه‌های متنی و تزریق دستی Adapter ۵۸

۳-۵- جمع‌بندی فصل ۶۱
۴- ارزیابی نهایی و مقایسه روش‌های LoRA و Bottleneck Adapter ۶۳
۴-۱- مقایسه پارامترهای قابل‌آموزش ۶۳
۴-۱-۱- تحلیل پارامترکارایی دو روش ۶۴
۴-۲- مقایسه نتایج پرپلکسیتی سه مدل ۶۶
۴-۲-۱- جدول نهایی پرپلکسیتی ۶۶
۴-۲-۲- مقایسه عملکرد درون‌دامنه‌ای ۶۶
۴-۲-۳- مقایسه عملکرد برون‌دامنه‌ای ۶۷
۴-۳- تحلیل فراموشی فاجعه‌بار و نرخ افت عملکرد ۶۸
۴-۳-۱- اثر تنظیم دقیق خبری بر دانش عمومی مدل ۶۹
۴-۳-۲- محاسبه Degradation Ratio ۶۹
۴-۳-۳- بررسی وقوع و شدت فراموشی فاجعه‌بار ۷۰
۴-۳-۴- قابل‌توجیه‌بودن افت و راهکار نظری کاهش آن ۷۱
۴-۴- مقایسه مصالحه میان یادگیری اخبار و حفظ دانش قبلی ۷۲
۴-۴-۱- مقایسه بهبود اخبار در برابر افت ویکی‌پدیا ۷۲
۴-۴-۲- تعیین روش دارای مصالحه بهتر ۷۳
۴-۵- ارزیابی کیفی تولید متن ۷۴
۴-۵-۱- پرامپت‌ها و شرایط تولید ۷۵
۴-۵-۲- مقایسه خروجی‌ها برای پرامپت پارلمانی ۷۵
۴-۵-۳- مقایسه خروجی‌ها برای پرامپت هواشناسی ۷۷
۴-۵-۴- مقایسه خروجی‌ها برای پرامپت وزارت بهداشت ۷۸
۴-۵-۵- جمع‌بندی ارزیابی کیفی ۸۰
۴-۶- مقایسه منابع محاسباتی و زمان آموزش ۸۰
۴-۶-۱- مقایسه بیشینه حافظه GPU ۸۱
۴-۶-۲- مقایسه زمان آموزش و اعتبارسنجی ۸۲

۴-۶-۳- تحلیل فنی اختلاف مصرف منابع و سرعت آموزش ۸۴
۴-۷- تحلیل منحنی‌های زیان آموزش ۸۵
۴-۷-۱- تحلیل منحنی آموزش و اعتبارسنجی LoRA ۸۶
۴-۷-۲- تحلیل منحنی آموزش و اعتبارسنجی Bottleneck Adapter ۸۸
۴-۷-۳- مقایسه مستقیم روند آموزش دو روش ۸۹
۴-۷-۴- بررسی همگرایی دو روش ۹۱
۴-۷-۵- کفایت سه دور آموزش ۹۲
۴-۷-۶- بررسی خطر Overfitting ۹۲
۴-۸- جمع‌بندی فصل ۹۳

فصل اول

آماده‌سازی داده‌ها و ارزیابی مدل پایه

در این فصل، مجموعه‌داده‌های اخبار فارسی و ویکی‌پدیا آماده‌سازی شده و مدل google/gemma-3n-E2B بدون اعمال تغییر در وزن‌ها ارزیابی می‌شود. سپس با پیاده‌سازی دستی معیار پرپلکسیتی و حذف توکن‌های Padding از محاسبه زیان، یک معیار پایه برای سنجش اثر روش‌های تنظیم دقیق در فصل‌های بعد به دست می‌آید.

۱-۱- آماده‌سازی داده‌ها و تنظیمات آزمایش

به‌منظور مقایسه منصفانه عملکرد مدل پایه با روش‌های LoRA و Adapter، لازم است نحوه تفکیک داده‌ها، مراحل آماده‌سازی ورودی و تنظیمات محاسباتی در تمام آزمایش‌ها ثابت نگه داشته شود. در این بخش، ابتدا مجموعه‌داده‌های موجود و نقش هر یک در فرایند آموزش و ارزیابی معرفی می‌شوند؛ سپس مراحل بارگذاری، پاک‌سازی و توکن‌سازی متن‌ها شرح داده شده و در پایان، مدل پایه و پیکربندی محاسباتی آزمایش ارائه می‌شود.

۱-۱-۱- معرفی مجموعه‌داده‌ها و تفکیک دامنه‌ها

مجموعه‌داده مورد استفاده از دو حوزه محتوایی متفاوت تشکیل شده است. داده‌های درون‌دامنه شامل متون خبری فارسی هستند و به‌عنوان دامنه هدف تنظیم دقیق در نظر گرفته می‌شوند. داده‌های برون‌دامنه نیز از متون عمومی ویکی‌پدیا به زبان‌های مختلف تشکیل شده‌اند و برای ارزیابی میزان حفظ توانایی عمومی مدل پس از یادگیری سبک نگارش اخبار به کار می‌روند.

بررسی فایل‌های مجموعه‌داده نشان داد که برای هر دو دامنه، بخش‌های آموزش، اعتبارسنجی و آزمون وجود دارد. داده‌های اخبار شامل ۱۰۰۰۰ نمونه آموزشی، ۱۰۰۰ نمونه اعتبارسنجی و ۱۰۰۰ نمونه آزمون هستند. در بخش ویکی‌پدیا نیز ۲۰۰۰ نمونه آموزشی، ۲۰۰۰ نمونه اعتبارسنجی و ۶۰۰۰ نمونه آزمون وجود دارد. در اجرای نهایی، بخش آموزش اخبار برای تنظیم دقیق، بخش اعتبارسنجی اخبار برای پایش روند آموزش و بررسی همگرایی، و بخش‌های آزمون اخبار و ویکی‌پدیا برای ارزیابی نهایی استفاده شدند. داده‌های آموزش و اعتبارسنجی ویکی‌پدیا وارد خط لوله اصلی آزمایش نشدند.

جدول ۱–۱: مشخصات مجموعه‌داده‌های موجود و نحوه استفاده از آن‌ها در آزمایش

مجموعه‌داده	نوع دامنه	بخش داده	تعداد نمونه	نحوه استفاده در آزمایش
اخبار فارسی	درون‌دامنه	آموزش	۱۰۰۰۰	تنها مجموعه مورد استفاده برای آموزش پارامترهای قابل‌آموزش در روش‌های LoRA وAdapter
اخبار فارسی	درون‌دامنه	اعتبارسنجی	۱۰۰۰	برای پایش روند آموزش استفاده شد. یک Mini-batch اعتبارسنجی پس از هر ۲۰ ریزBatch آموزشی ارزیابی شد و پس از پایان هر Epoch نیز زیان و پرپلکسیتی روی کل ۱۰۰۰ نمونه اعتبارسنجی محاسبه گردید. این داده‌ها در محاسبه گرادیان یا به‌روزرسانی پارامترها مشارکت نداشتند.
اخبار فارسی	درون‌دامنه	آزمون	۱۰۰۰	ارزیابی عملکرد مدل پایه و مدل‌های تنظیم‌شده روی داده‌های دیده‌نشده در دامنه خبری
ویکی‌پدیا	برون‌دامنه	آموزش	۲۰۰۰	در فرایند آموزش کاربرد نداشت؛ زیرا مطابق صورت تمرین، تنظیم دقیق باید فقط روی داده‌های آموزش اخبار انجام می‌شد. واردکردن این بخش به آموزش، هدف آزمایش در سنجش حفظ دانش عمومی مدل بدون بازآموزی روی ویکی‌پدیا را تغییر می‌داد.
ویکی‌پدیا	برون‌دامنه	اعتبارسنجی	۲۰۰۰	در اجرای نهایی استفاده نشد؛ زیرا پایش دوره‌ای آموزش با بخش اعتبارسنجی اخبار انجام گرفت و ویکی‌پدیا صرفاً به‌عنوان دامنه برون‌دامنه‌ای برای ارزیابی نهایی حفظ دانش عمومی در نظر گرفته شد.
ویکی‌پدیا	برون‌دامنه	آزمون	۶۰۰۰	ارزیابی عملکرد برون‌دامنه‌ای مدل و اندازه‌گیری میزان حفظ توانایی عمومی آن پس از تنظیم دقیق روی اخبار

در اجرای اصلی، چهار بخش از داده‌ها وارد خط لوله محاسباتی شدند: بخش آموزش اخبار، بخش اعتبارسنجی اخبار، بخش آزمون اخبار و بخش آزمون ویکی‌پدیا. داده‌های آموزش اخبار تنها برای بهینه‌سازی پارامترهای قابل‌آموزش در روش‌های LoRA و Adapter استفاده شدند. داده‌های اعتبارسنجی اخبار برای اندازه‌گیری زیان دوره‌ای و پرپلکسیتی پایان هر Epoch به کار رفتند، اما در محاسبه گرادیان یا به‌روزرسانی وزن‌ها مشارکت نداشتند. مجموعه‌های آزمون نیز کاملاً از آموزش و اعتبارسنجی جدا ماندند و فقط برای ارزیابی نهایی استفاده شدند.

عدم استفاده از داده‌های آموزش ویکی‌پدیا ناشی از نبود داده یا محدودیت در دسترسی به آن نبود، بلکه با طراحی اصلی آزمایش مطابقت داشت. هدف این بود که مدل فقط با متون خبری سازگار شود و سپس تأثیر این سازگارسازی بر عملکرد آن در متون عمومی بررسی شود. چنانچه داده‌های ویکی‌پدیا نیز در آموزش وارد می‌شدند، حفظ یا کاهش پرپلکسیتی این دامنه دیگر نمی‌توانست مستقیماً به توانایی روش تنظیم دقیق در حفظ دانش پیشین مدل نسبت داده شود.

بخش اعتبارسنجی اخبار در اجرای نهایی برای پایش روند آموزش مورد استفاده قرار گرفت. در طول آموزش هر روش، پس از هر ۲۰ ریزBatch، زیان یک Mini-batch از داده‌های اعتبارسنجی بدون محاسبه گرادیان اندازه‌گیری شد تا منحنی پیوسته‌تری از رفتار Validation به دست آید. علاوه بر این، پس از پایان هر یک از سه Epoch، زیان و پرپلکسیتی روی کل ۱۰۰۰ نمونه اعتبارسنجی اخبار محاسبه شد. بنابراین، برای هر روش سه مقدار کامل Validation در سطح Epoch و ۳۷۵ نقطه Mini-validation در طول آموزش ثبت شد. داده‌های اعتبارسنجی در هیچ مرحله‌ای برای انتخاب گرادیان، به‌روزرسانی وزن‌ها یا ارزیابی نهایی استفاده نشدند.

وجود دو مجموعه آزمون با حوزه‌های متفاوت، امکان بررسی دو جنبه از عملکرد مدل را فراهم می‌کند. آزمون اخبار، میزان یادگیری و تعمیم مدل در دامنه هدف را نشان می‌دهد؛ درحالی‌که آزمون ویکی‌پدیا برای سنجش تعمیم برون‌دامنه‌ای و میزان حفظ دانش عمومی مدل به کار می‌رود. در نتیجه، کاهش پرپلکسیتی اخبار پس از تنظیم دقیق به‌عنوان نشانه سازگاری بهتر با سبک خبری و افزایش پرپلکسیتی ویکی‌پدیا به‌عنوان افت عملکرد برون‌دامنه‌ای تحلیل می‌شود.

۱-۱-۲- بارگذاری، پاک‌سازی و آماده‌سازی متن‌ها

داده‌ها در قالب فایل‌های JSONL و در دو پوشه in_domain و out_domain سازمان‌دهی شده بودند. در اجرای نهایی، فایل in_domain/train.jsonl برای آموزش روی اخبار، فایل in_domain/dev.jsonl برای اعتبارسنجی، فایل in_domain/test.jsonl برای ارزیابی درون‌دامنه‌ای و فایل out_domain/test.jsonl برای ارزیابی برون‌دامنه‌ای بارگذاری شدند. فایل‌های آموزش و اعتبارسنجی ویکی‌پدیا در پوشه داده موجود بودند، اما وارد خط لوله اصلی آزمایش نشدند.

هر سطر از فایل‌های JSONL شامل یک نمونه مستقل و دارای دو فیلد text و lang بود. فیلد text محتوای متنی نمونه و فیلد lang زبان آن را مشخص می‌کرد. تابع بارگذاری، محتوای فیلد متنی را استخراج کرده، مقادیر تهی را حذف و تمام ورودی‌ها را به نوع رشته تبدیل می‌کرد. همچنین فاصله‌های اضافی ابتدا و انتهای متن‌ها حذف می‌شد و نمونه‌هایی که پس از پاک‌سازی فاقد محتوای معتبر بودند، در مجموعه نهایی قرار نمی‌گرفتند.

در اجرای نهایی، متغیر DEBUG_LIMIT برابر None تنظیم شد؛ بنابراین هیچ زیرمجموعه آزمایشی یا محدودیت مصنوعی روی تعداد نمونه‌ها اعمال نشد. در نتیجه، تمام ۱۰۰۰۰ خبر آموزشی، ۱۰۰۰ خبر اعتبارسنجی، ۱۰۰۰ خبر آزمون و ۶۰۰۰ متن آزمون ویکی‌پدیا در مراحل متناظر مورد استفاده قرار گرفتند.

پس از بارگذاری متن‌های خام، توکن‌ساز متناظر با مدل Gemma 3n فراخوانی شد. این توکن‌ساز هر متن را به دنباله‌ای از شناسه‌های عددی تبدیل کرده و توکن‌های ویژه موردنیاز مدل را نیز به ورودی اضافه می‌کرد. طول حداکثر توالی برابر ۲۵۶ توکن تعیین شد؛ بنابراین نمونه‌هایی که پس از توکن‌سازی طولی بیشتر از این مقدار داشتند، تا ۲۵۶ توکن برش داده شدند. این مقدار در آموزش، اعتبارسنجی و آزمون برای هر سه مدل ثابت نگه داشته شد تا مقایسه نتایج تحت شرایط یکسان انجام گیرد.

Padding به‌صورت پویا و در سطح هر Batch انجام شد. در این روش، توالی‌های کوتاه‌تر تنها تا طول بلندترین نمونه موجود در همان Batch تکمیل می‌شوند و برخلاف Padding ثابت، تمام نمونه‌ها الزاماً تا طول ۲۵۶ توکن گسترش نمی‌یابند. این انتخاب

تعداد توکن‌های غیرواقعی و محاسبات غیرضروری را کاهش می‌دهد و با توجه به محدودیت حافظه GPU، نسبت به Padding ثابت مناسب‌تر است. جهت Padding نیز در سمت راست توالی تنظیم شد.

توکن‌ساز برای هر Batch دو ورودی اصلی تولید می‌کرد. input_ids شامل شناسه توکن‌های متن و attention_mask مشخص‌کننده موقعیت توکن‌های واقعی و Padding بود. در attention_mask، مقدار یک به توکن‌های معتبر و مقدار صفر به موقعیت‌های Padding اختصاص داده می‌شد. این ماسک علاوه بر هدایت سازوکار توجه مدل، در محاسبه دستی پرپلکسیتی نیز برای حذف موقعیت‌های Padding از مجموع زیان مورد استفاده قرار گرفت.

برای داده‌های آموزشی، برچسب‌ها از یک نسخه کپی‌شده از input_ids ساخته شدند. سپس مقدار برچسب در موقعیت‌های Padding برابر ۱۰۰- قرار گرفت. تابع Cross-Entropy در PyTorch موقعیت‌هایی با این مقدار را نادیده می‌گیرد؛ بنابراین توکن‌هایی که صرفاً برای هم‌اندازه‌کردن توالی‌های یک Batch اضافه شده‌اند، در محاسبه زیان و به‌روزرسانی پارامترها مشارکت نمی‌کنند.

داده‌های آموزش اخبار در یک DataLoader با اندازه ریزBatch برابر ۴ قرار گرفتند. درهم‌ریزی نمونه‌ها برای مجموعه آموزش فعال بود تا مدل در هر دور داده‌ها را با ترتیب ثابتی مشاهده نکند. با وجود ۱۰۰۰۰ نمونه آموزشی، هر دور شامل ۲۵۰۰ ریزBatch بود. همچنین با استفاده از دو مرحله تجمیع گرادیان، اندازه مؤثر Batch برابر ۸ حفظ شد.

مجموعه‌های اعتبارسنجی و آزمون بدون درهم‌ریزی و در حالت استنتاج پردازش شدند. اندازه Batch عمومی ارزیابی برابر ۴ بود. برای ارزیابی کامل بخش اعتبارسنجی مدل Adapter، به‌منظور کاهش احتمال کمبود حافظه، اندازه Batch برابر ۲ در نظر گرفته شد. این تفاوت بر تعریف نظری زیان یا پرپلکسیتی اثر نمی‌گذارد؛ زیرا مجموع Cross-Entropy تمام توکن‌های معتبر بر تعداد کل همان توکن‌ها تقسیم می‌شود.

۱-۱-۳- مدل پایه و پیکربندی محاسباتی

مطابق محدودیت صورت تمرین، مدل مورد استفاده باید از خانواده Gemma 3n انتخاب می‌شد. در این آزمایش، مدل google/gemma-3n-E2B به‌عنوان نسخه متناسب با سخت‌افزار موجود انتخاب شد. مدل در نخستین اجرا از مخزن Hugging Face دریافت و در پوشه محلی model ذخیره شد. در اجراهای بعدی، مدل و توکن‌ساز از همین مسیر محلی بارگذاری شدند تا نیازی به دریافت مجدد فایل‌ها نباشد و نسخه مورد استفاده در تمام مراحل ثابت باقی بماند.

مدل با کلاس Gemma3nForConditionalGeneration بارگذاری شد. در مرحله ارزیابی پایه، هیچ تغییر معماری، تنظیم دقیق یا به‌روزرسانی وزنی روی آن انجام نگرفت. مدل در حالت ارزیابی قرار گرفت و تمام محاسبات بدون اجرای عملیات backward یا به‌روزرسانی توسط Optimizer انجام شدند. در نتیجه، مقادیر پرپلکسیتی پایه مستقیماً عملکرد مدل پیش‌آموزش‌دیده را پیش از سازگارسازی با داده‌های خبری نشان می‌دهند.

محاسبات روی GPU و از طریق CUDA انجام شدند. پارامترهای مدل با نوع داده float16 بارگذاری شدند تا مصرف حافظه کاهش یافته و از محاسبات بهینه FP16 روی GPU استفاده شود. برای جلوگیری از Overflow هنگام ساخت شاخه صوتی استفاده‌نشده Gemma 3n، مقدار پیکربندی gradient_clipping این شاخه پیش از بارگذاری به بزرگ‌ترین مقدار محدود و

قابل‌نمایش در float16 محدود شد. این اصلاح تنها به پیکربندی شاخه صوتی مربوط بود و ورودی‌های این آزمایش کاملاً متنی بودند. در محاسبه دستی Cross-Entropy، Logitهای هر قطعه به float32 تبدیل شدند تا زیان با پایداری عددی بیشتری محاسبه و تجمیع شود.

هیچ روش کوانتیزه‌سازی چهار یا هشت‌بیتی در بارگذاری مدل استفاده نشد. کاهش مصرف حافظه از طریق نوع داده float16، اندازه ریزBatch برابر ۴، تجمیع گرادیان و Gradient Checkpointing انجام شد، نه با تقریب کوانتیزه‌شده وزن‌ها. Gradient Checkpointing فقط در مراحل آموزش LoRA و Adapter فعال بود و در ارزیابی مدل پایه کاربردی نداشت. این روش بخشی از فعال‌سازی‌های میانی را هنگام پس‌انتشار مجدداً محاسبه می‌کند و در نتیجه مصرف حافظه را کاهش می‌دهد، اما می‌تواند زمان آموزش را افزایش دهد.

توکن‌ساز مدل دارای اندازه واژگان ۲۶۲۴۰۰ بود. شناسه توکن Padding برابر صفر و شناسه توکن پایان توالی برابر یک ثبت شد. از آنجا که توکن‌ساز از ابتدا دارای توکن Padding معتبر بود، نیازی به تعریف یا جایگزینی توکن جدید وجود نداشت. جهت Padding نیز برای تمام مراحل در سمت راست توالی قرار گرفت.

برای افزایش بازتولیدپذیری، بذر تصادفی کتابخانه استاندارد پایتون، PyTorch و CUDA برابر ۴۲ تنظیم شد. اگرچه ارزیابی مدل پایه ماهیتی قطعی دارد، ثابت‌کردن بذر برای مراحل آموزشی LoRA و Adapter اهمیت بیشتری دارد؛ زیرا ترتیب نمونه‌های آموزشی و برخی عملیات محاسباتی می‌توانند تحت تأثیر تصادفی‌بودن قرار گیرند.

جدول ۱–۲: پیکربندی مدل و تنظیمات اصلی آزمایش

مؤلفه	مقدار استفاده‌شده
مدل پایه	google/gemma-3n-E2B
کلاس مدل	Gemma3nForConditionalGeneration
کلاس توکن‌ساز	AutoTokenizer
اندازه واژگان	۲۶۲۴۰۰
شناسه توکنPadding	0
شناسه توکنEOS	1
جهتPadding	راست
طول حداکثر توالی	۲۵۶ توکن
بستر اجرا	CUDA
ظرفیت حافظهGPU	32گیگابایت
نوع داده مدل	float16

کوانتیزه‌سازی	استفاده نشد
اندازه ریز Batchآموزش	۴
مراحل تجمیع گرادیان	۲
اندازه مؤثرBatch	۸
اندازه Batchارزیابی پایه	۴
Gradient Checkpointing	فعال در آموزش
بخش اعتبارسنجی	in_domain/dev.jsonl با ۱۰۰۰ نمونه
تناوب Mini-validation	هر ۲۰ ریزBatch آموزشی
اندازه Batch Mini-validation	۲
اندازه Batch Full validation	Adapter: 2 ،LoRA: 4
بذر تصادفی	۴۲
نرخ یادگیری	2×10^{-4}
تعداد دورهای آموزش	۳

اندازه مؤثر Batch از حاصل‌ضرب اندازه ریزBatch در تعداد مراحل تجمیع گرادیان به دست می‌آید:

$$B_{effective} = B_{micro} \times B_{accumulation} = 4 \times 2 = 8$$

تجمیع گرادیان امکان حفظ اندازه مؤثر Batch برابر ۸ را بدون پردازش هم‌زمان تمام نمونه‌ها فراهم کرد. در تنظیم نهایی، افزایش اندازه ریزBatch از ۲ به ۴ و کاهش مراحل تجمیع گرادیان از ۴ به ۲ تعداد ریزBatchهای هر Epoch را کاهش داد، درحالی‌که اندازه مؤثر Batch ثابت باقی ماند. نرخ یادگیری برابر 2×10^{-4} و تعداد Epochها برابر ۳ نیز دقیقاً مطابق مقادیر تعیین‌شده در صورت تمرین انتخاب شدند؛ هرچند این دو ابرپارامتر در ارزیابی مدل پایه نقشی ندارند و اثر آن‌ها در بخش‌های مربوط به LoRA و Adapter بررسی خواهد شد.

با آماده‌سازی مجموعه‌داده‌ها و تثبیت تنظیمات مدل و محیط محاسباتی، شرایط لازم برای استخراج یک معیار پایه قابل‌مقایسه فراهم شد. در بخش بعد، تعریف ریاضی پرپلکسیتی و نحوه محاسبه دستی آن با حذف توکن‌های Padding تشریح می‌شود.

۱-۲- روش محاسبه پرپلکسیتی

برای تعیین عملکرد مدل پیش از اعمال روش‌های تنظیم دقیق، معیار پرپلکسیتی به‌صورت دستی پیاده‌سازی شد. در این پیاده‌سازی، ابتدا زیان Cross-Entropy مربوط به پیش‌بینی توکن بعدی برای تمام توکن‌های معتبر مجموعه ارزیابی محاسبه شده

و سپس با اعمال تابع نمایی بر میانگین این زیان، مقدار پرپلکسیتی به دست آمد. مطابق الزام تمرین، موقعیت‌های Padding در محاسبه میانگین زیان مشارکت داده نشدند.

۱-۲-۱- تعریف ریاضی پرپلکسیتی

مدل Gemma 3n یک مدل زبانی خودرگرسیو است؛ بنابراین احتمال هر توکن را با شرط‌کردن بر توکن‌های پیش از آن برآورد می‌کند. اگر یک توالی شامل توکن‌های $x_1, x_2, \dots, x_T$ باشد، احتمال اختصاص‌یافته از سوی مدل به این توالی به‌صورت زیر تعریف می‌شود:

$$p_\theta(x_1, x_2, \dots, x_T) = \prod_{t=1}^{T} p_\theta(x_t | x_{<t}), \quad x_{<t} = (x_1, \dots, x_{t-1})$$

برای هر موقعیت معتبر، زیان منفی لگاریتم احتمال توکن صحیح محاسبه می‌شود. اگر Z_t بردار Logitهای مدل در موقعیت t و V اندازه واژگان باشد، احتمال توکن هدف با تابع Softmax و زیان Cross-Entropy آن به‌شکل زیر به دست می‌آید:

$$p_\theta(x_t | x_{<t}) = \frac{\exp(Z_{t,x_t})}{\sum_{j=1}^{V} \exp(Z_{t,j})}$$

$$\ell_t = -\log p_\theta(x_t | x_{<t})$$

$$\mathcal{L}_{mean} = \frac{1}{N} \sum_{t=1}^{N} \ell_t$$

در این روابط، N تعداد کل توکن‌های هدف معتبر در مجموعه ارزیابی است. پس از محاسبه میانگین Cross-Entropy، پرپلکسیتی از طریق اعمال تابع نمایی به دست می‌آید:

$$PPL = \exp(\mathcal{L}_{mean}) = \exp(-\frac{1}{N} \sum_{t=1}^{N} \log p_\theta(x_t | x_{<t}))$$

این رابطه را می‌توان به‌صورت معادل زیر نیز نوشت:

$$PPL = (\prod_{t=1}^{N} p_\theta(x_t | x_{<t}))^{-\frac{1}{N}}$$

بر این اساس، پرپلکسیتی را می‌توان معکوس میانگین هندسی احتمال‌هایی در نظر گرفت که مدل به توکن‌های صحیح اختصاص داده است. مقدار کمتر پرپلکسیتی نشان می‌دهد مدل به‌طور متوسط احتمال بیشتری به توکن‌های واقعی داده و در پیش‌بینی توکن بعدی عملکرد بهتری داشته است. برای مثال، کاهش پرپلکسیتی یک مدل پس از تنظیم دقیق روی اخبار نشان می‌دهد عدم‌اطمینان آن در مدل‌سازی توالی‌های خبری کاهش یافته است.

از آنجا که مدل برای هر موقعیت ورودی یک بردار Logit تولید می‌کند، خروجی‌ها و برچسب‌ها پیش از محاسبه زیان یک موقعیت نسبت به یکدیگر جابه‌جا شدند. آخرین Logit کنار گذاشته شد؛ زیرا در توالی ورودی، توکن بعدی متناظر با آن وجود

ندارد. به‌طور مشابه، نخستین توکن ورودی از برچسب‌ها حذف شد؛ زیرا هدف Logit نخست، پیش‌بینی توکن دوم توالی است. این جابه‌جایی را می‌توان به‌صورت زیر نمایش داد:

$$Z_{shift} = Z_{[:,0:T-1,:]}$$

$$Y_{shift} = Y_{[:,1:T]}$$

در نتیجه، Logit تولیدشده در موقعیت t با توکن واقعی موقعیت $t+1$ مقایسه شد. این هم‌ترازی برای محاسبه صحیح زیان در مدل‌سازی زبان علّی ضروری است؛ زیرا مقایسه Logit هر موقعیت با توکن همان موقعیت، مدل را به بازسازی ورودی و نه پیش‌بینی توکن بعدی ارزیابی می‌کرد.

۱-۲-۲- حذف توکن‌های Padding و تجمیع صحیح زیان

به دلیل استفاده از Padding پویا، توالی‌های هر Batch تا طول بلندترین نمونه همان Batch تکمیل شدند. این توکن‌های افزوده‌شده بخشی از متن واقعی نیستند و نباید در محاسبه Cross-Entropy یا پرپلکسیتی دخالت کنند. برای تشخیص این موقعیت‌ها، از attention_mask تولیدشده توسط توکن‌ساز استفاده شد. مقدار ماسک برای توکن‌های واقعی برابر یک و برای توکن‌های Padding برابر صفر بود.

در مرحله آماده‌سازی Batch، برچسب‌ها از input_ids ساخته شدند و مقدار برچسب تمام موقعیت‌های Padding برابر ۱۰۰- قرار گرفت. پس از جابه‌جایی علّی، این مقدار نیز در shifted_labels حفظ شد. بنابراین، در محاسبه دستی پرپلکسیتی نیازی به اعمال مستقیم ماسک بولی روی Logitها نبود و معتبر یا نامعتبر بودن هر موقعیت از مقدار برچسب آن تعیین شد.

$$Y_{shift} = Y_{[:,1:T]}$$

پس از این مرحله، تنها موقعیت‌هایی برای محاسبه زیان انتخاب شدند که مقدار ماسک جابه‌جاشده آن‌ها برابر یک بود. اگر مجموعه موقعیت‌های معتبر با $\mathcal{V}$ نمایش داده شود، تعداد توکن‌های معتبر و مجموع زیان به‌صورت زیر تعریف می‌شوند:

$$\mathcal{V} = \{(b,t) | Y_{b,t}^{shift} \neq -100\}$$

$$N_{valid} = \sum_{b,t} 1[Y_{b,t}^{shift} \neq -100]$$

$$NLL_{total} = \sum_{(b,t)\in\mathcal{V}} -\log p_\theta(Y_{b,t}^{shift} | x_{b,\leq t})$$

در پیاده‌سازی نهایی، ابعاد Batch و طول توالی پس از جابه‌جایی به یک بعد تبدیل شدند. سپس Cross-Entropy با گزینه ignore_index=-100 اجرا شد. از آنجا که برچسب تمام موقعیت‌های Padding برابر ۱۰۰- بود، این موقعیت‌ها به‌طور خودکار از مجموع زیان کنار گذاشته شدند. تعداد توکن‌های معتبر نیز با شمارش برچسب‌هایی محاسبه شد که مقدار آن‌ها با ۱۰۰- برابر نبود. بنابراین، توکن‌های Padding نه در صورت مجموع زیان و نه در مخرج میانگین سراسری مشارکت داشتند.

به دلیل اندازه بسیار بزرگ واژگان مدل Gemma 3n، تبدیل هم‌زمان تمام Logitهای یک Batch به float32 می‌توانست حافظه زیادی مصرف کند. به همین دلیل، Logitها و برچسب‌های مسطح‌شده به بلوک‌های کوچک‌تر از موقعیت‌ها تقسیم شدند. برای هر

بلوک، تنها همان بخش از Logitها به float32 تبدیل و Cross-Entropy آن با "reduction="sum محاسبه شد. سپس مجموع زیان بلوک‌ها به متغیر سراسری افزوده شد. این تقسیم‌بندی فقط نحوه اجرای محاسبات را تغییر می‌دهد و بر مقدار ریاضی نهایی پرپلکسیتی اثری ندارد:

$$\mathcal{L}_{global} = \frac{\sum_{k=1}^{K} NLL_k}{\sum_{k=1}^{K} N_k}$$

$$PPL_{global} = \exp(\mathcal{L}_{global})$$

در این روابط، K تعداد Batchها، NLL_k مجموع زیان توکن‌های معتبر Batch شماره k و N_k تعداد توکن‌های معتبر همان Batch است. این روش باعث می‌شود سهم هر Batch متناسب با تعداد واقعی توکن‌های آن تعیین شود. برای مثال، Batch دارای ۸۰۰ توکن معتبر باید نسبت به Batch دارای ۳۰۰ توکن معتبر سهم بیشتری در میانگین نهایی داشته باشد.

در مقابل، میانگین‌گیری ساده از Loss یا Perplexity جداگانه Batchها می‌توانست نتیجه‌ای نادرست ایجاد کند؛ زیرا تعداد توکن‌های معتبر Batchها به دلیل تفاوت طول متن‌ها و Padding پویا یکسان نبود. افزون بر این، تابع نمایی یک تابع غیرخطی است و میانگین پرپلکسیتی Batchها با پرپلکسیتی حاصل از میانگین سراسری زیان برابر نیست:

$$\frac{1}{K}\sum_{k=1}^{K} \exp(\mathcal{L}_k) \neq \exp(\frac{\sum_{k=1}^{K} N_k \mathcal{L}_k}{\sum_{k=1}^{K} N_k})$$

بنابراین، تابع نمایی تنها یک مرتبه و پس از تجمیع زیان تمام توکن‌های معتبر مجموعه ارزیابی اعمال شد. این شیوه، پرپلکسیتی را مستقل از نحوه تقسیم نمونه‌ها میان Batchها می‌کند؛ در نتیجه، کاهش اندازه Batch به دلیل محدودیت حافظه GPU نباید مقدار نظری پرپلکسیتی را تغییر دهد و تنها بر زمان اجرا اثر می‌گذارد.

مدل در زمان ارزیابی در حالت eval و تحت ()torch.inference_mode اجرا شد و قابلیت Cache نیز برای کاهش مصرف حافظه غیرفعال بود. خروجی مدل در نوع داده float16 تولید شد، اما در هر بلوک محاسبه Cross-Entropy، فقط Logitهای همان بلوک به float32 تبدیل شدند تا پایداری عددی افزایش یابد و از تبدیل یک‌باره کل Tensor بزرگ Logitها جلوگیری شود. تابع ارزیابی برای هر مجموعه، میانگین Cross-Entropy، پرپلکسیتی، تعداد کل توکن‌های هدف معتبر و مجموع Negative Log-Likelihood را برمی‌گرداند. همین پیاده‌سازی بدون تغییر برای مدل پایه، LoRA و Adapter و همچنین برای ارزیابی کامل Validation استفاده شد تا تمام مقادیر تحت یک تعریف مشترک قابل‌مقایسه باشند.

۱-۳- نتایج ارزیابی مدل پایه

پس از آماده‌سازی داده‌ها و پیاده‌سازی تابع محاسبه پرپلکسیتی، مدل پایه بدون اعمال هیچ‌گونه تنظیم دقیق یا تغییر در وزن‌ها روی مجموعه آزمون اخبار فارسی و مجموعه آزمون ویکی‌پدیا ارزیابی شد. هدف از این مرحله، تعیین یک نقطه مرجع کمی برای اندازه‌گیری اثر روش‌های LoRA و Adapter در فصل‌های بعد است. مطابق صورت تمرین، ارزیابی پایه باید روی داده‌های آزمون درون‌دامنه و برون‌دامنه انجام شود تا تغییر عملکرد مدل پس از تنظیم دقیق در هر دو حوزه قابل‌اندازه‌گیری باشد.

۱-۳-۱- نتایج کمی ارزیابی

در ارزیابی مدل پایه، برای هر مجموعه آزمون مجموع زیان Cross-Entropy تنها روی توکن‌های معتبر محاسبه شد و توکن‌های Padding از محاسبات کنار گذاشته شدند. سپس میانگین زیان سراسری با تقسیم مجموع منفی لگاریتم درست‌نمایی بر تعداد کل توکن‌های معتبر به دست آمد و پرپلکسیتی از اعمال تابع نمایی بر این مقدار محاسبه شد.

نتایج حاصل از اجرای نوت‌بوک در جدول ۱-۳ ارائه شده‌اند.

جدول ۱–۳: نتایج ارزیابی صفرشات مدل پایه روی مجموعه‌های آزمون اخبار و ویکی‌پدیا

مجموعه آزمون	نوع دامنه	تعداد توکن معتبر	میانگین Cross-Entropy	پرپلکسیتی
اخبار فارسی	درون‌دامنه	۲۵۱,۷۴۵	۲.۰۴۲۴۴۵	۷.۷۰۹۴۳۹
ویکی‌پدیا	برون‌دامنه	۱,۲۹۲,۶۱۱	۱.۹۹۳۱۸۵	۷.۳۳۸۸۶۸

مقادیر جدول نشان می‌دهند که مدل پایه روی مجموعه آزمون اخبار فارسی به میانگین زیان ۲.۰۴۲۴۴۵ و پرپلکسیتی ۷.۷۰۹۴۳۹ رسیده است. در مجموعه ویکی‌پدیا نیز میانگین زیان و پرپلکسیتی به‌ترتیب برابر ۱.۹۹۳۱۸۵ و ۷.۳۳۸۸۶۸ به دست آمده‌اند. این نتایج بدون استفاده از داده‌های آموزش اخبار و پیش از اعمال هرگونه تغییر در وزن‌های مدل محاسبه شده‌اند؛ بنابراین مستقیماً عملکرد مدل پیش‌آموزش‌دیده را تحت خط لوله ارزیابی نهایی این آزمایش نشان می‌دهند.

تعداد توکن‌های معتبر مجموعه ویکی‌پدیا حدود ۵.۱۳ برابر مجموعه اخبار است:

$$\frac{N_{Wiki}}{N_{News}} = \frac{1{,}292{,}611}{251{,}745} \approx 5.13$$

بااین‌حال، تفاوت اندازه دو مجموعه اثری بر وزن‌دهی درونی پرپلکسیتی هر مجموعه ندارد؛ زیرا میانگین زیان هر مجموعه مستقل از دیگری و بر اساس تعداد واقعی توکن‌های معتبر همان مجموعه محاسبه شده است. تعداد بیشتر توکن‌های ویکی‌پدیا فقط به این معناست که مقدار گزارش‌شده برای این دامنه بر حجم بزرگ‌تری از پیش‌بینی‌های توکن بعدی استوار است.

۱-۳-۲- تفسیر و تحلیل نتایج

برای تحلیل اختلاف عملکرد مدل پایه در دو دامنه، تفاوت میانگین زیان و پرپلکسیتی آن‌ها محاسبه می‌شود:

$$\Delta\mathcal{L} = \mathcal{L}_{Wiki} - \mathcal{L}_{News} = 1.993185 - 2.042445 = -0.049260$$

$$\Delta PPL = PPL_{Wiki} - PPL_{News} = 7.338868 - 7.709439 = -0.370571$$

اختلاف نسبی پرپلکسیتی ویکی‌پدیا نسبت به اخبار نیز برابر است با:

$$\Delta PPL_{\%} = \frac{7.338868 - 7.709439}{7.709439} \times 100 = -4.807\%$$

بنابراین، پرپلکسیتی مدل پایه روی ویکی‌پدیا حدود ۴.۸۱ درصد کمتر از پرپلکسیتی آن روی اخبار بوده است. از آنجا که پرپلکسیتی تابع نمایی میانگین Cross-Entropy است، رابطه میان اختلاف زیان و نسبت پرپلکسیتی‌ها به‌صورت زیر نیز قابل‌بررسی است:

$$\frac{PPL_{Wiki}}{PPL_{News}} = \exp(\mathcal{L}_{Wiki} - \mathcal{L}_{News}) = \exp(-0.049260) \approx 0.951933$$

این نسبت نشان می‌دهد پرپلکسیتی مدل روی ویکی‌پدیا تقریباً ۹۵.۱۹ درصد پرپلکسیتی آن روی اخبار بوده است؛ به‌عبارت دیگر، عدم‌اطمینان مؤثر مدل در پیش‌بینی توکن‌های ویکی‌پدیا حدود ۴.۸۱ درصد کمتر بوده است. این اختلاف بسیار بزرگ نیست، اما از اختلاف نسخه قبلی محسوس‌تر است و نشان می‌دهد مدل پایه در شرایط این آزمایش، پیش از تنظیم دقیق اندکی عملکرد بهتری روی مجموعه ویکی‌پدیا داشته است.

عملکرد اندکی بهتر مدل پایه روی ویکی‌پدیا می‌تواند ناشی از نزدیکی بیشتر بخشی از متون عمومی ویکی‌پدیا به توزیع پیش‌آموزش مدل، تفاوت در ساختار جمله‌ها، ترکیب زبانی، طول نمونه‌ها یا توزیع توکن‌ها باشد. بااین‌حال، اختلاف مشاهده‌شده به‌تنهایی برای نسبت‌دادن این برتری به یک عامل مشخص کافی نیست؛ زیرا دو مجموعه از نظر موضوع، سبک نگارش و ترکیب داده یکسان نیستند. بنابراین، نتیجه قابل‌دفاع این است که مدل پایه در شرایط این آزمایش روی ویکی‌پدیا اندکی پرپلکسیتی کمتری ثبت کرده است، نه اینکه ویکی‌پدیا ذاتاً مجموعه ساده‌تری باشد.

با وجود برتری نسبی ویکی‌پدیا، مقادیر پرپلکسیتی دو مجموعه همچنان در یک بازه نزدیک قرار دارند و تفاوت آن‌ها حدود ۴.۸۱ درصد است. این ویژگی برای ادامه آزمایش اهمیت دارد؛ زیرا تغییرات مشاهده‌شده پس از آموزش را می‌توان نسبت به خط مبنای مستقل هر دامنه بررسی کرد. در فصل‌های بعد، کاهش پرپلکسیتی اخبار نسبت به ۷.۷۰۹۴۳۹ به‌عنوان شواهد سازگاری بهتر با دامنه هدف در نظر گرفته می‌شود؛ درحالی‌که افزایش پرپلکسیتی ویکی‌پدیا نسبت به ۷.۳۳۸۸۶۸ میزان افت عملکرد برون‌دامنه‌ای را نشان می‌دهد.

با وجود نزدیکی دو مقدار، مقایسه اصلی روش‌های تنظیم دقیق نباید صرفاً بر اختلاف مستقیم اخبار و ویکی‌پدیا استوار باشد. مبنای دقیق‌تر آن است که عملکرد هر مدل تنظیم‌شده روی هر مجموعه با مقدار پایه همان مجموعه مقایسه شود:

$$\Delta PPL_{News} = PPL_{News}^{Fine-tuned} - 7.709439$$

$$\Delta PPL_{Wiki} = PPL_{Wiki}^{Fine-tuned} - 7.338868$$

در این چارچوب، منفی‌بودن تغییر پرپلکسیتی اخبار نشان‌دهنده بهبود مدل در دامنه هدف است، درحالی‌که مثبت‌بودن تغییر پرپلکسیتی ویکی‌پدیا نشان‌دهنده افت عملکرد برون‌دامنه‌ای خواهد بود. این شیوه مقایسه اثر تفاوت‌های ذاتی میان دو مجموعه را کاهش می‌دهد و امکان تحلیل دقیق‌تر مصالحه میان سازگاری با اخبار و حفظ دانش عمومی را فراهم می‌کند.

همچنین باید توجه داشت که پرپلکسیتی فقط کیفیت مدل در پیش‌بینی توکن بعدی را اندازه‌گیری می‌کند. مقدار نزدیک پرپلکسیتی در دو دامنه الزاماً به معنای برابری دقت factual، کیفیت استدلال، انسجام متن تولیدشده یا میزان توهم مدل نیست. این جنبه‌ها در ارزیابی کیفی تولید متن بررسی خواهند شد. بااین‌حال، برای سنجش کمی اثر تنظیم دقیق و مقایسه مدل‌ها تحت یک Tokenizer، طول توالی و روش تجمیع یکسان، پرپلکسیتی یک معیار مناسب و قابل‌مقایسه فراهم می‌کند.

در مجموع، مدل پایه روی اخبار فارسی و ویکی‌پدیا به‌ترتیب پرپلکسیتی‌های ۷.۷۰۹۴۳۹ و ۷.۳۳۸۸۶۸ را ثبت کرد. پرپلکسیتی ویکی‌پدیا حدود ۴.۸۱ درصد کمتر از اخبار بود؛ بنابراین مدل پایه در اجرای نهایی اندکی عملکرد بهتری روی داده‌های برون‌دامنه‌ای ویکی‌پدیا نشان داد. بااین‌حال، این تفاوت باید با احتیاط تفسیر شود، زیرا دو مجموعه از نظر سبک، موضوع، ترکیب زبانی و توزیع توکن‌ها متفاوت‌اند. این دو مقدار در ادامه به‌عنوان خطوط مبنای مستقل برای ارزیابی یادگیری اخبار و میزان حفظ عملکرد عمومی در روش‌های LoRA و Adapter استفاده می‌شوند.

۱-۴- نکات پیاده‌سازی و ملاحظات عملی

در این بخش، دو قطعه‌کد اصلی که مستقیماً بر صحت آماده‌سازی ورودی‌ها و محاسبه پرپلکسیتی اثر دارند، بررسی می‌شوند. نخست، سازوکار ساخت Dataset و Batchهای مدل زبان علّی تشریح می‌شود و سپس پیاده‌سازی دستی پرپلکسیتی ارائه خواهد شد. هدف این بخش، توضیح خط‌به‌خط برنامه نیست؛ بلکه تصمیم‌های پیاده‌سازی مؤثر بر مصرف حافظه، حذف Padding و قابلیت مقایسه نتایج بیان می‌شوند.

۱-۴-۱- توکن‌سازی، Padding پویا و ساخت Batchها

آماده‌سازی داده‌ها برای آموزش و ارزیابی با استفاده از کلاس‌های TextDataset و CausalLMCollator و تابع make_loader انجام شد. این قطعه‌کد وظیفه دارد متن‌های خام را به ورودی‌های سازگار با مدل تبدیل کرده و Batchهای موردنیاز حلقه آموزش را با ساختاری یکسان تولید کند.

```python
class TextDataset(Dataset):
    def __init__(self, texts):
        self.texts = texts

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, index):
        return self.texts[index]

class CausalLMCollator:
    def __init__(self, tokenizer, max_length):
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __call__(self, batch_texts):
        batch = self.tokenizer(
            batch_texts,
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=self.max_length,
            add_special_tokens=True,
        )

        labels = batch["input_ids"].clone()
        labels[batch["attention_mask"] == 0] = -100

        batch["labels"] = labels
        return batch

def make_loader(texts, batch_size, shuffle):
    return DataLoader(
        TextDataset(texts),
        batch_size=batch_size,
        shuffle=shuffle,
        collate_fn=collator,
        num_workers=0,
        pin_memory=DEVICE.type == "cuda",
    )
```

*شکل ۱–۱: پیاده‌سازی کلاس‌های **TextDataset** و **CausalLMCollator** و تابع **make_loader** برای ساخت **Batch**های مدل زبان علّی*

کلاس TextDataset یک واسط ساده میان فهرست متن‌ها و DataLoader فراهم می‌کند و هر نمونه متنی را بر اساس اندیس در اختیار Collator قرار می‌دهد. عملیات توکن‌سازی در سطح Batch و داخل CausalLMCollator انجام می‌شود. در این مرحله، متن‌ها با طول حداکثر ۲۵۶ توکن پردازش شده و توالی‌های بلندتر در همین طول برش داده می‌شوند.

Padding به‌صورت پویا اعمال می‌شود؛ بنابراین طول هر Batch بر اساس بلندترین نمونه همان Batch تعیین می‌شود و تمام نمونه‌ها الزاماً تا ۲۵۶ توکن تکمیل نمی‌شوند. این تصمیم، تعداد توکن‌های Padding و محاسبات غیرضروری را کاهش می‌دهد و با توجه به محدودیت حافظه GPU، از Padding ثابت مناسب‌تر است.

برای آموزش مدل زبان علّی، برچسب‌ها از روی input_ids ساخته می‌شوند. سپس موقعیت‌های متناظر با Padding در آرایه برچسب‌ها با مقدار ۱۰۰- جایگزین می‌شوند. این مقدار توسط تابع Cross-Entropy نادیده گرفته می‌شود؛ در نتیجه، توکن‌های Padding در محاسبه زیان و به‌روزرسانی پارامترها مشارکت ندارند. استفاده از attention_mask نیز سبب می‌شود مدل میان توکن‌های واقعی و موقعیت‌های افزوده‌شده برای هم‌اندازه‌سازی توالی‌ها تمایز قائل شود.

تابع make_loader این اجزا را در قالب یک DataLoader ترکیب می‌کند. در مجموعه آموزش، درهم‌ریزی نمونه‌ها فعال است تا ترتیب مشاهده داده‌ها در هر Epoch ثابت نباشد. اندازه ریزBatch آموزش برابر ۴ در نظر گرفته شد و با دو مرحله تجمیع گرادیان، اندازه مؤثر Batch برابر ۸ حفظ شد. برای مجموعه‌های اعتبارسنجی و آزمون، درهم‌ریزی غیرفعال بود. گزینه pin_memory نیز در محیط CUDA برای انتقال کارآمدتر داده‌ها میان حافظه اصلی و GPU فعال شد. این طراحی باعث شد خط لوله آماده‌سازی داده برای مدل پایه، LoRA و Adapter یکسان باقی بماند و اختلاف نتایج ناشی از تفاوت روش تنظیم دقیق باشد، نه تفاوت در پیش‌پردازش ورودی.

۱-۴-۲- پیاده‌سازی دستی محاسبه پرپلکسیتی

محاسبه پرپلکسیتی در تابع evaluate_perplexity و مستقل از Loss آماده مدل پیاده‌سازی شد. این تابع برای مدل پایه، LoRA و Adapter بدون تغییر مورد استفاده قرار گرفت تا تمام مقادیر پرپلکسیتی بر اساس یک روش محاسباتی مشترک به دست آیند. این پیاده‌سازی همچنین شرط صورت تمرین مبنی بر محاسبه دستی پرپلکسیتی و حذف توکن‌های Padding از میانگین زیان را برآورده می‌کند.

```python
@torch.inference_mode()
def evaluate_perplexity(
    model, tokenizer, texts, device, description,
    max_length=MAX_LENGTH, batch_size=EVAL_BATCH_SIZE, token_chunk_size=32,
):
    """
    Manually calculate token-weighted perplexity using shifted logits, shifted labels, cross-entropy
    ,and exponential.
    Padding tokens are excluded with ignore_index=-100.
    """

    model.eval()
    model.config.use_cache = False
    total_nll, total_tokens = 0.0, 0
    progress = tqdm(range(0, len(texts), batch_size), desc=description)

    for start in progress:
        batch_texts = texts[start:start + batch_size]
        encodings = tokenizer(
            batch_texts,
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=max_length,
            add_special_tokens=True,
        )

        input_ids = encodings["input_ids"].to(device, non_blocking=True)
        attention_mask = encodings["attention_mask"].to(device, non_blocking=True)
        labels = input_ids.clone()

        # Exclude padding tokens from loss.
        labels[attention_mask == 0] = -100

        with autocast_context():
            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                use_cache=False,
            )

        # Causal language-model shift:
        # logits at position t predict token t + 1.
        shift_logits = outputs.logits[:, :-1, :]
        shift_labels = labels[:, 1:]
        sequence_length = shift_labels.size(1)

        # Calculate cross-entropy in smaller sequence chunks to avoid
        # converting the complete logits tensor to FP32.
        for token_start in range(0, sequence_length, token_chunk_size):
            token_end = min(token_start + token_chunk_size, sequence_length)
            logits_chunk = shift_logits[:, token_start:token_end, :]
            labels_chunk = shift_labels[:, token_start:token_end]
            valid_token_count = labels_chunk.ne(-100).sum().item()

            if valid_token_count == 0:
                continue

            # Cross-entropy expects vocabulary dimension at index 1:
            # [batch, vocabulary, sequence]
            chunk_nll = F.cross_entropy(
                logits_chunk.transpose(1, 2).float(),
                labels_chunk,
                ignore_index=-100,
                reduction="sum",
            )

            total_nll += chunk_nll.detach().item()
            total_tokens += valid_token_count
            del logits_chunk, labels_chunk, chunk_nll

        del (
            outputs,
            shift_logits,
            shift_labels,
            encodings,
            input_ids,
            attention_mask,
            labels,
        )

    if total_tokens == 0:
        raise RuntimeError(
            "No valid tokens were found for perplexity calculation."
        )

    mean_loss = total_nll / total_tokens
    perplexity = math.exp(mean_loss)

    result = {
        "mean_loss": mean_loss,
        "perplexity": perplexity,
        "tokens": int(total_tokens),
    }

    print(
        f"{description} | mean loss: {mean_loss:.6f} | "
        f"perplexity: {perplexity:.6f} | tokens: {total_tokens:,}"
    )

    return result
```

*شکل ۱–۲: پیاده‌سازی تابع **evaluate_perplexity** برای محاسبه دستی و توکن‌وزن‌دار پرپلکسیتی*

در ابتدای ارزیابی، مدل در حالت eval قرار می‌گیرد و محاسبات در بستر torch.inference_mode() انجام می‌شوند. در این وضعیت، گرادیان‌ها و گراف پس‌انتشار ذخیره نمی‌شوند. همچنین هنگام فراخوانی مدل، گزینه use_cache=False تنظیم شد تا

حافظه مربوط به Key/Value Cache در ارزیابی نگهداری نشود. ورودی‌ها به‌صورت Batch پردازش می‌شوند و مدل برای تمام موقعیت‌های توالی، بردار Logit روی واژگان تولید می‌کند.

برای تبدیل مسئله به پیش‌بینی توکن بعدی، آخرین موقعیت Logitها و نخستین موقعیت برچسب‌ها حذف می‌شوند. پس از این جابه‌جایی، خروجی موقعیت t با توکن واقعی موقعیت t+1 مقایسه می‌شود. از آنجا که موقعیت‌های Padding پیش‌تر در آرایه labels با مقدار -۱۰۰ مشخص شده بودند، پس از جابه‌جایی نیز این علامت در shifted_labels حفظ شد. بنابراین، حذف Padding در تابع Cross-Entropy با استفاده از ignore_index=-100 انجام گرفت و نیازی به حذف مستقیم Logitها با ماسک بولی نبود.

به دلیل اندازه واژگان ۲۶۲۴۰۰ تایی مدل، تبدیل یک‌باره تمام Logitهای یک Batch به float32 می‌توانست حافظه قابل‌توجهی مصرف کند. به همین دلیل، پس از مسطح‌کردن ابعاد Batch و طول توالی، موقعیت‌ها به بلوک‌های کوچک‌تر تقسیم شدند. در هر مرحله فقط Logitهای همان بلوک به float32 تبدیل شدند و تابع F.cross_entropy با گزینه‌های ignore_index=-100 و reduction="sum" روی آن‌ها اجرا شد. مجموع زیان بلوک‌ها به مجموع سراسری افزوده شد و تعداد توکن‌های معتبر نیز با شمارش برچسب‌هایی به دست آمد که مقدارشان -۱۰۰ نبود. پس از پردازش کامل مجموعه ارزیابی، مجموع زیان بر تعداد کل توکن‌های معتبر تقسیم شد و تابع نمایی تنها یک مرتبه بر میانگین نهایی اعمال گردید:

$$\mathcal{L}_{mean} = \frac{\sum_{k=1}^{K} NLL_k}{\sum_{k=1}^{K} N_k}$$

$$PPL = \exp(\mathcal{L}_{mean})$$

این شیوه برخلاف میانگین‌گیری ساده از Loss یا Perplexity Batchها، تفاوت تعداد توکن‌های معتبر در Batchهای مختلف را در نظر می‌گیرد. در نتیجه، هر توکن واقعی سهم یکسانی در معیار نهایی دارد و Batchهای کوتاه و بلند به‌طور مصنوعی وزن برابر دریافت نمی‌کنند. منطق نوت‌بوک نیز بر تجمیع Cross-Entropy جابه‌جاشده روی همه اهداف غیر Padding و اعمال یک‌باره تابع نمایی استوار است.

یکی از ملاحظات عملی این پیاده‌سازی، امکان تغییر اندازه Batch ارزیابی بدون تغییر تعریف معیار است. اندازه Batch عمومی ارزیابی برابر ۴ بود، اما برای ارزیابی کامل مجموعه اعتبارسنجی مدل Adapter، اندازه Batch برابر ۲ در نظر گرفته شد تا احتمال بروز خطای OOM کاهش یابد. از آنجا که مجموع Negative Log-Likelihood و تعداد توکن‌های معتبر در سطح کل مجموعه تجمیع می‌شوند، این تغییر تنها بر زمان اجرا و حافظه لحظه‌ای اثر دارد و مقدار نظری پرپلکسیتی را تغییر نمی‌دهد. محاسبه بلوکی Cross-Entropy نیز همین هدف را دنبال می‌کند و بدون تغییر تعریف ریاضی معیار، Peak Memory مرحله ارزیابی را کاهش می‌دهد.

برای اطمینان از صحت جابه‌جایی Logitها و برچسب‌ها، یک آزمون کنترلی کوتاه نیز در نوت‌بوک در نظر گرفته شد که Loss داخلی مدل را با Cross-Entropy دستی روی یک ورودی کوتاه مقایسه می‌کرد. این آزمون بخشی از محاسبه نهایی پرپلکسیتی نبود و صرفاً برای بررسی صحت هم‌ترازی علّی و حذف Padding طراحی شده بود. به دلیل اندازه بزرگ فضای واژگان، این بررسی با Batch و طول توالی کوچک اجرا شد تا حافظه اضافی محدودی مصرف کند.

۱-۵- جمع‌بندی فصل

در این فصل، خط پایه‌ای منسجم برای ارزیابی اثر روش‌های تنظیم دقیق ایجاد شد. تفکیک روشن داده‌های خبری و ویکی‌پدیا سبب شد عملکرد مدل در دامنه هدف و دامنه عمومی به‌صورت مستقل سنجیده شود و از اختلاط داده‌های آموزشی با مجموعه‌های ارزیابی جلوگیری گردد. همچنین، استفاده از یک خط لوله مشترک برای پاک‌سازی، توکن‌سازی، Padding پویا و ساخت Batchها، شرایط یکسانی را برای ارزیابی مدل پایه و مقایسه آن با مدل‌های تنظیم‌شده در فصل‌های بعد فراهم کرد.

پیاده‌سازی دستی پرپلکسیتی به‌گونه‌ای انجام شد که زیان بر اساس تمام توکن‌های معتبر مجموعه محاسبه شود و توکن‌های Padding در نتیجه نهایی اثری نداشته باشند. تجمیع زیان در سطح توکن، به‌جای میانگین‌گیری ساده میان Batchها، باعث شد مقدار نظری معیار به نحوه تقسیم نمونه‌ها میان Batchها وابسته نباشد. این ویژگی از نظر عملی اهمیت دارد؛ زیرا در مراحل بعد، اندازه Batch برخی ارزیابی‌ها برای مدیریت محدودیت حافظه کاهش یافت، بدون آنکه تعریف معیار یا اعتبار مقایسه تغییر کند.

نتایج مدل پایه نشان داد که پیش از تنظیم دقیق، مدل روی مجموعه ویکی‌پدیا اندکی عملکرد بهتری نسبت به اخبار فارسی داشته است؛ به‌طوری‌که پرپلکسیتی ویکی‌پدیا حدود ۴.۸۱ درصد کمتر از اخبار بود. بااین‌حال، مقادیر دو مجموعه همچنان در یک بازه نسبتاً نزدیک قرار داشتند و باید با توجه به تفاوت‌های موضوعی، سبکی و زبانی آن‌ها تفسیر شوند. بنابراین، مقادیر به‌دست‌آمده دو خط مبنای مستقل و مناسب برای سنجش تغییرات بعدی فراهم می‌کنند؛ به‌گونه‌ای که کاهش پرپلکسیتی اخبار می‌تواند به‌عنوان میزان یادگیری دامنه هدف و افزایش احتمالی پرپلکسیتی ویکی‌پدیا به‌عنوان هزینه این سازگارسازی از منظر حفظ عملکرد عمومی تفسیر شود. در مجموع، این فصل مبنای فنی و تحلیلی لازم برای ارزیابی منصفانه روش‌های LoRA و Adapter را فراهم ساخت.

فصل دوم

پیاده‌سازی و آموزش مدل با روش LoRA

در این فصل، مدل پایه با استفاده از روش سازگارسازی کم‌رتبه یا LoRA و کتابخانه PEFT برای دامنه اخبار فارسی تنظیم دقیق می‌شود. ابتدا مبانی روش و نحوه اعمال آن بر نگاشت‌های سازوکار توجه تشریح شده و سپس فرایند آموزش و ارزیابی مدل تنظیم‌شده بررسی خواهد شد. مطابق صورت تمرین، LoRA فقط روی نگاشت‌های q_proj، k_proj، v_proj و o_proj اعمال شده و هیچ‌یک از وزن‌ها یا بایاس‌های مدل اصلی در فرایند آموزش تغییر نمی‌کنند.

۲-۱- مبانی و پیکربندی روش LoRA

روش LoRA یکی از رویکردهای تنظیم دقیق پارامترکارا است که به‌جای به‌روزرسانی تمام وزن‌های یک مدل زبانی بزرگ، تعداد محدودی پارامتر جدید به بخش‌های مشخصی از معماری اضافه می‌کند. در این بخش، ابتدا سازوکار ریاضی LoRA توضیح داده می‌شود؛ سپس محل اعمال آن در مدل Gemma 3n و ابرپارامترهای مورد استفاده معرفی شده و در پایان، نحوه انجماد مدل پایه و کنترل پارامترهای قابل‌آموزش تشریح خواهد شد.

۲-۱-۱- سازوکار تنظیم کم‌رتبه

در تنظیم دقیق کامل، تمام یا بخش بزرگی از وزن‌های مدل پیش‌آموزش‌دیده در جریان آموزش تغییر می‌کنند. این روش اگرچه انعطاف‌پذیری بالایی دارد، برای مدل‌های زبانی بزرگ مستلزم نگهداری گرادیان و حالت‌های بهینه‌ساز برای میلیاردها پارامتر است و در نتیجه به حافظه و توان محاسباتی زیادی نیاز دارد. افزون بر آن، ذخیره یک نسخه مستقل از مدل برای هر وظیفه یا دامنه جدید، هزینه ذخیره‌سازی قابل‌توجهی ایجاد می‌کند.

LoRA بر این فرض استوار است که تغییر لازم برای سازگارکردن یک مدل بزرگ با یک وظیفه جدید را می‌توان در فضایی با ابعاد بسیار کمتر از فضای کامل وزن‌های مدل نمایش داد. بر این اساس، ماتریس وزن اصلی ثابت نگه داشته می‌شود و به‌جای به‌روزرسانی مستقیم آن، یک تصحیح کم‌رتبه به خروجی لایه افزوده می‌شود.

فرض شود ماتریس وزن یک لایه خطی پیش‌آموزش‌دیده به‌صورت زیر تعریف شده باشد:

$$W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$$

در تنظیم دقیق کامل، یک ماتریس تغییر با همان ابعاد، یعنی ΔW، آموخته می‌شود:

$$W = W_0 + \Delta W$$

در روش LoRA، این ماتریس تغییر به حاصل‌ضرب دو ماتریس کوچک‌تر تجزیه می‌شود:

$A \in \mathbb{R}^{r \times d_{in}}$

$B \in \mathbb{R}^{d_{out} \times r}$

$\Delta W = \frac{\alpha}{r} BA$

بنابراین، خروجی لایه برای بردار ورودی x به‌صورت زیر محاسبه می‌شود:

$h = W_0 x + \frac{\alpha}{r} BAx$

عبارت نخست، همان تبدیل اصلی مدل پیش‌آموزش‌دیده است و بدون تغییر باقی می‌ماند. عبارت دوم، تصحیحی است که در جریان تنظیم دقیق از داده‌های جدید آموخته می‌شود. به این ترتیب، LoRA مسیر محاسباتی اصلی مدل را حفظ کرده و تنها یک مؤلفه قابل‌آموزش به آن می‌افزاید.

پارامتر r رتبه ماتریس تصحیح و تعیین‌کننده ظرفیت LoRA است. هرچه رتبه بزرگ‌تر باشد، روش توانایی نمایش تغییرات پیچیده‌تری را خواهد داشت، اما تعداد پارامترهای قابل‌آموزش و هزینه محاسباتی نیز افزایش می‌یابد. شرط اصلی کارایی این روش آن است که:

$r \ll \min(d_{in}, d_{out})$

در تنظیم دقیق کامل، تعداد پارامترهای قابل‌آموزش یک لایه خطی برابر $d_{in} d_{out}$ است:

درحالی‌که در LoRA این تعداد به رابطه زیر کاهش می‌یابد:

$N_{LoRA} = r d_{in} + r d_{out} = r(d_{in} + d_{out})$

برای ابعاد بزرگ لایه و رتبه کوچک، این کاهش می‌تواند قابل‌توجه باشد. در نتیجه، تنها پارامترهای ماتریس‌های A و B نیازمند گرادیان و به‌روزرسانی توسط بهینه‌ساز هستند و ماتریس W_0 ثابت باقی می‌ماند.

پارامتر α شدت اثر به‌روزرسانی کم‌رتبه را کنترل می‌کند. ضریب $\frac{\alpha}{r}$ باعث می‌شود مقیاس خروجی LoRA به‌طور مستقیم به انتخاب رتبه وابسته نباشد. در این آزمایش، با توجه به مقادیر $r = 16$ و $\alpha = 32$، ضریب مقیاس برابر است با:

$\frac{\alpha}{r} = \frac{32}{16} = 2$

بنابراین، خروجی حاصل از دو ماتریس کم‌رتبه با ضریب ۲ به خروجی لایه اصلی افزوده می‌شود.

مزیت مهم این ساختار آن است که برای هر دامنه یا وظیفه جدید، فقط وزن‌های کم‌حجم LoRA ذخیره می‌شوند. برای استفاده مجدد، مدل پایه ثابت بارگذاری شده و وزن‌های LoRA روی آن اعمال می‌شوند. ازاین‌رو نیازی به ذخیره نسخه‌ای کامل از میلیاردها پارامتر مدل برای هر تنظیم دقیق وجود ندارد.

۲-۱-۲- لایه‌های هدف و ابرپارامترهای LoRA

در معماری Transformer، سازوکار توجه با ساخت نگاشت‌های Query، Key و Value از بازنمایی ورودی عمل می‌کند. اگر ماتریس بازنمایی ورودی با X نشان داده شود، این نگاشت‌ها به‌صورت زیر محاسبه می‌شوند:

$$Q = XW_q$$

$$K = XW_k$$

$$V = XW_v$$

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

بردارهای Query مشخص می‌کنند هر توکن در جست‌وجوی چه اطلاعاتی است، بردارهای Key ویژگی‌های قابل‌مقایسه توکن‌ها را نمایش می‌دهند و بردارهای Value محتوایی را در بر می‌گیرند که پس از محاسبه وزن‌های توجه ترکیب می‌شود. خروجی نهایی توجه نیز با نگاشت W_o به فضای بازنمایی مدل بازگردانده می‌شود:

$$H_{attn} = Attention(Q, K, V)W_o$$

در پیاده‌سازی انجام‌شده، LoRA منحصراً به چهار نگاشت زیر متصل شد:

- q_proj برای تولید Query؛
- k_proj برای تولید Key؛
- v_proj برای تولید Value؛
- o_proj برای نگاشت خروجی Attention.

اعمال LoRA بر این نگاشت‌ها به مدل اجازه می‌دهد نحوه ایجاد ارتباط میان توکن‌ها، انتخاب اطلاعات زمینه‌ای و ترکیب خروجی Attention را با الگوهای موجود در متون خبری سازگار کند؛ بدون آنکه وزن‌های اصلی این نگاشت‌ها تغییر کنند. هیچ لایه‌ای در بخش MLP، Embedding، سر خروجی یا سایر اجزای مدل به‌عنوان هدف LoRA انتخاب نشد.

نام ماژول‌های هدف مستقیماً از معماری بارگذاری‌شده مدل استخراج شد. در این فرایند فقط ماژول‌های خطی موجود در مسیر language_model و داخل self_attn پذیرفته شدند که نام آن‌ها به یکی از چهار پسوند مجاز ختم می‌شد. این روش از اتصال ناخواسته LoRA به ماژول‌هایی با نام مشابه در بخش‌های دیگر مدل جلوگیری کرد.

در خروجی نوت‌بوک، تعداد ماژول‌های هدف شناسایی‌شده برابر ۱۰۰ گزارش شد و این مقدار با تعداد مورد انتظار براساس ساختار واقعی لایه‌های مدل مطابقت داشت. این کنترل از آن جهت اهمیت دارد که در معماری Gemma 3n برخی نگاشت‌های Key و Value میان لایه‌ها به‌صورت اشتراکی سازمان‌دهی شده‌اند؛ بنابراین به‌جای فرض‌کردن وجود چهار ماژول مستقل در همه لایه‌ها، فقط نمونه‌های واقعی این چهار نگاشت در معماری هدف قرار گرفتند.

پیکربندی LoRA با استفاده از کلاس LoraConfig در کتابخانه PEFT انجام شد. مقادیر اصلی این پیکربندی در جدول ۲-۱ ارائه شده‌اند.

جدول ۲-۱: ابرپارامترها و تنظیمات پیکربندی ***LoRA***

پارامتر	مقدار استفاده‌شده	نقش
نوع وظیفه	CAUSAL_LM	تنظیم مدل برای پیش‌بینی خودرگرسیو توکن بعدی
رتبه (r)	16	تعیین ظرفیت ماتریس‌های کم‌رتبه
ضریب (α)	32	تنظیم مقیاس اثر به‌روزرسانی LoRA
ضریب (α/r)	2	مقیاس نهایی مؤلفه کم‌رتبه
لایه‌های هدف	$q_{proj}, k_{proj}, v_{proj}, o_{proj}$	اعمال LoRA فقط بر نگاشت‌های Attention
تعداد ماژول‌های هدف	۱۰۰	تعداد نگاشت‌های واقعی شناسایی‌شده در مدل
مقدار Dropout	0.0	عدم اعمال Dropout در مسیر LoRA
تنظیم Bias	none	جلوگیری از آموزش پارامترهای بایاس
کتابخانه پیاده‌سازی	PEFT	ایجاد و مدیریت ماژول‌های LoRA

مقدار رتبه ۱۶ و آلفای ۳۲ مطابق ابرپارامترهای اجباری صورت تمرین انتخاب شدند. همچنین bias="none" تعیین شد تا PEFT هیچ پارامتر بایاسی را قابل‌آموزش نکند. بررسی نهایی پارامترهای دارای requires_grad=True نیز نشان داد که هیچ Bias قابل‌آموزشی در مدل وجود ندارد. مقدار Dropout نیز برابر صفر قرار گرفت؛ در نتیجه، خروجی مسیر LoRA در زمان آموزش به‌صورت تصادفی حذف نشد و تمام ظرفیت کم‌رتبه در هر مرحله مورد استفاده قرار گرفت.

۲-۱-۳- انجماد مدل پایه و پارامترهای قابل‌آموزش

پیش از اعمال LoRA، تمام پارامترهای مدل پایه در وضعیت ثابت قرار گرفتند. در عمل، ویژگی requires_grad برای وزن‌های اصلی غیرفعال شد؛ بنابراین گرادیانی برای آن‌ها محاسبه نشده و بهینه‌ساز نیز قادر به تغییرشان نبود. پس از ایجاد مدل PEFT، تنها پارامترهای جدید مربوط به ماتریس‌های کم‌رتبه LoRA قابل‌آموزش باقی ماندند.

این وضعیت را می‌توان به‌صورت زیر بیان کرد:

$$\nabla_{W_0}\mathcal{L} = 0$$

$$\nabla_{A}\mathcal{L} \neq 0$$

$$\nabla_{B}\mathcal{L} \neq 0$$

بنابراین، دانش ذخیره‌شده در وزن‌های اصلی مدل مستقیماً بازنویسی نمی‌شود و سازگارسازی مدل با اخبار از طریق ماتریس‌های افزوده‌شده انجام می‌گیرد. البته ثابت‌بودن وزن‌های اصلی به این معنا نیست که رفتار مدل بدون تغییر باقی می‌ماند؛

زیرا خروجی کم‌رتبه به خروجی لایه اصلی اضافه شده و می‌تواند نحوه پردازش زمینه و توزیع احتمال توکن‌های بعدی را تغییر دهد.

برای اطمینان از صحت پیکربندی، پس از ایجاد مدل LoRA چند کنترل اجرا شد. ابتدا تعداد ماژول‌های هدف یافت‌شده با تعداد مورد انتظار معماری مقایسه گردید. سپس تمام پارامترهای قابل‌آموزش بررسی شدند تا هیچ پارامتر Bias در میان آن‌ها وجود نداشته باشد. خروجی این کنترل، فهرستی خالی برای بایاس‌های قابل‌آموزش نشان داد:

Trainable bias parameters: []

همچنین بررسی نام پارامترها نشان داد که تمام پارامترهای دارای گرادیان به ماژول‌های LoRA متصل به چهار نگاشت مجاز تعلق دارند. به این ترتیب، احتمال آموزش ناخواسته وزن‌های مدل اصلی یا هدف‌گیری لایه‌ای خارج از محدوده تعیین‌شده در تمرین برطرف شد.

پس از ایجاد مدل PEFT و تأیید پارامترهای قابل‌آموزش، وزن‌های جدید LoRA به float32 تبدیل شدند، درحالی‌که پارامترهای ثابت مدل پایه در نوع داده float16 باقی ماندند. این تصمیم باعث می‌شود به‌روزرسانی تعداد محدود پارامترهای LoRA با دقت عددی بیشتری انجام شود، بدون آنکه تمام وزن‌های چندمیلیاردپارامتری مدل پایه به float32 تبدیل شوند. در نتیجه، افزایش حافظه فقط به پارامترهای کم‌حجم قابل‌آموزش محدود می‌شود.

در اجرای نهایی، تعداد پارامترهای قابل‌آموزش LoRA برابر با ۵,۵۷۰,۵۶۰ به دست آمد. این تعداد تنها حدود ۰.۱۰۲۸۰۷ درصد از پارامترهای مدل پایه را تشکیل می‌دهد. بنابراین، حدود ۹۹.۸۹۷۱۹۳ درصد از پارامترهای مدل پایه ثابت باقی ماندند و سازگارسازی مدل فقط با آموزش بخش بسیار کوچکی از پارامترها انجام شد.

کاهش تعداد پارامترهای قابل‌آموزش، هزینه نگهداری گرادیان و حالت‌های بهینه‌ساز را نسبت به تنظیم دقیق کامل کاهش می‌دهد. بااین‌حال، این کاهش به معنای حذف هزینه عبور پیش‌رو و پس‌انتشار از مدل پایه نیست؛ زیرا برای محاسبه خروجی و انتقال گرادیان به ماژول‌های LoRA همچنان باید فعال‌سازی‌های لایه‌های اصلی مدل پردازش شوند. در نتیجه، LoRA حافظه موردنیاز برای پارامترهای قابل‌آموزش را به‌شدت کاهش می‌دهد، اما مصرف حافظه کلی آموزش همچنان به طول توالی، اندازه Batch، فعال‌سازی‌های میانی و ابعاد مدل پایه وابسته است.

در مجموع، پیکربندی انجام‌شده توانست با ثابت نگه‌داشتن تمام وزن‌ها و بایاس‌های مدل اصلی، یک مسیر کم‌رتبه قابل‌آموزش در نگاشت‌های Attention ایجاد کند. این ساختار، مبنای تنظیم دقیق مدل روی مجموعه آموزش اخبار فارسی را فراهم کرد که فرایند آن در بخش بعد تشریح می‌شود.

۲-۲- فرایند تنظیم دقیق روی اخبار فارسی

پس از پیکربندی مدل با روش LoRA و اطمینان از ثابت‌بودن وزن‌های مدل پایه، فرایند تنظیم دقیق با استفاده از حلقه آموزشی استاندارد PyTorch اجرا شد. در این مرحله، تنها پارامترهای کم‌رتبه افزوده‌شده به نگاشت‌های Attention به‌روزرسانی شدند و مدل منحصراً با بخش آموزش اخبار فارسی سازگار شد. داده‌های ویکی‌پدیا و مجموعه‌های آزمون در هیچ مرحله‌ای از

محاسبه گرادیان یا به‌روزرسانی پارامترهای مدل دخالت نداشتند. بخش اعتبارسنجی اخبار نیز صرفاً برای پایش روند آموزش استفاده شد و هیچ گرادیانی بر اساس آن محاسبه نشد.

تمام تنظیمات اجباری شامل نرخ یادگیری 2×10^{-4}، سه دور آموزش، رتبه ۱۶ و آلفای ۳۲ مطابق صورت تمرین ثابت نگه داشته شدند. تنظیماتی مانند اندازه Batch و تجمیع گرادیان نیز بر اساس ظرفیت حافظه GPU تعیین شدند.

۲-۲-۱- داده‌ها و شرایط آموزش

آموزش LoRA فقط روی ۱۰۰۰۰ نمونه بخش Train مجموعه اخبار فارسی انجام شد. استفاده‌نکردن از داده‌های ویکی‌پدیا در این مرحله یک تصمیم اصلی در طراحی آزمایش بود؛ زیرا هدف، سازگارکردن مدل با ساختار و لحن متون خبری و سپس بررسی اثر این سازگارسازی بر توانایی عمومی مدل بود. درصورتی‌که ویکی‌پدیا نیز در آموزش وارد می‌شد، امکان سنجش مستقل میزان حفظ دانش برون‌دامنه‌ای مدل کاهش می‌یافت.

متن‌ها با حداکثر طول ۲۵۶ توکن پردازش شدند و Padding به‌صورت پویا در سطح هر Batch انجام گرفت. برای موقعیت‌های Padding، مقدار برچسب برابر ۱۰۰- قرار داده شد تا این موقعیت‌ها در محاسبه زیان مدل زبان علّی مشارکت نکنند. ترتیب نمونه‌های آموزشی نیز در هر Epoch به‌صورت تصادفی درهم‌ریزی شد. بخش اعتبارسنجی اخبار بدون درهم‌ریزی و بدون محاسبه گرادیان پردازش گردید.

شرایط اصلی آموزش در جدول ۲-۲ ارائه شده‌اند.

جدول ۲-۲: پیکربندی فرایند آموزش مدل ***LoRA***

تنظیم آموزشی	**مقدار استفاده‌شده**
مجموعه آموزشی	بخش آموزش اخبار فارسی
تعداد نمونه‌های آموزشی	۱۰۰۰۰
مجموعه اعتبارسنجی	بخش dev اخبار فارسی
تعداد نمونه‌های اعتبارسنجی	۱۰۰۰
تعداد دورهای آموزش	۳
نرخ یادگیری	2×10^{-4}
طول حداکثر توالی	۲۵۶ توکن
اندازه ریز Batch	۴
مراحل تجمیع گرادیان	۲
اندازه مؤثر Batch	۸
تعداد ریز Batch در هر دور	۲۵۰۰

تعداد به‌روزرسانی بهینه‌ساز در هر دور	۱۲۵۰
تعداد کل به‌روزرسانی‌ها	۳۷۵۰
فاصله Mini-validation	هر ۲۰ ریزBatch آموزشی
اندازه Batch در Mini-validation	۲
اندازه Batch در Full validation	۴
نوع داده محاسباتی	float16
نوع داده پارامترهای LoRA	float32
بهینه‌ساز	AdamW
Gradient Checkpointing	فعال
کوانتیزه‌سازی چهار یا هشت‌بیتی	استفاده نشد

با اندازه ریزBatch برابر ۴، مجموعه ۱۰۰۰۰ نمونه‌ای آموزش در هر دور به ۲۵۰۰ ریزBatch تقسیم شد. از آنجا که گرادیان‌ها طی دو ریزBatch متوالی تجمیع می‌شدند، بهینه‌ساز پس از پردازش مؤثر هر هشت نمونه یک‌بار پارامترها را به‌روزرسانی می‌کرد:

$$B_{effective} = B_{micro} \times N_{accumulation} = 4 \times 2 = 8$$

تعداد مراحل به‌روزرسانی بهینه‌ساز در هر دور نیز برابر بود با:

$$N_{updates/epoch} = \frac{10000}{4 \times 2} = 1250$$

بنابراین، طی سه Epoch در مجموع ۳۷۵۰ به‌روزرسانی روی پارامترهای LoRA انجام شد. تجمیع گرادیان امکان حفظ Batch مؤثر برابر ۸ را فراهم کرد، درحالی‌که در هر عبور پیش‌رو فقط چهار نمونه به‌صورت هم‌زمان پردازش می‌شدند.

فعال‌سازی Gradient Checkpointing برای کاهش حافظه موردنیاز فعال‌سازی‌های میانی مدل انجام شد. در این روش، همه خروجی‌های میانی عبور پیش‌رو در حافظه نگهداری نمی‌شوند و بخشی از آن‌ها هنگام پس‌انتشار مجدداً محاسبه می‌شوند. این مبادله مصرف حافظه را کاهش می‌دهد، اما می‌تواند زمان آموزش را افزایش دهد. مقدار دقیق Peak VRAM و زمان آموزش LoRA در فصل تحلیل نتایج گزارش می‌شود.

۲-۲-۲- بهینه‌سازی و ثبت روند آموزش

در ابتدای آموزش، مدل در حالت train قرار گرفت و قابلیت Cache غیرفعال شد؛ زیرا استفاده هم‌زمان از Cache با آموزش و Gradient Checkpointing می‌تواند باعث افزایش مصرف حافظه یا ناسازگاری در محاسبه گرادیان شود. سپس بهینه‌ساز AdamW فقط با پارامترهایی ساخته شد که ویژگی requires_grad=True داشتند. به این ترتیب، وزن‌های اصلی مدل وارد بهینه‌ساز نشدند و فقط ماتریس‌های LoRA امکان به‌روزرسانی داشتند. از آنجا که پارامترهای قابل‌آموزش LoRA پیش‌تر به float32 تبدیل شده

بودند، بهینه‌ساز فقط همین پارامترهای کم‌رتبه را با دقت عددی FP32 به‌روزرسانی می‌کرد و وزن‌های ثابت مدل پایه در float16 باقی می‌ماندند.

برای هر ریزBatch، داده‌ها ابتدا به GPU منتقل شدند و مدل با دریافت input_ids، attention_mask و labels عبور پیش‌رو را اجرا کرد. زیان خروجی مدل، Cross-Entropy مربوط به پیش‌بینی توکن بعدی بود و موقعیت‌های Padding به دلیل داشتن برچسب ۱۰۰- در آن لحاظ نمی‌شدند.

از آنجا که دو ریزBatch برای تشکیل یک Batch مؤثر تجمیع می‌شدند، زیان پیش از اجرای پس‌انتشار بر تعداد مراحل تجمیع گرادیان تقسیم شد:

$$\mathcal{L}_{backward} = \frac{\mathcal{L}_{microbatch}}{N_{accumulation}} = \frac{\mathcal{L}_{microbatch}}{2}$$

این تقسیم باعث می‌شود مجموع گرادیان‌های دو ریزBatch از نظر مقیاس با گرادیان میانگین یک Batch مؤثر هشت‌نمونه‌ای سازگار باشد. پس از پردازش دو ریزBatch، نرم گرادیان‌های پارامترهای قابل‌آموزش محدود شد، بهینه‌ساز یک مرحله به‌روزرسانی انجام داد و سپس گرادیان‌ها برای گروه بعدی پاک شدند.

محدودسازی نرم گرادیان برای جلوگیری از به‌روزرسانی‌های بسیار بزرگ و ناپایداری عددی به کار رفت. این مسئله در آموزش مدل‌های عمیق اهمیت دارد؛ زیرا تغییرات ناگهانی در اندازه گرادیان می‌تواند روند بهینه‌سازی را بی‌ثبات کرده یا موجب ایجاد مقادیر نامعتبر عددی شود.

زیان خام هر ریزBatch پیش از تقسیم بر تعداد مراحل تجمیع ثبت شد. در نتیجه، مقادیر تاریخچه آموزش نمایانگر زیان واقعی مدل بودند و کوچک‌شدن مصنوعی ناشی از Gradient Accumulation وارد گزارش نشد. میانگین زیان هر Epoch نیز از زیان ریزBatchهای همان Epoch محاسبه و همراه با تاریخچه کامل مراحل ذخیره شد. علاوه بر این، شماره ریزBatchهای متناظر با نقاط Mini-validation، زیان کامل Validation و پرپلکسیتی Validation در پایان هر Epoch نیز ثبت گردید.

با توجه به تفاوت طول و دشواری متن‌های خبری و تصادفی‌بودن نمونه‌های هر ریزBatch، انتظار می‌رود منحنی زیان آموزش در سطح ریزBatch دارای نوسان باشد. بنابراین، نوسان نقطه‌ای زیان به‌تنهایی نشانه ناپایداری یا بیش‌برازش نیست. در اجرای نهایی، علاوه بر میانگین زیان Epochها، نقاط Mini-validation در طول آموزش و ارزیابی کامل بخش Validation پس از هر Epoch ثبت شدند. تحلیل هم‌زمان روند Training و Validation امکان بررسی دقیق‌تری از همگرایی و خطر بیش‌برازش فراهم می‌کند که در فصل چهارم و در مقایسه مستقیم با Adapter ارائه خواهد شد.

در طول آموزش، پس از هر ۲۰ ریزBatch، یک Batch کوچک از داده‌های اعتبارسنجی اخبار در حالت eval و بدون محاسبه گرادیان ارزیابی شد. این Mini-validation برای ثبت روند پیوسته‌تر زیان اعتبارسنجی به کار رفت و پس از آن مدل دوباره به حالت train بازگردانده شد. با وجود ۲۵۰۰ ریزBatch در هر Epoch، در هر دور ۱۲۵ نقطه Mini-validation و در مجموع سه دور ۳۷۵ نقطه ثبت شد. همچنین پس از پایان هر Epoch، تابع ارزیابی کامل روی تمام ۱۰۰۰ نمونه بخش اعتبارسنجی اجرا شد و میانگین Cross-Entropy و پرپلکسیتی آن ثبت گردید. در نتیجه، برای LoRA سه مقدار Full-validation در سطح Epoch به دست آمد. هیچ‌یک از این ارزیابی‌ها در محاسبه گرادیان یا به‌روزرسانی پارامترها مشارکت نداشتند.

۲-۲-۳- ذخیره مدل و خروجی‌های آموزش

پس از پایان سه دور آموزش، پارامترهای LoRA با استفاده از سازوکار save_pretrained کتابخانه PEFT ذخیره شدند. در این روش، وزن‌های چندمیلیاردپارامتری مدل پایه دوباره ذخیره نمی‌شوند و فقط ماتریس‌های کم‌رتبه و فایل پیکربندی LoRA نگهداری می‌شوند. برای بازیابی مدل تنظیم‌شده، ابتدا همان نسخه مدل پایه بارگذاری شده و سپس آداپتور LoRA روی آن اعمال می‌شود.

این شیوه ذخیره‌سازی یکی از مزایای عملی تنظیم دقیق پارامترکارا است. مدل پایه می‌تواند میان چند وظیفه مشترک باقی بماند و برای هر دامنه فقط یک مجموعه کوچک از پارامترهای LoRA نگهداری شود. درنتیجه، مدیریت نسخه‌های مختلف مدل و انتقال وزن‌های تنظیم‌شده ساده‌تر از ذخیره یک مدل کامل برای هر آزمایش خواهد بود.

علاوه بر پارامترهای LoRA، تاریخچه آموزش و اطلاعات لازم برای تحلیل‌های بعدی نیز ثبت شدند. این خروجی‌ها شامل زیان ریزBatchها، میانگین زیان هر Epoch، شماره مراحل و زیان‌های Mini-validation، زیان و پرپلکسیتی Full-validation در پایان هر Epoch، زمان خالص آموزش، زمان صرف‌شده برای Validation، Peak VRAM و تعداد پارامترهای قابل‌آموزش بودند. نتایج پرپلکسیتی روی آزمون اخبار و ویکی‌پدیا و خروجی‌های تولید متن نیز پس از آموزش به ساختار نتایج افزوده شدند تا در فصل چهارم برای مقایسه مدل پایه، LoRA و Adapter استفاده شوند.

تفکیک زمان خالص آموزش از زمان ارزیابی اعتبارسنجی باعث شد مقایسه سرعت LoRA و Adapter بر اساس خود فرایند بهینه‌سازی انجام شود و هزینه اضافه‌شده توسط پایش Validation نیز به‌طور جداگانه قابل‌گزارش باشد.

در مجموع، فرایند آموزش LoRA با یک حلقه استاندارد PyTorch و تنها با استفاده از بخش آموزش اخبار فارسی انجام شد. وزن‌های مدل پایه در تمام مراحل ثابت ماندند و ۳۷۵۰ مرحله به‌روزرسانی فقط روی پارامترهای کم‌رتبه اعمال شد. بخش اعتبارسنجی اخبار نیز بدون مشارکت در گرادیان، برای ثبت ۳۷۵ نقطه Mini-validation و سه ارزیابی کامل در سطح Epoch استفاده شد. مدل حاصل پس از ذخیره وزن‌های LoRA برای ارزیابی نهایی درون‌دامنه‌ای روی اخبار و برون‌دامنه‌ای روی ویکی‌پدیا آماده شد که نتایج آن در بخش بعد ارائه می‌شوند.

۲-۳- ارزیابی مدل تنظیم‌شده با LoRA

پس از پایان سه دور تنظیم دقیق، مدل LoRA روی دو مجموعه آزمون اخبار فارسی و ویکی‌پدیا ارزیابی شد. هدف از این ارزیابی، بررسی هم‌زمان میزان سازگاری مدل با دامنه‌ای بود که روی آن آموزش دیده است و سنجش اثر این سازگارسازی بر عملکرد برون‌دامنه‌ای مدل. مطابق صورت تمرین، مقدار پرپلکسیتی مدل تنظیم‌شده باید روی هر دو مجموعه آزمون محاسبه و ثبت شود.

برای حفظ قابلیت مقایسه، ارزیابی LoRA با همان توکن‌ساز، طول حداکثر توالی، Padding پویا و تابع دستی پرپلکسیتی فصل اول انجام شد. در نتیجه، تفاوت میان خروجی مدل پایه و مدل LoRA ناشی از تغییر روش اندازه‌گیری نیست و می‌توان آن را به

اثر تنظیم دقیق روی پارامترهای کم‌رتبه نسبت داد. منطق نوت‌بوک نیز بر محاسبه Cross-Entropy جابه‌جاشده، حذف اهداف Padding، تجمیع زیان در سطح تمام توکن‌های معتبر و اعمال یک‌باره تابع نمایی استوار است.

۲-۳-۱- نتایج کمی روی مجموعه‌های آزمون

در هر مجموعه آزمون، مدل در حالت ارزیابی و بدون محاسبه گرادیان اجرا شد. مجموع منفی لگاریتم احتمال توکن‌های صحیح بر تعداد کل توکن‌های معتبر تقسیم شد و سپس پرپلکسیتی از اعمال تابع نمایی بر میانگین Cross-Entropy به دست آمد. نتایج اجرای مدل تنظیم‌شده با LoRA در جدول ۲-۳ ارائه شده‌اند.

*جدول ۲–۳: نتایج ارزیابی مدل تنظیم‌شده با **LoRA** روی مجموعه‌های آزمون اخبار و ویکی‌پدیا*

مجموعه آزمون	نوع دامنه	تعداد توکن معتبر	میانگین Cross-Entropy	پرپلکسیتی
اخبار فارسی	درون‌دامنه	۲۵۱,۷۴۵	۱.۸۰۱۵۶۷	۶.۰۵۹۱۳۶
ویکی‌پدیا	برون‌دامنه	۱,۲۹۲,۶۱۱	۲.۲۸۲۷۵۶	۹.۸۰۳۶۶۲

روی مجموعه آزمون اخبار، مدل LoRA به میانگین Cross-Entropy برابر ۱.۸۰۱۵۶۷ و پرپلکسیتی برابر ۶.۰۵۹۱۳۶ دست یافت. این مجموعه شامل نمونه‌هایی بود که در فرایند آموزش یا اعتبارسنجی برای به‌روزرسانی وزن‌ها استفاده نشده بودند، اما از همان دامنه خبری داده‌های آموزش تشکیل شده بود. بنابراین، عملکرد ثبت‌شده فقط نشان‌دهنده کاهش زیان روی نمونه‌های آموزشی نیست و میزان تعمیم مدل به متون خبری دیده‌نشده را نیز منعکس می‌کند.

در مجموعه آزمون ویکی‌پدیا، میانگین Cross-Entropy برابر ۲.۲۸۲۷۵۶ و پرپلکسیتی برابر ۹.۸۰۳۶۶۲ به دست آمد. این مجموعه در فرایند آموزش، اعتبارسنجی یا بهینه‌سازی LoRA نقشی نداشت و صرفاً به‌عنوان داده برون‌دامنه‌ای برای بررسی حفظ عملکرد عمومی مدل استفاده شد.

تعداد توکن‌های معتبر در ارزیابی LoRA با ارزیابی مدل پایه یکسان باقی ماند؛ زیرا مجموعه‌های آزمون، توکن‌سازی و قواعد حذف Padding در هر دو آزمایش ثابت بودند. این ثبات از نظر مهندسی اهمیت دارد؛ زیرا تغییر مقدار پرپلکسیتی را نمی‌توان به حذف یا اضافه‌شدن نمونه‌ها، تغییر تعداد اهداف معتبر یا تفاوت در پیش‌پردازش نسبت داد.

۲-۳-۲- تحلیل عملکرد درون‌دامنه و برون‌دامنه

برای تحلیل اثر LoRA، نتایج مدل تنظیم‌شده با خط پایه مربوط به همان مجموعه آزمون مقایسه شدند. در مجموعه اخبار، پرپلکسیتی مدل پایه برابر ۷.۷۰۹۴۳۹ بود که پس از تنظیم دقیق با LoRA به ۶.۰۵۹۱۳۶ کاهش یافت. کاهش مطلق پرپلکسیتی اخبار برابر است با:

$$\Delta PPL_{News} = 7.709439 - 6.059136 = 1.650303$$

میزان بهبود نسبی نیز از رابطه زیر به دست می‌آید:

$$Improvement_{News} = \frac{7.709439 - 6.059136}{7.709439} \times 100 \approx 21.41\%$$

بنابراین، تنظیم دقیق LoRA باعث کاهش حدود ۲۱.۴۱ درصدی پرپلکسیتی روی اخبار دیده‌نشده شده است. ازآنجاکه مقدار کمتر پرپلکسیتی به معنای اختصاص احتمال بیشتر به توکن‌های صحیح است، این نتیجه نشان می‌دهد مدل پس از آموزش، توزیع واژگانی و روابط زمینه‌ای متون خبری را بهتر مدل‌سازی کرده است.

همین نتیجه در مقیاس لگاریتمی نیز مشاهده می‌شود. میانگین Cross-Entropy اخبار از ۲.۰۴۲۴۴۵ در مدل پایه به ۱.۸۰۱۵۶۷ در مدل LoRA کاهش یافته است:

$$\Delta\mathcal{L}_{News} = 1.801567 - 2.042445 = -0.240878$$

کاهش زیان روی آزمون اخبار نشان می‌دهد مؤلفه‌های کم‌رتبه LoRA صرفاً متن‌های آموزشی را حفظ نکرده‌اند، بلکه الگوهای آموخته‌شده به نمونه‌های خبری دیده‌نشده نیز منتقل شده‌اند. این الگوها می‌توانند شامل ساختار رایج جمله‌های خبری، توالی نام نهادها و سازمان‌ها، افعال گزارش‌دهی، روابط زمانی و نحوه ادامه‌دادن متن‌های رسمی باشند.

از دیدگاه معماری، این بهبود با محل اعمال LoRA سازگار است. ماتریس‌های کم‌رتبه به نگاشت‌های Query، Key، Value و خروجی Attention متصل شده‌اند. در نتیجه، مدل می‌تواند نحوه برقراری ارتباط میان توکن‌ها و انتخاب اطلاعات زمینه‌ای را با ساختار متن خبری تطبیق دهد. بااین‌حال، وزن‌های اصلی مدل تغییر نکرده‌اند و سازگارسازی از طریق مؤلفه‌های افزوده‌شده انجام شده است.

رفتار مدل روی داده‌های ویکی‌پدیا جهت متفاوتی داشت. پرپلکسیتی این مجموعه از ۷.۳۳۸۸۶۸ در مدل پایه به ۹.۸۰۳۶۶۲ پس از تنظیم دقیق افزایش یافت. تغییر مطلق برابر است با:

$$\Delta PPL_{Wiki} = 9.803662 - 7.338868 = 2.464794$$

افزایش نسبی پرپلکسیتی ویکی‌پدیا نیز به‌صورت زیر محاسبه می‌شود:

$$Degradation_{Wiki} = \frac{9.803662 - 7.338868}{7.338868} \times 100 \approx 33.59\%$$

در مقیاس Cross-Entropy نیز زیان ویکی‌پدیا از ۱.۹۹۳۱۸۵ به ۲.۲۸۲۷۵۶ افزایش یافته است:

$$\Delta\mathcal{L}_{Wiki} = 2.282756 - 1.993185 = 0.289571$$

این افزایش نشان می‌دهد مدل LoRA پس از تنظیم دقیق، به‌طور متوسط احتمال کمتری به توکن‌های صحیح متون عمومی ویکی‌پدیا اختصاص داده است. بنابراین، یادگیری بهتر دامنه خبری بدون هزینه نبوده و با افت عملکرد برون‌دامنه‌ای همراه شده است.

این رفتار را می‌توان با تفاوت توزیع داده‌های آموزش و ارزیابی توضیح داد. تمام به‌روزرسانی‌های LoRA از اخبار فارسی به دست آمده‌اند، درحالی‌که مجموعه ویکی‌پدیا شامل متون عمومی با سبک، موضوع و ترکیب زبانی متفاوت است. بهینه‌ساز در طول آموزش فقط برای کاهش زیان اخبار هدایت شده و هیچ مؤلفه‌ای در تابع هدف وجود نداشته است که حفظ عملکرد روی ویکی‌پدیا

را مستقیماً تضمین کند. در نتیجه، پارامترهای کم‌رتبه به جهتی تغییر کرده‌اند که برای پیش‌بینی متون خبری مناسب‌تر است، اما این جهت الزاماً برای متون عمومی بهینه نیست.

ثابت‌بودن وزن‌های مدل پایه نیز مانع کامل افت برون‌دامنه‌ای نشده است. هرچند دانش اصلی مدل مستقیماً بازنویسی نشده، خروجی LoRA به خروجی لایه‌های توجه افزوده می‌شود و در زمان استنتاج روی تمام ورودی‌ها، از جمله متون ویکی‌پدیا، فعال است. بنابراین، مؤلفه کم‌رتبه می‌تواند مسیر جریان اطلاعات و توزیع احتمال خروجی را در دامنه‌های دیگر نیز تغییر دهد. این موضوع نشان می‌دهد فریزکردن وزن‌های اصلی به‌تنهایی تضمین نمی‌کند که رفتار قبلی مدل بدون تغییر حفظ شود.

نسبت پرپلکسیتی مدل LoRA به مدل پایه در دو دامنه نیز جهت این تغییر را روشن‌تر می‌کند:

$$\frac{PPL_{News}^{LoRA}}{PPL_{News}^{Base}} = \frac{6.059136}{7.709439} \approx 0.7859$$

$$\frac{PPL_{News}^{LoRA}}{PPL_{News}^{Base}} = \frac{9.803662}{7.338868} \approx 1.3359$$

نسبت نخست کمتر از یک است و نشان می‌دهد عدم‌اطمینان مؤثر مدل در اخبار کاهش یافته است. نسبت دوم بیشتر از یک است و بیان می‌کند عدم‌اطمینان مدل در ویکی‌پدیا افزایش پیدا کرده است. درنتیجه، LoRA یک جابه‌جایی روشن در رفتار مدل به سمت دامنه هدف ایجاد کرده است.

از نظر مهندسی، نتیجه اصلی این بخش وجود یک مصالحه میان تخصص‌یافتگی در دامنه اخبار و حفظ عملکرد عمومی است. مدل در اخبار بهبود قابل‌توجهی برابر با حدود ۲۱.۴۱ درصد داشته است، اما افت نسبی ویکی‌پدیا حدود ۳۳.۵۹ درصد بوده و در نتیجه از درصد بهبود اخبار بیشتر است. بااین‌حال، مقایسه مستقیم این دو درصد برای قضاوت نهایی کافی نیست؛ زیرا مجموعه‌های اخبار و ویکی‌پدیا از نظر موضوع، زبان، اندازه و توزیع توکن متفاوت‌اند. همچنین، میزان قابل‌قبول‌بودن افت به کاربرد نهایی مدل بستگی دارد. برای سامانه‌ای که فقط متون خبری فارسی را پردازش می‌کند، این مصالحه ممکن است قابل‌پذیرش باشد؛ اما برای یک مدل چندمنظوره که باید هم‌زمان توانایی عمومی خود را حفظ کند، افزایش مشاهده‌شده در پرپلکسیتی ویکی‌پدیا اهمیت بیشتری خواهد داشت.

افت عملکرد برون‌دامنه‌ای و افزایش پرپلکسیتی ویکی‌پدیا را می‌توان نشانه‌ای از تداخل دامنه‌ای و فراموشی بخشی از رفتار پیشین مدل دانست. نسبت افت رسمی LoRA برابر با ۰.۳۳۵۸۵۵ یا حدود ۳۳.۵۹ درصد است که در تحلیل خودکار اجرای نهایی، افزایشی قابل‌توجه و سازگار با فراموشی برون‌دامنه‌ای شدید در این آزمایش تشخیص داده شد. بااین‌حال، قضاوت نهایی درباره قابل‌توجیه‌بودن این افت باید با درنظرگرفتن نتایج Adapter، رفتار Validation، مصرف منابع و خروجی‌های کیفی انجام شود. این تحلیل مقایسه‌ای در فصل چهارم ارائه خواهد شد.

در مجموع، LoRA هدف اصلی تنظیم دقیق را محقق کرد و پرپلکسیتی اخبار را از ۷.۷۰۹۴۳۹ به ۶.۰۵۹۱۳۶، معادل حدود ۲۱.۴۱ درصد، کاهش داد. در مقابل، پرپلکسیتی ویکی‌پدیا از ۷.۳۳۸۸۶۸ به ۹.۸۰۳۶۶۲ افزایش یافت که نشان‌دهنده افتی حدود ۳۳.۵۹ درصدی در عملکرد برون‌دامنه‌ای است. بنابراین، نتایج این بخش نشان می‌دهند که آموزش تنها ۵,۵۷۰,۵۶۰ پارامتر

کم‌رتبه نیز می‌تواند رفتار مدل بزرگ را به‌طور معناداری تغییر دهد؛ هم در جهت مطلوب در دامنه هدف و هم به‌صورت افت قابل‌توجه در دامنه‌ای که در آموزش حضور نداشته است.

۲-۴- نکات پیاده‌سازی و ملاحظات عملی

در این بخش، دو قطعه‌کد اصلی مرتبط با اجرای LoRA بررسی می‌شوند. قطعه نخست، ماژول‌های مجاز سازوکار توجه را مستقیماً از معماری مدل استخراج کرده و تعداد آن‌ها را با ساختار مورد انتظار تطبیق می‌دهد. قطعه دوم، حلقه عمومی آموزش را پیاده‌سازی می‌کند که وظیفه بهینه‌سازی پارامترهای قابل‌آموزش، مدیریت دقت محاسباتی، تجمیع گرادیان، اجرای ارزیابی‌های دوره‌ای و ثبت شاخص‌های آموزش و اعتبارسنجی را بر عهده دارد. این پیاده‌سازی با الزام استفاده از حلقه استاندارد PyTorch و محدودکردن LoRA به نگاشت‌های توجه سازگار است.

۲-۴-۱- شناسایی ماژول‌های هدف LoRA

پیش از اعمال LoRA، لازم بود نام دقیق نگاشت‌های مجاز در معماری بارگذاری‌شده مدل شناسایی شود. برای این منظور، فهرست projection_names شامل چهار نام q_proj، k_proj، v_proj و o_proj تعریف شد و سپس ماژول‌های منطبق با این نام‌ها از خروجی named_modules() استخراج شدند.

```python
projection_names = [
    "q_proj", "k_proj", "v_proj", "o_proj"
]

lora_target_modules = [
    name
    for name, module in lora_base_model.named_modules()
    if "language_model" in name.split(".")
    and ".self_attn." in name
    and name.rsplit(".", 1)[-1] in projection_names
    and isinstance(module, nn.Linear)
]

expected_target_count = (
    num_layers * 2
    + (num_layers - num_kv_shared_layers) * 2
)
```

*شکل ۲–۱: تعریف نگاشت‌های مجاز، استخراج ماژول‌های هدف **LoRA** و کنترل تعداد مورد انتظار آن‌ها*

در فرایند استخراج lora_target_modules، تنها ماژول‌های خطی متعلق به بخش language_model و زیرماژول self_attn پذیرفته شدند. این محدودیت چندمرحله‌ای مانع از آن می‌شود که ماژول‌هایی با نام مشابه در قسمت‌های دیگر مدل به‌صورت ناخواسته هدف LoRA قرار گیرند. همچنین بررسی نوع ماژول با isinstance(module, nn.Linear) تضمین می‌کند که فقط نگاشت‌های خطی سازوکار توجه در فهرست نهایی وارد شوند.

تعداد مورد انتظار ماژول‌های هدف با استفاده از اطلاعات پیکربندی معماری محاسبه شد:

$$N_{expected} = 2N_{layers} + 2(N_{layers} - N_{KV\ shared})$$

در این رابطه، دو نگاشت Query و Output برای تمام لایه‌های Transformer در نظر گرفته می‌شوند؛ درحالی‌که تعداد نگاشت‌های مستقل Key و Value با توجه به لایه‌های دارای اشتراک KV محاسبه می‌شود. در پایان، تعداد ماژول‌های استخراج‌شده با مقدار مورد انتظار مقایسه شد و در صورت عدم تطابق، اجرای برنامه با خطا متوقف می‌شد. این کنترل اهمیت دارد؛ زیرا صرفاً یافتن تعدادی ماژول با پسوندهای موردنظر برای اطمینان از پوشش صحیح کل معماری کافی نیست.

۲-۴-۲- پیاده‌سازی حلقه آموزش با تابع train_model

آموزش مدل LoRA با تابع عمومی train_model انجام شد. این تابع علاوه بر مدل، DataLoader آموزش و نام اجرای آزمایش، متن‌های اعتبارسنجی و اندازه Batch ارزیابی کامل را دریافت می‌کند. سپس انتخاب پارامترهای قابل‌آموزش، ساخت بهینه‌ساز، آموزش، تجمیع گرادیان، Mini-validation دوره‌ای، Full validation پایان هر Epoch، اندازه‌گیری زمان و ثبت Peak VRAM را مدیریت می‌کند.

```python
def train_model(model, dataloader, run_name):
    trainable_parameters = [parameter for parameter in model.parameters() if parameter.requires_grad]

    if not trainable_parameters:
        raise RuntimeError("No trainable parameters were found.")

    optimizer = AdamW(trainable_parameters, lr=LEARNING_RATE)
    scaler = torch.cuda.amp.GradScaler(enabled=USE_FP16_SCALER)

    if USE_GRADIENT_CHECKPOINTING:
        model.gradient_checkpointing_enable()
        if hasattr(model, "enable_input_require_grads"):
            model.enable_input_require_grads()

    model.config.use_cache = False
    model.train()

    if torch.cuda.is_available():
        torch.cuda.empty_cache()
        torch.cuda.reset_peak_memory_stats()
        torch.cuda.synchronize()

    start_time = time.perf_counter()

    step_losses, epoch_losses = [], []
    optimizer.zero_grad(set_to_none=True)

    for epoch in range(NUM_EPOCHS):
        epoch_loss_sum = 0.0

        progress = tqdm(dataloader, desc=f"{run_name} | epoch {epoch + 1}/{NUM_EPOCHS}")

        for step, batch in enumerate(progress):
            input_ids = batch["input_ids"].to(DEVICE)
            attention_mask = batch["attention_mask"].to(DEVICE)
            labels = batch["labels"].to(DEVICE)

            with autocast_context():
                outputs = model(
                    input_ids=input_ids,
                    attention_mask=attention_mask,
                    labels=labels,
                    use_cache=False,
                )
                raw_loss = outputs.loss
                loss = raw_loss / GRAD_ACCUM_STEPS

            scaler.scale(loss).backward()

            should_update = (step + 1) % GRAD_ACCUM_STEPS == 0 or step + 1 == len(dataloader)

            if should_update:
                scaler.unscale_(optimizer)
                torch.nn.utils.clip_grad_norm_(trainable_parameters, max_norm=1.0)

                scaler.step(optimizer)
                scaler.update()
                optimizer.zero_grad(set_to_none=True)

            loss_value = raw_loss.detach().float().item()

            step_losses.append(loss_value)
            epoch_loss_sum += loss_value
            progress.set_postfix(loss=f"{loss_value:.4f}")

        epoch_mean_loss = epoch_loss_sum / len(dataloader)
        epoch_losses.append(epoch_mean_loss)

        print(f"{run_name} epoch {epoch + 1} mean loss: {epoch_mean_loss:.6f}")

    if torch.cuda.is_available():
        torch.cuda.synchronize()

    training_seconds = time.perf_counter() - start_time

    peak_vram_gb = torch.cuda.max_memory_allocated() / (1024 ** 3) if torch.cuda.is_available() else 0.0

    if hasattr(model, "gradient_checkpointing_disable"):
        model.gradient_checkpointing_disable()

    model.eval()

    return {
        "step_losses": step_losses,
        "epoch_losses": epoch_losses,
        "training_seconds": training_seconds,
        "peak_vram_gb": peak_vram_gb
    }
```

شکل ۲–۲: پیاده‌سازی کامل تابع ***train_model*** *برای آموزش پارامترهای* ***LoRA*** *با حلقه استاندارد* ***PyTorch***

در ابتدای تابع، فقط پارامترهایی انتخاب می‌شوند که requires_grad=True دارند. بنابراین بهینه‌ساز AdamW صرفاً پارامترهای فعال LoRA را دریافت می‌کند و وزن‌های ثابت مدل پایه وارد فرایند بهینه‌سازی نمی‌شوند. وجود حداقل یک پارامتر قابل‌آموزش نیز پیش از شروع آموزش کنترل می‌شود تا اجرای حلقه روی مدلی کاملاً فریزشده به‌صورت ناخواسته ادامه پیدا نکند.

در عبور پیش‌رو، تابع autocast_context فراخوانی می‌شود. این تابع در محیط CUDA از torch.autocast با نوع داده float16 استفاده می‌کند. در نتیجه، عملیات سازگار با دقت کاهش‌یافته اجرا می‌شوند تا مصرف حافظه و هزینه محاسباتی نسبت به اجرای کامل با float32 کاهش یابد. وزن‌های ثابت مدل پایه در float16 قرار دارند، اما پارامترهای قابل‌آموزش LoRA پیش از آموزش به float32 تبدیل شده‌اند تا به‌روزرسانی آن‌ها با دقت عددی بیشتری انجام شود.

پس از محاسبه زیان خام، مقدار زیان بر تعداد مراحل تجمیع گرادیان تقسیم می‌شود:

$$\mathcal{L}_{backward} = \frac{\mathcal{L}_{raw}}{N_{accumulation}}$$

در اجرای نهایی، $N_{accumulation} = 2$ بود؛ بنابراین زیان هر ریزBatch پیش از پس‌انتشار بر ۲ تقسیم شد. این کار باعث می‌شود مجموع گرادیان‌های دو ریزBatch متوالی، از نظر مقیاس با گرادیان میانگین یک Batch مؤثر هشت‌نمونه‌ای سازگار باشد. بهینه‌ساز پس از تکمیل دو مرحله تجمیع، یا در آخرین ریزBatch هر Epoch، پارامترها را به‌روزرسانی می‌کند.

از آنجا که اجرای CUDA با float16 انجام شد، متغیر USE_FP16_SCALER برابر True قرار گرفت و GradScaler در حلقه آموزش فعال بود. در دقت float16، برخی گرادیان‌های کوچک ممکن است به دلیل محدودیت دامنه عددی به صفر میل کنند. GradScaler با ضرب موقت زیان در یک ضریب پویا، اندازه گرادیان‌ها را پیش از پس‌انتشار افزایش می‌دهد و احتمال Underflow را کاهش می‌دهد.

پس از اجرای backward روی زیان مقیاس‌یافته، تابع scaler.unscale_(optimizer) فراخوانی شد تا گرادیان‌ها پیش از Gradient Clipping به مقیاس واقعی بازگردند. سپس نرم گرادیان پارامترهای قابل‌آموزش با مقدار حداکثر یک محدود شد. در نهایت، scaler.step(optimizer) مرحله بهینه‌ساز را اجرا کرد و scaler.update() مقدار مقیاس را برای مراحل بعدی تنظیم نمود. این ترتیب اهمیت دارد؛ زیرا محدودسازی گرادیان باید روی مقادیر واقعی و بدون مقیاس انجام شود.

پس از هر به‌روزرسانی، گرادیان‌ها با optimizer.zero_grad(set_to_none=True) پاک شدند. استفاده از set_to_none=True از نگهداری Tensorهای صفر اضافی جلوگیری می‌کند و مدیریت حافظه را نسبت به صفرکردن صریح همه گرادیان‌ها کارآمدتر می‌سازد.

زیان ثبت‌شده در تاریخچه، raw_loss پیش از تقسیم بر تعداد مراحل تجمیع است. بنابراین، نمودار آموزش مقدار واقعی Loss مدل را نمایش می‌دهد و به‌طور مصنوعی به نصف کاهش پیدا نمی‌کند. تابع علاوه بر زیان هر ریزBatch و میانگین زیان هر Epoch، شماره مراحل و زیان‌های Mini-validation، زیان و پرپلکسیتی Full validation، تعداد توکن‌های معتبر Validation، زمان خالص آموزش، زمان Mini-validation، زمان Full validation و Peak VRAM مربوط به آموزش و اعتبارسنجی را به‌صورت جداگانه ثبت می‌کند. این خروجی‌ها در فصل چهارم برای تحلیل همگرایی، بیش‌برازش و مقایسه هزینه محاسباتی LoRA و Adapter استفاده خواهند شد.

۲-۴-۳- اجرای اعتبارسنجی دوره‌ای

تابع train_model دو سطح از اعتبارسنجی را اجرا می‌کند. در سطح نخست، پس از هر ۲۰ ریزBatch آموزشی و فقط پس از انجام یک مرحله به‌روزرسانی بهینه‌ساز، یک Batch از مجموعه اعتبارسنجی با اندازه ۲ ارزیابی می‌شود. شرط اجرای Validation پس از مرحله بهینه‌ساز باعث می‌شود نقطه ثبت‌شده به وضعیت به‌روزشده مدل مربوط باشد و ارزیابی در میانه یک چرخه ناقص تجمیع گرادیان انجام نشود.

برای این منظور، یک DataLoader مستقل و بدون Shuffle برای منحنی اعتبارسنجی ساخته شد. Iterator این DataLoader پس از رسیدن به انتهای مجموعه دوباره از ابتدا ایجاد می‌شود. ارزیابی Mini-validation در حالت eval و تحت torch.inference_mode() انجام می‌شود و پس از پایان آن، مدل به حالت train بازمی‌گردد. بنابراین، هیچ گرادیانی از داده اعتبارسنجی محاسبه یا در بهینه‌سازی استفاده نمی‌شود.

در سطح دوم، پس از پایان هر Epoch، تابع دستی پرپلکسیتی روی کل ۱۰۰۰ نمونه بخش dev اجرا می‌شود. این ارزیابی، میانگین Cross-Entropy، پرپلکسیتی و تعداد توکن‌های معتبر را ثبت می‌کند. در نتیجه، Mini-validation برای نمایش روند پیوسته‌تر درون Epoch و Full validation برای مقایسه رسمی عملکرد پایان Epochها به کار می‌روند.

برای جلوگیری از مخلوط‌شدن هزینه Validation با هزینه واقعی آموزش، زمان و Peak VRAM این دو بخش جداگانه اندازه‌گیری شدند. پیش از اجرای Validation، Peak Memory ثبت‌شده برای آموزش ذخیره و شمارنده حافظه CUDA بازنشانی شد. پس از Validation نیز Peak مربوط به همان مرحله به‌طور مستقل ثبت گردید. همچنین زمان Mini-validation از زمان حلقه آموزش هر Epoch کم شد تا متغیر training_seconds زمان خالص بهینه‌سازی را نشان دهد.

۲-۵- جمع‌بندی فصل

در این فصل، روش LoRA با موفقیت روی مدل پایه اعمال شد و سازگارسازی مدل فقط از طریق ماتریس‌های کم‌رتبه افزوده‌شده به نگاشت‌های توجه انجام گرفت. وزن‌ها و بایاس‌های مدل اصلی در تمام مراحل ثابت باقی ماندند و تنها ۵،۵۷۰،۵۶۰ پارامتر، معادل حدود ۰.۱۰۲۸۰۷ درصد از پارامترهای مدل پایه، وارد فرایند بهینه‌سازی شدند. این موضوع نشان داد که می‌توان بدون تنظیم دقیق کامل مدل، رفتار آن را به‌صورت معناداری به سمت یک دامنه تخصصی هدایت کرد.

آموزش مدل منحصراً بر داده‌های اخبار فارسی انجام شد و نتایج ارزیابی نشان دادند که LoRA توانسته است مدل را با الگوهای زبانی، ساختار جمله و روابط زمینه‌ای متون خبری سازگارتر کند. پرپلکسیتی اخبار از ۷.۷۰۹۴۳۹ در مدل پایه به ۶.۰۵۹۱۳۶ کاهش یافت که معادل بهبودی حدود ۲۱.۴۱ درصد است. این بهبود روی نمونه‌های آزمون دیده‌نشده بیانگر آن است که مدل صرفاً داده‌های آموزشی را حفظ نکرده، بلکه بخشی از ویژگی‌های عمومی دامنه خبری را فراگرفته و به داده‌های جدید همان حوزه منتقل کرده است.

در مقابل، این تخصص‌یافتگی با افت قابل‌توجه عملکرد روی متون عمومی ویکی‌پدیا همراه بود. پرپلکسیتی ویکی‌پدیا از ۷.۳۳۸۸۶۸ در مدل پایه به ۹.۸۰۳۶۶۲ افزایش یافت که نشان‌دهنده افتی حدود ۳۳.۵۹ درصدی است. این نتیجه نشان می‌دهد

که ثابت‌بودن وزن‌های مدل پایه، به‌تنهایی حفظ کامل رفتار عمومی مدل را تضمین نمی‌کند؛ زیرا مؤلفه‌های LoRA در زمان استنتاج بر خروجی لایه‌های توجه اثر می‌گذارند و می‌توانند توزیع احتمال مدل را در دامنه‌های دیگر نیز تغییر دهند. بنابراین، LoRA در این آزمایش یک مصالحه روشن میان سازگاری با دامنه هدف و حفظ عملکرد برون‌دامنه‌ای ایجاد کرده است.

از نظر پیاده‌سازی، شناسایی مستقیم ماژول‌های هدف از معماری واقعی مدل و کنترل تعداد مورد انتظار آن‌ها، احتمال تزریق اشتباه LoRA به بخش‌های غیرمجاز را کاهش داد. همچنین، استفاده از حلقه استاندارد PyTorch، تجمیع گرادیان، Gradient Checkpointing، Autocast مبتنی بر float16، GradScaler و ثبت Mini-validation و Full validation امکان اجرای آموزش و پایش روند یادگیری را تحت محدودیت حافظه فراهم کرد. در مجموع، LoRA روشی کارآمد برای تخصصی‌سازی مدل در دامنه اخبار بود، اما نتایج نشان دادند که ارزیابی آن نباید فقط بر بهبود درون‌دامنه‌ای متمرکز باشد و اثر آن بر حفظ عملکرد عمومی، رفتار Validation و کیفیت تولید متن نیز باید هم‌زمان در نظر گرفته شود.

فصل سوم

پیاده‌سازی دستی و آموزش مدل با روش Bottleneck Adapter

در این فصل، مدل پایه با استفاده از روش Bottleneck Adapter و بدون بهره‌گیری از کتابخانه‌های آماده‌ای مانند PEFT یا AdapterHub تنظیم دقیق می‌شود. برخلاف روش LoRA که تغییرات قابل‌آموزش را درون نگاشت‌های خطی Attention وارد می‌کند، در این روش شبکه‌های عصبی کوچکی به مسیر محاسباتی بلوک‌های Transformer افزوده می‌شوند. وزن‌های اصلی مدل ثابت باقی می‌مانند و فقط پارامترهای این ماژول‌های جدید در جریان آموزش به‌روزرسانی می‌شوند.

مطابق طراحی تمرین، هر Bottleneck Adapter از یک لایه Linear برای کاهش بُعد، تابع فعال‌ساز GELU و یک لایه Linear برای بازگرداندن بردار به بُعد اصلی تشکیل شده است. این ماژول پس از خروجی MLP هر بلوک متنی Transformer قرار می‌گیرد و خروجی آن به‌صورت Residual با خروجی اصلی MLP جمع می‌شود. مدل حاصل، تحت همان شرایط فصل دوم و منحصراً روی داده‌های آموزش اخبار فارسی بهینه‌سازی شده و سپس روی مجموعه‌های آزمون اخبار و ویکی‌پدیا ارزیابی می‌شود.

۳-۱- مبانی و طراحی Bottleneck Adapter

Bottleneck Adapter یکی از روش‌های تنظیم دقیق پارامترکارا است که به‌جای تغییر مستقیم وزن‌های مدل پیش‌آموزش‌دیده، ماژول‌های کوچک و قابل‌آموزشی را به نقاط مشخصی از معماری اضافه می‌کند. در این روش، مدل پایه نقش یک استخراج‌کننده بازنمایی عمومی را حفظ می‌کند و Adapterها وظیفه سازگارکردن این بازنمایی‌ها با دامنه یا وظیفه جدید را بر عهده می‌گیرند.

در ادامه، ابتدا مفهوم کلی تنظیم دقیق با Adapter توضیح داده می‌شود. سپس ساختار داخلی Bottleneck Adapter، محل قرارگیری آن پس از MLP، نحوه استفاده از اتصال Residual و وضعیت پارامترهای قابل‌آموزش بررسی خواهد شد.

۳-۱-۱- مفهوم تنظیم دقیق با Adapter

مدل‌های زبانی بزرگ در مرحله پیش‌آموزش، حجم زیادی از اطلاعات زبانی و الگوهای عمومی متن را در پارامترهای خود ذخیره می‌کنند. برای سازگارکردن چنین مدلی با یک حوزه تخصصی، می‌توان تمام پارامترهای آن را روی داده‌های جدید تنظیم دقیق کرد. در تنظیم دقیق کامل، اگر مجموعه پارامترهای مدل با θ نمایش داده شود، همه یا بخش بزرگی از این پارامترها در جریان کمینه‌سازی تابع زیان تغییر می‌کنند:

$$\theta^* = \underset{\theta}{\operatorname{argmin}}\, \mathcal{L}\left(\theta; \mathcal{D}_{target}\right)$$

این رویکرد انعطاف‌پذیری زیادی دارد، اما برای مدل‌های بزرگ مستلزم محاسبه و نگهداری گرادیان، حالت‌های بهینه‌ساز و نسخه‌های به‌روزشده تعداد بسیار زیادی پارامتر است. در نتیجه، هزینه حافظه، زمان آموزش و ذخیره‌سازی مدل افزایش می‌یابد. علاوه بر این، برای هر وظیفه یا دامنه جدید باید نسخه دیگری از وزن‌های مدل نگهداری شود.

روش Adapter این مسئله را با جداسازی پارامترهای عمومی و پارامترهای تخصصی حل می‌کند. وزن‌های مدل پیش‌آموزش‌دیده ثابت می‌مانند و مجموعه جدید و کوچکی از پارامترها با نماد ϕ به معماری افزوده می‌شود. در این حالت، مسئله آموزش به‌صورت زیر تعریف می‌شود:

$$\phi^* = \underset{\phi}{\operatorname{argmin}}\, \mathcal{L}(\theta_{base}, \phi; \mathcal{D}_{target})$$

$$\theta_{base} = constant$$

بنابراین، مدل اصلی همچنان همان بازنمایی‌های پیش‌آموخته‌شده را تولید می‌کند، اما Adapterها روی این بازنمایی‌ها تغییرات محدودی اعمال می‌کنند تا خروجی مدل با داده‌های دامنه هدف سازگارتر شود. در این آزمایش، دامنه هدف اخبار فارسی است و Adapterها باید الگوهای موردنیاز برای مدل‌سازی بهتر این متون را فرا بگیرند.

از نظر مفهومی، Adapter را می‌توان یک مسیر پردازشی کوچک درون مدل دانست. ورودی این مسیر یک بازنمایی با بُعد بالا است. این بازنمایی ابتدا به فضای کم‌بُعدتری منتقل می‌شود، در آن فضا یک تبدیل غیرخطی روی آن صورت می‌گیرد و سپس دوباره به بُعد اولیه بازگردانده می‌شود. خروجی این مسیر جایگزین خروجی اصلی مدل نمی‌شود، بلکه به آن افزوده می‌شود.

این طراحی چند نتیجه مهم دارد. نخست اینکه تعداد پارامترهای جدید بسیار کمتر از تعداد پارامترهای مدل پایه است. دوم اینکه برای هر دامنه می‌توان Adapter جداگانه‌ای آموزش داد، درحالی‌که مدل پایه میان همه دامنه‌ها مشترک باقی می‌ماند. سوم اینکه حذف یا تعویض Adapter به معنای تغییر وزن‌های اصلی مدل نیست و مدیریت نسخه‌های تخصصی مدل ساده‌تر می‌شود.

بااین‌حال، ثابت‌بودن وزن‌های اصلی به معنای ثابت‌ماندن کامل رفتار مدل نیست. Adapterها در مسیر پیش‌رو فعال‌اند و خروجی لایه‌ها را تغییر می‌دهند؛ بنابراین می‌توانند توزیع احتمال مدل را هم در دامنه هدف و هم در دامنه‌های دیگر تغییر دهند. میزان واقعی حفظ توانایی عمومی مدل باید از طریق نتایج تجربی، مانند پرپلکسیتی ویکی‌پدیا، بررسی شود.

تفاوت اصلی این روش با LoRA در محل و نوع پارامترسازی تغییرات است. LoRA یک تغییر کم‌رتبه را مستقیماً به نگاشت‌های خطی مشخصی در Attention اضافه می‌کند، درحالی‌که Bottleneck Adapter یک شبکه کوچک مستقل را پس از خروجی MLP قرار می‌دهد. در LoRA، تغییرات درون محاسبات نگاشت‌های Attention ایجاد می‌شوند؛ اما در Adapter، مسیر اصلی MLP حفظ شده و یک اصلاح جدید به خروجی آن افزوده می‌شود.

۳-۱-۲- معماری Bottleneck Adapter

فرض شود خروجی یک لایه MLP برای هر توکن، برداری با بُعد d باشد:

$$h \in \mathbb{R}^d$$

یک Bottleneck Adapter این بردار را در سه مرحله پردازش می‌کند. در مرحله نخست، لایه Down-projection بازنمایی را از بُعد اصلی d به بُعد میانی کوچک‌تر m منتقل می‌کند:

$$z = W_{down}h + b_{down}$$

$$W_{down} \in \mathbb{R}^{m \times d}, \quad b_{down} \in \mathbb{R}^{m}$$

در این آزمایش، بُعد اصلی بازنمایی متنی مدل برابر ۲۰۴۸ و بُعد میانی Adapter برابر ۱۶ است. بنابراین، هر بردار ۲۰۴۸بعدی پیش از ورود به بخش قابل‌آموزش اصلی Adapter به یک بردار ۱۶بعدی تبدیل می‌شود:

$$\mathbb{R}^{2048} \rightarrow \mathbb{R}^{16}$$

این کاهش شدید بُعد باعث می‌شود Adapter نتواند یک تبدیل کاملاً آزاد با ابعاد مدل اصلی ایجاد کند. در عوض، مجبور است تغییرات موردنیاز دامنه را در یک فضای فشرده یاد بگیرد. این محدودیت، علت اصلی کاهش تعداد پارامترهای قابل‌آموزش است.

در مرحله دوم، تابع فعال‌ساز GELU روی خروجی Down-projection اعمال می‌شود:

$$a = GELU(z)$$

GELU یک تابع غیرخطی است. در صورت نبود تابع فعال‌ساز، حاصل‌ضرب دو لایه Linear را می‌توان به یک تبدیل Linear واحد تبدیل کرد و ظرفیت مدل‌سازی Adapter محدودتر می‌شد. استفاده از GELU باعث می‌شود رابطه میان بازنمایی ورودی و اصلاح ایجادشده توسط Adapter صرفاً خطی نباشد.

تعریف GELU را می‌توان به‌صورت زیر نمایش داد:

$$GELU(x) = x\Phi(x)$$

که در آن $\Phi(x)$ تابع توزیع تجمعی نرمال استاندارد است. این تابع برخلاف فعال‌سازهایی که مقادیر منفی را به‌طور کامل حذف می‌کنند، ورودی‌ها را به‌صورت نرم و وابسته به مقدارشان تعدیل می‌کند.

در مرحله سوم، لایه Up-projection بردار ۱۶بعدی را دوباره به بُعد اصلی مدل بازمی‌گرداند:

$$u = W_{up}a + b_{up}$$

$$W_{up} \in \mathbb{R}^{d \times m}, \quad b_{up} \in \mathbb{R}^{d}$$

در نتیجه، تبدیل کامل Bottleneck Adapter به‌صورت زیر نوشته می‌شود:

$$Adapter(h) = W_{up}GELU(W_{down}h + b_{down}) + b_{up}$$

خروجی Adapter باید همان بُعد ورودی را داشته باشد؛ زیرا در مرحله بعد با خروجی اصلی MLP جمع می‌شود. اگر ورودی مدل به‌صورت یک Tensor با شکل $[B, T, d]$ باشد، لایه‌های Linear روی آخرین بُعد اعمال می‌شوند. بنابراین، هر توکن در هر نمونه Batch به‌طور مستقل از مسیر زیر عبور می‌کند:

$$[B, T, 2048] \rightarrow [B, T, 16] \rightarrow [B, T, 2048]$$

تعداد پارامترهای یک Bottleneck Adapter شامل وزن و بایاس Down-projection و وزن و بایاس Up-projection است:

$N_{Adapter} = dm + m + md + d = 2dm + m + d$

با جای‌گذاری $d = 2048$ و $m = 16$، تعداد پارامترهای هر Adapter برابر است با:

$N_{Adapter} = 2(2048)(16) + 16 + 2048 = 67{,}600$

در مدل مورد استفاده، ۳۰ بلوک متنی Transformer وجود دارد و پس از MLP هر بلوک یک Adapter قرار گرفته است. بنابراین، تعداد کل پارامترهای افزوده‌شده برابر است با:

$N_{total} = 30 \times 67{,}600 = 2{,}028{,}000$

این تعداد فقط حدود ۰.۰۳۷۴۲۸٪ از پارامترهای مدل پایه را تشکیل می‌دهد. در نتیجه، سازگارسازی مدل با آموزش بخش بسیار کوچکی از پارامترها انجام می‌شود. مقایسه کامل این مقدار با تعداد پارامترهای قابل‌آموزش LoRA در فصل چهارم ارائه خواهد شد.

*جدول ۳–۱: مشخصات معماری **Bottleneck Adapter** در اجرای نهایی*

مؤلفه	مقدار
بُعد ورودی و خروجی Adapter	2048
بُعد میانی Adapter	16
تابع فعال‌ساز	GELU
تعداد Adapter های تزریق‌شده	۳۰
پارامترهای هر Adapter	67,600
کل پارامترهای قابل‌آموزش	۲,۰۲۸,۰۰۰
درصد نسبت به مدل پایه	%۰.۰۳۷۴۲۸
نوع داده پارامترهای Adapter	float32
نوع داده مدل پایه	float32

۳-۱-۳- محل قرارگیری و اتصال Residual

مطابق طراحی تمرین، Bottleneck Adapter پس از خروجی MLP هر بلوک متنی Transformer قرار می‌گیرد. برای بیان دقیق این ساختار، فرض شود x ورودی MLP و h خروجی آن باشد:

$h = MLP(x)$

سپس Adapter روی خود خروجی MLP اعمال می‌شود:

$u = Adapter(h)$

خروجی ساختار جدید از جمع خروجی اصلی MLP و اصلاح تولیدشده توسط Adapter به دست می‌آید:

$$y = h + Adapter(h)$$

بنابراین، ترتیب محاسبات به‌شکل زیر است:

$$x \rightarrow MLP \rightarrow h \rightarrow \begin{cases} h: \text{مسیر مستقیم} \\ Adapter(h): Adapter \text{ مسیر} \end{cases} \rightarrow y$$

این ساختار باعث می‌شود مسیر اصلی خروجی MLP بدون حذف یا جایگزینی حفظ شود. Adapter صرفاً یک مقدار اصلاحی را به آن اضافه می‌کند. اگر Adapter اصلاح مفیدی برای یک نمونه تولید کند، این مقدار می‌تواند بازنمایی را در جهت سازگار با دامنه اخبار تغییر دهد؛ در غیر این صورت، مسیر مستقیم همچنان اطلاعات اصلی MLP را در اختیار ادامه مدل قرار می‌دهد.

این اتصال با Residualهای اصلی بلوک Transformer تعارضی ندارد. در پیاده‌سازی انجام‌شده، MLP اصلی درون یک Wrapper نگهداری شده و خروجی آن با خروجی Adapter جمع می‌شود. سپس ساختار بیرونی بلوک Transformer مطابق معماری اصلی مدل به کار خود ادامه می‌دهد. بنابراین، وزن‌ها و منطق داخلی MLP حذف نمی‌شوند و فقط خروجی تحویلی آن به ادامه بلوک تغییر می‌کند.

در مقداردهی اولیه Adapter، وزن Down-projection با روش Xavier Uniform مقداردهی شد و بایاس آن برابر صفر قرار گرفت. این انتخاب باعث می‌شود نگاشت اولیه Down-projection دارای مقیاس کنترل‌شده‌ای باشد و از شروع با مقادیر بسیار بزرگ یا بسیار کوچک جلوگیری شود. بااین‌حال، به دلیل صفر بودن وزن و بایاس Up-projection، خروجی نهایی Adapter در ابتدای آموزش همچنان دقیقاً صفر باقی می‌ماند.

یکی از تصمیم‌های مهم در پیاده‌سازی، مقداردهی اولیه صفر برای وزن و بایاس Up-projection بود:

$$W_{up}^0 = 0$$

$$b_{up}^0 = 0$$

با وجود مقداردهی غیرصفر Down-projection، صفر بودن وزن و بایاس Up-projection باعث می‌شود خروجی نهایی Adapter پیش از شروع آموزش دقیقاً برابر صفر باشد.

$$Adapter^0(h) = 0$$

در نتیجه، بلافاصله پس از تزریق Adapter و پیش از انجام به‌روزرسانی آموزشی داریم:

$$y^0 = h + 0 = h$$

بنابراین، افزودن Adapter در آغاز موجب تغییر ناگهانی خروجی MLP نمی‌شود و مدل تزریق‌شده از همان رفتار مدل پایه شروع می‌کند. سپس در طول آموزش، Up-projection و سایر پارامترهای Adapter به‌تدریج تغییر می‌کنند و مؤلفه اصلاحی موردنیاز برای دامنه اخبار شکل می‌گیرد.

این مقداردهی اولیه از نظر پایداری اهمیت دارد. اگر هر دو لایه با مقادیر تصادفی مؤثر شروع می‌شدند، Adapter پیش از یادگیری می‌توانست تغییر غیرضروری و کنترل‌نشده‌ای در خروجی همه بلوک‌ها ایجاد کند. با صفرکردن Up-projection، مسیر جدید در نقطه شروع از نظر تابعی خنثی است.

البته Residual بودن ساختار تضمین نمی‌کند که عملکرد مدل در همه دامنه‌ها پس از آموزش بدون تغییر باقی بماند. پس از یادگیری، خروجی Adapter دیگر صفر نیست و روی تمام ورودی‌ها، از جمله متون ویکی‌پدیا، اعمال می‌شود. بنابراین، اثر نهایی این طراحی باید بر اساس نتایج درون‌دامنه‌ای و برون‌دامنه‌ای ارزیابی شود.

۳-۱-۴- انجماد مدل پایه و پارامترهای قابل‌آموزش

پیش از تزریق Bottleneck Adapterها، تمام پارامترهای موجود در مدل پایه فریز شدند. این کار با غیرفعال‌کردن ویژگی requires_grad برای پارامترهای اصلی انجام شد:

$$requires_{grad(\theta_{base})} = False$$

پس از فریزکردن مدل، ماژول‌های Adapter ایجاد شدند. پارامترهای جدید این ماژول‌ها به‌طور پیش‌فرض قابل‌آموزش بودند و در نتیجه فقط آن‌ها در فهرست پارامترهای بهینه‌ساز قرار گرفتند:

$$\nabla_{\theta_{base}} \mathcal{L} = 0$$

$$\nabla_{\theta_{adapter}} \mathcal{L} \neq 0$$

این تفکیک به معنای آن است که وزن‌های Attention، MLPهای اصلی، Embeddingها، لایه‌های نرمال‌سازی و سر خروجی مدل در جریان آموزش تغییر نمی‌کنند. تنها وزن‌ها و بایاس‌های Down-projection و Up-projection در 30 Adapter تزریق‌شده به‌روزرسانی می‌شوند.

در هر Adapter چهار Tensor قابل‌آموزش وجود دارد:

- وزن Down-projection؛
- بایاس Down-projection؛
- وزن Up-projection؛
- بایاس Up-projection.

با وجود 30 Adapter، تعداد Tensorهای قابل‌آموزش برابر است با:

$$30 \times 4 = 120$$

در اجرای نهایی، کنترل شد که تمام این 120 Tensor دارای عبارت .adapter در نام خود باشند و هیچ پارامتر دیگری از مدل قابل‌آموزش باقی نمانده باشد. همچنین تعداد Adapterهای تزریق‌شده با تعداد لایه‌های متنی Transformer مقایسه شد و هر دو برابر ۳۰ بودند. این کنترل مانع از دو خطای مهم می‌شود: آموزش ناخواسته بخشی از مدل پایه و تزریق‌نشدن Adapter در یک یا چند بلوک متنی.

در اجرای نهایی، پارامترهای Adapter از ابتدا روی همان GPU مدل اما با نوع داده float32 قرار گرفتند، درحالی‌که پارامترهای ثابت مدل پایه در float16 باقی ماندند. علاوه بر این، محاسبات داخلی شاخه Adapter با غیرفعال‌کردن Autocast و تبدیل موقت ورودی آن به float32 انجام شد. خروجی Adapter پس از پایان محاسبه دوباره به نوع داده خروجی MLP بازگردانده شد و سپس اتصال Residual اجرا گردید. این طراحی برای جلوگیری از Overflow و تولید مقادیر NaN در شاخه Adapter ضروری بود، بدون آنکه لازم باشد کل مدل پایه به float32 تبدیل شود.

اگرچه فریزکردن مدل پایه، حافظه موردنیاز برای گرادیان و حالت‌های بهینه‌ساز وزن‌های اصلی را کاهش می‌دهد، همچنان عبور پیش‌رو و پس‌انتشار باید از میان بلوک‌های مدل انجام شود تا گرادیان پارامترهای Adapter محاسبه شود. همچنین فعال‌سازی‌های میانی مدل و خروجی‌های MLP برای آموزش موردنیاز هستند. بنابراین، تعداد کم پارامترهای قابل‌آموزش به‌تنهایی به معنای مصرف بسیار کم حافظه کلی نیست و طول توالی، اندازه Batch، نوع داده محاسباتی و فعال‌سازی‌های مدل نیز نقش تعیین‌کننده دارند.

در مجموع، طراحی انجام‌شده مسیر اصلی مدل را ثابت نگه می‌دارد و ظرفیت سازگارسازی را در مجموعه کوچکی از ماژول‌های Down–GELU–Up متمرکز می‌کند. اجرای شاخه Adapter در float32 نیز پایداری عددی لازم را فراهم کرد، درحالی‌که مدل پایه برای کنترل مصرف حافظه در float16 باقی ماند. این ساختار شرایط لازم را برای آموزش دستی Bottleneck Adapterها روی اخبار فارسی فراهم می‌سازد. نحوه شناسایی لایه‌های متنی، تزریق این ماژول‌ها و اجرای فرایند آموزش در بخش بعد بررسی خواهد شد.

۳-۲- تزریق دستی و فرایند آموزش Bottleneck Adapter

پس از طراحی ساختار داخلی Bottleneck Adapter، لازم بود این ماژول‌ها بدون استفاده از کتابخانه‌های آماده در معماری واقعی Gemma 3n قرار گیرند. در این مرحله، ابتدا بلوک‌های Transformer متعلق به بخش متنی مدل شناسایی شدند و MLP هر بلوک با یک Wrapper شامل MLP اصلی و Adapter جایگزین شد. سپس مدل تحت همان شرایط استفاده‌شده برای LoRA و منحصراً با داده‌های آموزش اخبار فارسی بهینه‌سازی گردید.

در تمام این فرایند، پارامترهای مدل پایه ثابت باقی ماندند و تنها پارامترهای Down-projection و Up-projection در Adapterهای افزوده‌شده وارد بهینه‌ساز شدند. این روند الزامات بخش سوم تمرین را در زمینه تزریق دستی پس از MLP، استفاده از اتصال Residual، فریز مدل اصلی و آموزش انحصاری روی اخبار پوشش می‌دهد.

۳-۲-۱- شناسایی لایه‌های متنی و تزریق Adapter

مدل Gemma 3n علاوه بر بخش زبانی، دارای اجزای دیگری نیز هست؛ بنابراین جست‌وجوی ساده همه ماژول‌های مدل می‌توانست باعث شناسایی اشتباه لایه‌هایی خارج از Transformer متنی شود. برای جلوگیری از این مسئله، لایه‌های هدف به‌صورت پویا و بر اساس ساختار واقعی مدل شناسایی شدند.

تابع find_text_transformer_layers در میان ماژول‌های مدل به دنبال یک ModuleList می‌گردد که سه ویژگی اصلی داشته باشد: در مسیر language_model قرار گرفته باشد، تعداد اعضای آن با تعداد لایه‌های پنهان ثبت‌شده در text_config برابر باشد و هر عضو آن دارای اجزای self_attn و mlp باشد. این شرایط باعث می‌شوند فهرست به‌دست‌آمده دقیقاً متناظر با بلوک‌های Transformer بخش زبانی باشد.

در اجرای نهایی، مسیر لایه‌های متنی به‌صورت زیر شناسایی شد:

model.language_model.layers

این مسیر شامل ۳۰ بلوک Transformer بود. همچنین تابع به‌گونه‌ای طراحی شد که وجود دقیقاً یک فهرست معتبر را الزامی می‌کند. اگر هیچ ساختار مناسبی یافت نمی‌شد یا چند فهرست هم‌زمان با شرایط موردنظر مطابقت داشتند، اجرای برنامه متوقف می‌شد. این کنترل مانع از آن می‌شود که تزریق Adapter بر اساس حدس درباره ساختار داخلی مدل انجام گیرد.

پس از شناسایی بلوک‌های متنی، تابع inject_adapters ابتدا تمام پارامترهای موجود در مدل را فریز می‌کند:

$$requires_{grad(\theta_{base})} = False$$

سپس برای هر بلوک، MLP اصلی از layer.mlp دریافت و داخل کلاس MLPWithAdapter قرار داده می‌شود. این Wrapper ابتدا MLP اصلی را اجرا می‌کند و خروجی آن را با نماد h در اختیار Bottleneck Adapter قرار می‌دهد. خروجی نهایی Wrapper مطابق رابطه زیر ساخته می‌شود:

$$h = MLP(x)$$
$$y = h + Adapter(h)$$

در نتیجه، MLP اصلی حذف یا بازنویسی نمی‌شود و همان وزن‌های پیش‌آموزش‌دیده را حفظ می‌کند. تنها یک مسیر قابل‌آموزش پس از خروجی آن افزوده می‌شود. این ساختار دقیقاً قرارگیری متوالی یا Sequential موردنظر تمرین را پیاده‌سازی می‌کند.

در زمان ساخت Adapter، دستگاه محاسباتی آن با استفاده از یکی از پارامترهای MLP اصلی تعیین شد؛ اما برخلاف وزن‌های ثابت مدل پایه، پارامترهای Adapter عمداً با نوع داده float32 روی همان GPU قرار گرفتند. وزن‌های مدل پایه در float16 باقی ماندند. همچنین، محاسبات داخلی شاخه Adapter خارج از Autocast و با ورودی تبدیل‌شده به float32 انجام شدند. در پایان، خروجی Adapter دوباره به نوع داده خروجی MLP تبدیل شد و سپس اتصال Residual اجرا گردید. این طراحی برای جلوگیری از Overflow و تولید مقادیر NaN در شاخه Adapter ضروری بود.

تابع تزریق همچنین بررسی می‌کند که MLP هر لایه از قبل داخل MLPWithAdapter قرار نگرفته باشد. در صورت اجرای دوباره تابع روی یک مدل تزریق‌شده، برنامه متوقف می‌شود. این کنترل از اضافه‌شدن چند Adapter پشت سر هم به یک MLP و تغییر ناخواسته معماری جلوگیری می‌کند.

پس از پایان تزریق، تعداد Adapterهای افزوده‌شده با تعداد بلوک‌های Transformer متنی مقایسه شد. خروجی اجرای نهایی نشان داد که تعداد لایه‌های مورد انتظار و تعداد Adapterهای تزریق‌شده هر دو برابر ۳۰ هستند. همچنین بررسی پارامترهای

قابل‌آموزش نشان داد که فقط Tensorهایی که در نام آن‌ها عبارت .adapter. وجود داشت قابل‌آموزش بودند و هیچ پارامتر غیرمنتظره‌ای از مدل پایه فعال باقی نمانده بود.

در هر Adapter چهار Tensor قابل‌آموزش شامل وزن و بایاس Down-projection و وزن و بایاس Up-projection وجود دارد. بنابراین، برای ۳۰ Adapter تعداد Tensorهای قابل‌آموزش برابر بود با:

$$N_{trainable\ tensors} = 30 \times 4 = 120$$

این کنترل‌ها نشان دادند که Adapter در تمام بلوک‌های موردنظر تزریق شده، هیچ بلوکی جا نیفتاده و هیچ بخش دیگری از مدل به‌صورت ناخواسته وارد آموزش نشده است.

۳-۲-۲- داده‌ها و شرایط آموزش

مدل دارای Bottleneck Adapter دقیقاً تحت شرایط اصلی استفاده‌شده برای LoRA آموزش داده شد تا تفاوت نتایج دو روش ناشی از معماری PEFT باشد، نه تفاوت در داده‌ها یا ابرپارامترهای آموزش. مطابق صورت تمرین، تنها بخش Train اخبار فارسی در فرایند بهینه‌سازی استفاده شد و هیچ نمونه‌ای از ویکی‌پدیا یا مجموعه‌های آزمون وارد محاسبه گرادیان نشد.

مجموعه آموزشی شامل ۱۰۰۰۰ متن خبری بود. متن‌ها با حداکثر طول ۲۵۶ توکن پردازش شدند و Padding به‌صورت پویا در سطح هر Batch انجام گرفت. موقعیت‌های Padding در برچسب‌ها با مقدار ۱۰۰- مشخص شدند تا در محاسبه Cross-Entropy مشارکت نداشته باشند. بخش dev اخبار شامل ۱۰۰۰ نمونه نیز صرفاً برای پایش روند آموزش و بدون محاسبه گرادیان استفاده شد.

تنظیمات آموزش Bottleneck Adapter در جدول ۳-۲ ارائه شده‌اند.

جدول ۳-۲: تنظیمات فرایند آموزش ***Bottleneck Adapter***

تنظیم آموزشی	**مقدار استفاده‌شده**
مجموعه آموزشی	بخش آموزش اخبار فارسی
تعداد نمونه‌های آموزشی	۱۰۰۰۰
مجموعه اعتبارسنجی	بخش dev اخبار فارسی
تعداد نمونه‌های اعتبارسنجی	۱۰۰۰
بُعد میانی Adapter	16
تعداد دورهای آموزش	۳
نرخ یادگیری	2×10^{-4}
طول حداکثر توالی	۲۵۶ توکن

اندازه ریز Batch	۴
مراحل تجمیع گرادیان	۲
اندازه مؤثر Batch	۸
تعداد ریز Batch در هر دور	۲۵۰۰
تعداد به‌روزرسانی بهینه‌ساز در هر دور	۱۲۵۰
تعداد کل به‌روزرسانی‌ها	۳۷۵۰
فاصله Mini-validation	هر ۲۰ ریز Batch آموزشی
اندازه Batch در Mini-validation	۲
اندازه Batch در Full validation	۲
بهینه‌ساز	AdamW
نوع داده مدل پایه	float16
نوع داده پارامترها و محاسبات Adapter	float32
GradScaler	فعال
Gradient Checkpointing	فعال
کوانتیزه‌سازی	استفاده نشد

اندازه ریز Batch آموزش برابر ۴ بود؛ بنابراین در هر دور، ۱۰۰۰۰ نمونه آموزشی در قالب ۲۵۰۰ ریز Batch پردازش شدند. با تجمیع گرادیان در دو مرحله، اندازه مؤثر Batch برابر ۸ شد:

$$B_{effective} = B_{micro} \times N_{accumulation} = 4 \times 2 = 8$$

بهینه‌ساز پس از هر دو ریز Batch یک‌بار به‌روزرسانی می‌شد. ازآنجاکه تعداد ریز Batchهای هر Epoch بر تعداد مراحل تجمیع بخش‌پذیر بود، تعداد به‌روزرسانی‌های هر Epoch برابر بود با:

$$N_{updates/epoch} = \frac{2500}{2} = 1250$$

در نتیجه، طی سه دور آموزش در مجموع ۳۷۵۰ مرحله بهینه‌سازی روی پارامترهای Adapter انجام شد.

به‌کارگیری Gradient Accumulation با محدودیت حافظه GPU ارتباط مستقیم داشت. این تنظیم امکان حفظ Batch مؤثر برابر ۸ را فراهم کرد، بدون آنکه هر هشت توالی با طول حداکثر ۲۵۶ به‌طور هم‌زمان در یک عبور پیش‌رو پردازش شوند. Gradient Checkpointing نیز برای کاهش حافظه فعال‌سازی‌های میانی فعال بود؛ هرچند محاسبه مجدد بخشی از فعال‌سازی‌ها در

پس‌انتشار می‌تواند زمان آموزش را افزایش دهد. Gradient Checkpointing نیز برای کاهش حافظه فعال‌سازی‌های میانی فعال بود؛ هرچند این روش به دلیل محاسبه مجدد بخشی از فعال‌سازی‌ها در پس‌انتشار، زمان اجرای آموزش را افزایش می‌دهد.

وزن‌های ثابت مدل پایه در float16 قرار داشتند، اما پارامترها و محاسبات داخلی Adapter با float32 انجام شدند. برای این منظور، Autocast در شاخه Adapter غیرفعال شد و ورودی MLP پیش از ورود به Down-projection به float32 تبدیل گردید. خروجی Adapter سپس به نوع داده خروجی MLP بازگردانده شد. این طراحی، ضمن حفظ مصرف حافظه کمتر مدل پایه، پایداری عددی شاخه Adapter را افزایش داد. هیچ کوانتیزه‌سازی چهار یا هشت‌بیتی نیز استفاده نشد.

۳-۲-۳- بهینه‌سازی و ثبت روند آموزش

آموزش Bottleneck Adapter با همان تابع عمومی train_model انجام شد که برای LoRA نیز مورد استفاده قرار گرفته بود. استفاده از یک حلقه مشترک، شرایط بهینه‌سازی و نحوه ثبت خروجی‌ها را میان دو روش ثابت نگه می‌دارد. تفاوت اصلی در مجموعه پارامترهای قابل‌آموزش است: در مدل LoRA ماتریس‌های کم‌رتبه و در مدل Adapter وزن‌ها و بایاس‌های ماژول‌های Down–GELU–Up وارد بهینه‌ساز می‌شوند. حلقه آموزش و منطق ثبت شاخص‌ها میان LoRA و Adapter مشترک بود؛ بااین‌حال، برای Full validation مدل Adapter از Batch برابر ۲ استفاده شد تا احتمال بروز OOM کاهش یابد.

در ابتدای تابع، فقط پارامترهایی که requires_grad=True دارند جمع‌آوری می‌شوند:

$$\Theta_{optimizer} = \{\theta | \theta.required_grad = True\}$$

از آنجا که تمام پارامترهای مدل پایه پیش از تزریق فریز شده بودند، فهرست ورودی به AdamW فقط شامل پارامترهای Adapter بود. به این ترتیب، حتی در صورت عبور گرادیان از میان بلوک‌های اصلی مدل، بهینه‌ساز امکان تغییر وزن‌های آن‌ها را نداشت.

برای هر ریزBatch، input_ids، attention_mask و labels به GPU منتقل شدند. مدل در حالت train و با use_cache=False اجرا شد. غیرفعال‌کردن Cache در زمان آموزش برای سازگاری با Gradient Checkpointing و کاهش نگهداری مقادیر اضافی در حافظه ضروری است.

عبور پیش‌روی کلی مدل داخل autocast_context و با float16 انجام شد تا مصرف حافظه کاهش یابد. بااین‌حال، شاخه داخلی Adapter از این Context مستثنا بود و محاسبات Down-projection، GELU و Up-projection در float32 انجام شدند. پس از تبدیل خروجی Adapter به نوع داده خروجی MLP، مدل با استفاده از labels، Cross-Entropy پیش‌بینی توکن بعدی را محاسبه می‌کرد.

زیان خام هر ریزBatch پیش از پس‌انتشار بر دو تقسیم شد:

$$\mathcal{L}_{scaled} = \frac{\mathcal{L}_{raw}}{2}$$

اصطلاح scaled در این رابطه به مقیاس ناشی از Gradient Accumulation اشاره دارد و نباید با GradScaler عددی اشتباه گرفته شود. این تقسیم باعث می‌شود مجموع گرادیان دو ریزBatch، مقیاسی مشابه میانگین گرادیان Batch مؤثر هشت‌نمونه‌ای داشته باشد.

ازآنجاکه عبور پیش‌روی اصلی مدل با float16 انجام می‌شد، GradScaler در اجرای نهایی فعال بود. زیان پس از تقسیم برای Gradient Accumulation توسط Scaler مقیاس‌دهی شد و سپس عملیات پس‌انتشار انجام گرفت. پیش از Gradient Clipping، تابع scaler.unscale_(optimizer) گرادیان‌ها را به مقیاس واقعی بازگرداند. در ادامه، scaler.step(optimizer) به‌روزرسانی پارامترها و scaler.update() تنظیم پویای ضریب مقیاس را انجام دادند.

پس از رسیدن به دومین ریزBatch، گرادیان‌ها پیش از مرحله بهینه‌سازی با حداکثر نرم یک محدود شدند:

$$||\nabla_\theta \mathcal{L}|| \leq 1$$

Gradient Clipping از ایجاد به‌روزرسانی‌های بسیار بزرگ جلوگیری کرده و پایداری حلقه آموزش را افزایش می‌دهد. سپس scaler.step(optimizer) و scaler.update() اجرا شدند و گرادیان‌ها با optimizer.zero_grad(set_to_none=True) پاک شدند. این روش نسبت به صفرکردن صریح Tensorهای گرادیان می‌تواند مدیریت حافظه کارآمدتری داشته باشد.

برای رسم منحنی آموزش، مقدار raw_loss پیش از تقسیم بر تعداد مراحل تجمیع ثبت شد. بنابراین، تاریخچه زیان مقدار واقعی Loss هر ریزBatch را نشان می‌دهد و به‌دلیل Gradient Accumulation به‌صورت مصنوعی نصف نشده است. علاوه بر زیان هر ریزBatch و میانگین زیان هر Epoch، زیان‌های Mini-validation در فاصله‌های ۲۰ ریزBatch و زیان و پرپلکسیتی Full validation پس از پایان هر Epoch نیز ثبت شدند.

میانگین زیان آموزش Adapter در سه Epoch به‌ترتیب برابر با ۱.۸۳۵۵۵۷، ۱.۷۸۶۴۸۴ و ۱.۷۵۰۰۰۷ بود و روندی نزولی داشت. زیان کامل Validation نیز از ۱.۸۰۵۵۸۱ در پایان Epoch نخست به ۱.۷۸۹۱۹۸ و سپس ۱.۷۸۳۸۲۶ کاهش یافت. بنابراین، برخلاف LoRA که زیان Validation آن در Epoch سوم اندکی افزایش یافت، در Adapter تا پایان Epoch سوم نشانه‌ای از برگشت زیان Validation مشاهده نشد. تحلیل کامل روند همگرایی، نوسان Mini-validation و مقایسه آن با LoRA در فصل چهارم انجام خواهد شد.

تابع آموزش علاوه بر تاریخچه زیان، زمان خالص آموزش، زمان صرف‌شده برای Mini-validation و Full validation و Peak VRAM مربوط به آموزش و Validation را به‌صورت جداگانه ثبت کرد. این تفکیک مانع از آن می‌شود که هزینه پایش اعتبارسنجی با هزینه واقعی بهینه‌سازی مخلوط شود. این دو شاخص برای تحلیل هزینه محاسباتی پیاده‌سازی دستی Adapter و مقایسه آن با پیاده‌سازی LoRA در کتابخانه PEFT استفاده خواهند شد.

در مجموع، مدل دارای Bottleneck Adapter با ۳۰ ماژول تزریق‌شده، ۱۲۰ Tensor و ۲,۰۲۸,۰۰۰ پارامتر قابل‌آموزش تحت همان ابرپارامترهای اصلی LoRA آموزش داده شد. تمام ۳۷۵۰ به‌روزرسانی منحصراً از بخش آموزش اخبار فارسی و فقط روی پارامترهای Adapter انجام شدند. بخش اعتبارسنجی اخبار نیز بدون محاسبه گرادیان برای ثبت ۳۷۵ نقطه Mini-validation و

سه ارزیابی کامل در سطح Epoch به کار رفت. مدل نهایی پس از سه Epoch برای ارزیابی پرپلکسیتی روی آزمون‌های دیده‌نشده اخبار و ویکی‌پدیا آماده شد.

۳-۳- ارزیابی مدل تنظیم‌شده با Bottleneck Adapter

پس از پایان سه دور آموزش، مدل دارای Bottleneck Adapter روی مجموعه آزمون اخبار فارسی و مجموعه آزمون ویکی‌پدیا ارزیابی شد. ارزیابی روی اخبار با هدف سنجش میزان سازگاری مدل با دامنه هدف و ارزیابی روی ویکی‌پدیا برای بررسی حفظ عملکرد برون‌دامنه‌ای انجام گرفت. مطابق صورت تمرین، محاسبه پرپلکسیتی پس از آموزش باید روی هر دو مجموعه آزمون انجام و ثبت شود.

برای حفظ قابلیت مقایسه با مدل پایه، همان تابع دستی پرپلکسیتی فصل اول مورد استفاده قرار گرفت. در این روش، زیان فقط برای توکن‌های واقعی محاسبه شد، موقعیت‌های Padding کنار گذاشته شدند و تابع نمایی پس از تجمیع زیان در سطح تمام توکن‌های معتبر اعمال گردید. در ارزیابی نهایی Adapter روی مجموعه‌های آزمون، اندازه Batch برابر ۴ بود. بااین‌حال، برای Full validation پایان هر Epoch اندازه Batch برابر ۲ در نظر گرفته شد تا احتمال بروز خطای OOM کاهش یابد. از آنجا که مجموع زیان و تعداد توکن‌های معتبر در سطح کل مجموعه تجمیع می‌شوند، تغییر اندازه Batch تعریف یا مقدار نظری پرپلکسیتی را عوض نمی‌کند و فقط زمان اجرا و حافظه لحظه‌ای را تحت تأثیر قرار می‌دهد.

۳-۳-۱- نتایج کمی روی مجموعه‌های آزمون

مدل در حالت eval و بدون محاسبه گرادیان اجرا شد. برای هر مجموعه، میانگین Cross-Entropy بر اساس تمام توکن‌های هدف معتبر محاسبه و سپس پرپلکسیتی از نمایی‌کردن این میانگین به دست آمد. نتایج نهایی در جدول ۳-۳ ارائه شده‌اند.

*جدول ۳-۳: نتایج ارزیابی مدل تنظیم‌شده با **Bottleneck Adapter** روی مجموعه‌های آزمون اخبار و ویکی‌پدیا*

مجموعه آزمون	نوع دامنه	تعداد توکن معتبر	میانگین Cross-Entropy	پرپلکسیتی
اخبار فارسی	درون‌دامنه	۲۵۱,۷۴۵	۱.۷۸۷۳۷۰	۵.۹۷۳۷۲۳
ویکی‌پدیا	برون‌دامنه	۱,۲۹۲,۶۱۱	۲.۱۷۴۶۵۹	۸.۷۹۹۱۸۷

روی آزمون اخبار فارسی، مدل به میانگین Cross-Entropy برابر ۱.۷۸۷۳۷۰ و پرپلکسیتی برابر ۵.۹۷۳۷۲۳ دست یافت. نمونه‌های این مجموعه در فرایند آموزش یا اعتبارسنجی برای به‌روزرسانی پارامترها استفاده نشده بودند؛ بنابراین، نتیجه ثبت‌شده عملکرد مدل روی متون خبری دیده‌نشده از همان دامنه آموزشی را نشان می‌دهد. نمونه‌های این مجموعه در فرایند آموزش مشاهده نشده بودند؛ بنابراین، نتیجه ثبت‌شده عملکرد مدل روی متون خبری جدید از همان دامنه آموزشی را نشان می‌دهد.

در آزمون ویکی‌پدیا، میانگین Cross-Entropy برابر ۲.۱۷۴۶۵۹ و پرپلکسیتی برابر ۸.۷۹۹۱۸۷ به دست آمد. این داده‌ها در هیچ مرحله‌ای از آموزش، اعتبارسنجی یا تنظیم پارامترهای Adapter استفاده نشده بودند و به همین دلیل، نتیجه آن‌ها معیاری برای

سنجش اثر تنظیم دقیق خبری بر عملکرد عمومی مدل محسوب می‌شود. این داده‌ها در هیچ مرحله‌ای از آموزش Adapter استفاده نشده بودند و به همین دلیل، نتیجه آن‌ها معیاری برای سنجش اثر تنظیم دقیق خبری بر رفتار عمومی مدل محسوب می‌شود.

تعداد توکن‌های معتبر دو مجموعه دقیقاً با ارزیابی مدل پایه یکسان باقی ماند. این موضوع نشان می‌دهد تفاوت نتایج از تغییر مجموعه داده، روش توکن‌سازی یا قواعد حذف Padding ناشی نشده است. در نتیجه، مقایسه مدل پایه و Adapter تحت شرایط ارزیابی یکسان انجام شده و تغییرات مشاهده‌شده را می‌توان به اثر پارامترهای آموخته‌شده در Bottleneck Adapter نسبت داد.

۳-۳-۲- تحلیل عملکرد درون‌دامنه و برون‌دامنه

برای بررسی اثر تنظیم دقیق، نتیجه Adapter در هر دامنه با خط پایه متناظر همان مجموعه مقایسه می‌شود. پرپلکسیتی مدل پایه روی اخبار برابر ۷.۷۰۹۴۳۹ بود که پس از آموزش Adapter به ۵.۹۷۳۷۲۳ کاهش یافت. کاهش مطلق پرپلکسیتی اخبار برابر است با:

$$\Delta PPL_{News} = 7.709439 - 5.973723 = 1.735716$$

بهبود نسبی عملکرد درون‌دامنه‌ای نیز از رابطه زیر به دست می‌آید:

$$Improvement_{News} = \frac{7.709439 - 5.973723}{7.709439} \times 100 \approx 22.51\%$$

بنابراین، آموزش Bottleneck Adapter باعث کاهش حدود ۲۲.۵۱ درصدی پرپلکسیتی روی اخبار دیده‌نشده شده است. این کاهش نشان می‌دهد مدل پس از تنظیم دقیق، به‌طور متوسط احتمال بیشتری به توکن‌های واقعی متون خبری اختصاص می‌دهد و عدم‌اطمینان آن در پیش‌بینی ادامه متن‌های این حوزه کاهش یافته است. این کاهش نشان می‌دهد مدل پس از تنظیم دقیق، به‌طور متوسط احتمال بیشتری به توکن‌های واقعی متون خبری اختصاص می‌دهد و عدم‌اطمینان آن در پیش‌بینی ادامه متن‌های این حوزه کاهش یافته است.

در مقیاس لگاریتمی نیز میانگین Cross-Entropy اخبار از ۲.۰۴۲۴۴۵ در مدل پایه به ۱.۷۸۷۳۷۰ در مدل Adapter کاهش یافته است:

$$\Delta \mathcal{L}_{News} = 1.787370 - 2.042445 = -0.255075$$

کاهش زیان روی مجموعه آزمون اهمیت بیشتری از کاهش زیان آموزش دارد؛ زیرا مجموعه آزمون در فرایند بهینه‌سازی حضور نداشته است. بنابراین، نتیجه به‌دست‌آمده نشان می‌دهد Adapterها تنها ویژگی‌های نمونه‌های آموزشی را حفظ نکرده‌اند و بخشی از الگوهای عمومی دامنه اخبار را به متون دیده‌نشده منتقل کرده‌اند.

از نظر معماری، این بهبود نشان می‌دهد فضای میانی ۱۶بعدی، با وجود ظرفیت محدود، برای یادگیری یک اصلاح مؤثر روی خروجی MLPها کافی بوده است. هر Adapter بازنمایی ۲۰۴۸بعدی را به فضای ۱۶بعدی منتقل کرده و سپس یک مؤلفه اصلاحی را به خروجی اصلی MLP افزوده است. بنابراین، حتی یک مسیر بسیار کوچک نسبت به مدل پایه توانسته توزیع خروجی مدل را در جهت سازگارتر با اخبار تغییر دهد.

این سازگاری می‌تواند بازتاب یادگیری الگوهایی مانند ساختار رسمی جمله‌های خبری، نحوه استفاده از افعال گزارش‌دهی، ترکیب‌های متداول مربوط به نهادها و رویدادها، روابط زمانی و ادامه‌های محتمل در متون خبری باشد. بااین‌حال، پرپلکسیتی به‌تنهایی مشخص نمی‌کند دقیقاً کدام ویژگی زبانی آموخته شده است؛ بلکه تنها نشان می‌دهد مدل در سطح پیش‌بینی توکن بعدی، متون خبری آزمون را بهتر مدل‌سازی می‌کند.

در حوزه ویکی‌پدیا، جهت تغییر متفاوت بود. پرپلکسیتی مدل پایه برابر ۷.۳۳۸۸۶۸ بود و پس از آموزش Adapter به ۸.۷۹۹۱۸۷ افزایش یافت. تغییر مطلق پرپلکسیتی ویکی‌پدیا برابر است با:

$$\Delta PPL_{Wiki} = 8.799187 - 7.338868 = 1.460319$$

میزان افزایش نسبی نیز برابر است با:

$$Degradation_{Wiki} = \frac{8.799187 - 7.338868}{7.338868} \times 100 \approx 19.90\%$$

در مقیاس Cross-Entropy نیز زیان ویکی‌پدیا از ۱.۹۹۳۱۸۵ به ۲.۱۷۴۶۵۹ افزایش یافته است:

$$\Delta \mathcal{L}_{Wiki} = 2.174659 - 1.993185 = 0.181474$$

این افزایش نشان می‌دهد مدل تنظیم‌شده، به‌طور متوسط احتمال کمتری به توکن‌های صحیح متون عمومی ویکی‌پدیا اختصاص داده است. به بیان دیگر، تغییرات آموخته‌شده برای دامنه خبری در تمام ورودی‌ها فعال هستند و لزوماً برای توزیع متون عمومی مناسب نیستند.

ممکن است در نگاه نخست تصور شود که فریزکردن کامل وزن‌های مدل پایه باید عملکرد قبلی آن را بدون تغییر حفظ کند؛ اما این برداشت دقیق نیست. وزن‌های اصلی تغییر نکرده‌اند، ولی خروجی Adapter پس از هر MLP به مسیر اصلی افزوده می‌شود:

$$y = h + Adapter(h)$$

پس از آموزش، مقدار Adapter(h) دیگر صفر نیست و برای ورودی‌های خبری و غیرخبری تولید می‌شود. در نتیجه، این مؤلفه می‌تواند بازنمایی‌های میانی مدل و در نهایت توزیع احتمال خروجی را روی ویکی‌پدیا نیز تغییر دهد.

نسبت پرپلکسیتی Adapter به مدل پایه در هر دو دامنه، جهت این تغییرات را روشن‌تر نشان می‌دهد:

$$\frac{PPL_{News}^{Adapter}}{PPL_{News}^{Base}} = \frac{5.973723}{7.709439} \approx 0.7749$$

$$\frac{PPL_{Wiki}^{Adapter}}{PPL_{Wiki}^{Base}} = \frac{8.799187}{7.338868} \approx 1.1990$$

نسبت نخست کمتر از یک است و کاهش عدم‌اطمینان مدل در حوزه اخبار را نشان می‌دهد. نسبت دوم بیشتر از یک است و بیانگر افزایش عدم‌اطمینان در حوزه ویکی‌پدیا است. بنابراین، Bottleneck Adapter رفتار مدل را به‌صورت مشخص به سمت توزیع داده‌های خبری منتقل کرده است.

از منظر طراحی آزمایش، این رفتار قابل‌انتظار است؛ زیرا تابع هدف آموزش فقط زیان اخبار را کمینه می‌کرد و هیچ مؤلفه‌ای برای حفظ مستقیم خروجی‌های مدل پایه یا عملکرد ویکی‌پدیا وجود نداشت. مدل در هر مرحله به سمتی به‌روزرسانی شد که پیش‌بینی متون خبری بهتر شود، حتی اگر این جهت برای داده‌های عمومی بهینه نباشد.

مقداردهی اولیه صفر Up-projection باعث شد تزریق Adapter پیش از آموزش رفتار مدل پایه را تغییر ندهد، اما این ویژگی فقط نقطه شروع خنثی را تضمین می‌کند. پس از انجام به‌روزرسانی‌ها، Adapterها اصلاحاتی غیرصفر یاد می‌گیرند و Residual بودن ساختار نیز مانع کامل افت برون‌دامنه‌ای نمی‌شود. نقش Residual آن است که مسیر اصلی مدل را حفظ کند و تغییرات را به‌صورت افزایشی اعمال نماید، نه اینکه اثر Adapter را در دامنه‌های دیگر حذف کند.

از نظر مهندسی، نتایج نشان‌دهنده یک مصالحه میان تخصص‌یافتگی و حفظ عملکرد عمومی هستند. پرپلکسیتی اخبار حدود ۲۲.۵۱ درصد کاهش یافت، درحالی‌که پرپلکسیتی ویکی‌پدیا حدود ۱۹.۹۰ درصد افزایش پیدا کرد. بنابراین، Adapter در دامنه هدف بهبود قابل‌توجهی ایجاد کرد، اما این سازگارسازی همچنان با هزینه‌ای قابل‌اندازه‌گیری در عملکرد برون‌دامنه‌ای همراه بود.

افزایش پرپلکسیتی ویکی‌پدیا شواهدی از تداخل دامنه‌ای و کاهش بخشی از عملکرد عمومی مدل فراهم می‌کند. Degradation Ratio مدل Adapter برابر با ۰.۱۹۸۹۸۴، معادل حدود ۱۹.۹۰ درصد، بود. این مقدار نشان می‌دهد افت برون‌دامنه‌ای Adapter قابل‌توجه است، اما شدت آن از افت ثبت‌شده برای LoRA کمتر است. قضاوت نهایی درباره مصالحه دو روش باید با درنظرگرفتن هم‌زمان بهبود اخبار، روند Validation، هزینه محاسباتی و خروجی‌های کیفی در فصل چهارم انجام شود.

در مجموع، Bottleneck Adapter با آموزش تنها ۲,۰۲۸,۰۰۰ پارامتر، پرپلکسیتی اخبار فارسی را از ۷.۷۰۹۴۳۹ به ۵.۹۷۳۷۲۳ کاهش داد که معادل بهبودی حدود ۲۲.۵۱ درصد است. این بهبود به نمونه‌های خبری دیده‌نشده نیز منتقل شد و نشان داد ساختار Down–GELU–Up با بُعد میانی ۱۶ ظرفیت لازم برای سازگارسازی دامنه‌ای را داشته است. در مقابل، پرپلکسیتی ویکی‌پدیا از ۷.۳۳۸۸۶۸ به ۸.۷۹۹۱۸۷ افزایش یافت که افتی حدود ۱۹.۹۰ درصدی را نشان می‌دهد. بنابراین، ارزیابی موفقیت این روش مستلزم درنظرگرفتن هم‌زمان بهبود در دامنه هدف و هزینه ایجادشده در عملکرد برون‌دامنه‌ای است.

۳-۴- نکات پیاده‌سازی و ملاحظات عملی

در این بخش، مهم‌ترین قسمت‌های اختصاصی پیاده‌سازی دستی Bottleneck Adapter بررسی می‌شوند. برخلاف روش LoRA که بخش اصلی تغییر معماری با استفاده از کتابخانه PEFT انجام شد، در این روش ساختار Adapter، اتصال آن به MLP و تزریق آن در بلوک‌های Transformer مستقیماً با PyTorch پیاده‌سازی شده است. مطابق صورت تمرین، این پیاده‌سازی باید بدون استفاده از کتابخانه‌های آماده انجام شود، Adapter پس از خروجی MLP قرار گیرد و خروجی آن به‌صورت Residual با ورودی Adapter جمع شود.

کدهای این بخش در دو قسمت ارائه می‌شوند. ابتدا کلاس‌های مسئول تعریف معماری Bottleneck Adapter و اتصال آن به MLP بررسی می‌شوند. سپس نحوه شناسایی بلوک‌های متنی مدل، فریزکردن پارامترهای پایه و تزریق Adapter در تمام MLPهای موردنظر توضیح داده خواهد شد.

۳-۴-۱- پیاده‌سازی Bottleneck Adapter و اتصال آن به MLP

ساختار اصلی روش با دو کلاس BottleneckAdapter و MLPWithAdapter پیاده‌سازی شده است. کلاس نخست مسیر Down–GELU–Up را تعریف می‌کند و کلاس دوم وظیفه قرار دادن این مسیر پس از خروجی MLP اصلی را بر عهده دارد.

```python
class BottleneckAdapter(nn.Module):
    def __init__(self, hidden_size, adapter_dim):
        super().__init__()

        self.down = nn.Linear(hidden_size, adapter_dim)
        self.activation = nn.GELU()
        self.up = nn.Linear(adapter_dim, hidden_size)

        # Stable initialization
        nn.init.xavier_uniform_(self.down.weight)
        nn.init.zeros_(self.down.bias)

        # Adapter contribution starts from zero
        nn.init.zeros_(self.up.weight)
        nn.init.zeros_(self.up.bias)

    def forward(self, hidden_states):
        output_dtype = hidden_states.dtype

        if hidden_states.is_cuda:
            adapter_context = torch.autocast(device_type="cuda", enabled=False)
        else:
            adapter_context = nullcontext()

        with adapter_context:
            adapter_hidden = hidden_states.float()
            adapter_hidden = self.down(adapter_hidden)
            adapter_hidden = self.activation(adapter_hidden)
            adapter_output = self.up(adapter_hidden)

        return adapter_output.to(dtype=output_dtype)


class MLPWithAdapter(nn.Module):
    def __init__(self, original_mlp, hidden_size, adapter_dim):
        super().__init__()

        self.original_mlp = original_mlp
        self.adapter = BottleneckAdapter(
            hidden_size=hidden_size,
            adapter_dim=adapter_dim,
        )

    def forward(self, hidden_states, *args, **kwargs):
        mlp_output = self.original_mlp(hidden_states, *args, **kwargs)
        adapter_output = self.adapter(mlp_output)

        return mlp_output + adapter_output
```

*شکل ۳–۱: پیاده‌سازی کلاس‌های **BottleneckAdapter** و **MLPWithAdapter** برای افزودن مسیر **Down–GELU–Up** پس از خروجی **MLP***

در کلاس BottleneckAdapter، لایه down بازنمایی ورودی را از بُعد پنهان مدل به بُعد میانی Adapter کاهش می‌دهد. سپس تابع فعال‌ساز GELU روی بازنمایی فشرده‌شده اعمال می‌شود و لایه up آن را دوباره به بُعد اصلی مدل بازمی‌گرداند. بنابراین، تبدیل انجام‌شده توسط این کلاس را می‌توان به‌صورت زیر بیان کرد:

$$Adapter(h) = Up\left(GELU\left(Down(h)\right)\right)$$

در زمان ایجاد Adapter، وزن لایه down با روش Xavier Uniform مقداردهی اولیه و بایاس آن برابر صفر قرار داده شد. در مقابل، وزن و بایاس لایه up با مقدار صفر مقداردهی شدند. بنابراین، با وجود غیرصفر بودن خروجی Down-projection، خروجی نهایی Adapter پیش از شروع آموزش دقیقاً برابر صفر است. در نتیجه، افزودن Adapter در نقطه آغاز رفتار مدل پایه را تغییر نمی‌دهد و تغییرات موردنیاز دامنه خبری به‌تدریج در جریان بهینه‌سازی شکل می‌گیرند.

کلاس MLPWithAdapter یک Wrapper برای MLP اصلی هر بلوک Transformer است. این کلاس MLP پیش‌آموزش‌دیده را در متغیر original_mlp نگهداری می‌کند و یک نمونه از BottleneckAdapter را در کنار آن می‌سازد. در عبور پیش‌رو، ابتدا خروجی MLP اصلی محاسبه می‌شود:

$h = MLP(x)$

سپس همین خروجی وارد Adapter شده و نتیجه آن به‌صورت Residual با خروجی اصلی جمع می‌شود:

$y = h + Adapter(h)$

در نتیجه، Adapter جایگزین MLP اصلی نمی‌شود و مسیر پیش‌آموزش‌دیده مدل همچنان حفظ می‌گردد. این ماژول تنها یک مؤلفه اصلاحی قابل‌آموزش به خروجی آن اضافه می‌کند. همچنین استفاده از *args و **kwargs در متد forward باعث می‌شود هر آرگومان اضافی موردنیاز MLP اصلی بدون تغییر به آن منتقل شود و Wrapper با شیوه فراخوانی داخلی مدل سازگار باقی بماند.

یکی از ملاحظات مهم این Wrapper، مدیریت جداگانه نوع داده شاخه اصلی و شاخه Adapter است. خروجی MLP اصلی در نوع داده محاسباتی مدل، یعنی float16، تولید می‌شود؛ اما محاسبات Down-projection، GELU و Up-projection در Adapter با float32 انجام می‌شوند. برای این منظور، Autocast در داخل شاخه Adapter غیرفعال شده و خروجی MLP پیش از ورود به Adapter به float32 تبدیل می‌شود.

پس از محاسبه خروجی Adapter، نتیجه دوباره به نوع داده خروجی MLP تبدیل می‌شود و سپس جمع Residual انجام می‌گیرد. این تبدیل ضروری است؛ زیرا دو Tensor شرکت‌کننده در اتصال Residual باید از نظر دستگاه و نوع داده سازگار باشند. اجرای داخلی Adapter با float32 نیز برای جلوگیری از Overflow و ایجاد مقادیر NaN در این شاخه در نظر گرفته شد، بدون آنکه لازم باشد کل مدل پایه به float32 تبدیل شود.

۳-۴-۲- شناسایی لایه‌های متنی و تزریق دستی Adapter

برای تزریق صحیح Adapter، ابتدا باید فهرست دقیق بلوک‌های Transformer بخش متنی مدل شناسایی شود. این عملیات با تابع find_text_transformer_layers انجام شده و سپس تابع inject_adapters فریزکردن مدل و جایگزینی MLPها را مدیریت می‌کند.

```python
def find_text_transformer_layers(model):
    expected_layer_count = model.config.text_config.num_hidden_layers
    candidates = []

    for name, module in model.named_modules():
        if not isinstance(module, nn.ModuleList):
            continue

        if "language_model" not in name.split("."):
            continue

        if len(module) != expected_layer_count:
            continue

        if all(hasattr(layer, "self_attn") and hasattr(layer, "mlp") for layer in module):
            candidates.append((name, module))

    if len(candidates) != 1:
        candidate_summary = [f"{name}: {len(module)} layers" for name, module in candidates]

        raise RuntimeError(
            "Expected exactly one text transformer "
            f"layer list, but found: {candidate_summary}"
        )

    layer_path, layers = candidates[0]

    print("Text-layer path:", layer_path)
    print("Text-layer count:", len(layers))

    return layers
```

*شکل ۳–۲: پیاده‌سازی تابع **find_text_transformer_layers** برای شناسایی بلوک‌های **Transformer** بخش متنی مدل*

تابع find_text_transformer_layers تعداد مورد انتظار لایه‌های متنی را از مقدار num_hidden_layers در text_config دریافت می‌کند. سپس با پیمایش خروجی named_modules()، ساختارهایی را بررسی می‌کند که از نوع nn.ModuleList هستند.

برای آنکه یک ساختار به‌عنوان فهرست بلوک‌های متنی پذیرفته شود، سه شرط اصلی در نظر گرفته شده است. مسیر آن باید در بخش language_model قرار داشته باشد، تعداد اعضای آن با تعداد لایه‌های متنی ثبت‌شده در پیکربندی برابر باشد و همه اعضای آن دارای مؤلفه‌های self_attn و mlp باشند. استفاده هم‌زمان از این شرایط، احتمال انتخاب اشتباه ماژول‌های متعلق به سایر قسمت‌های معماری را کاهش می‌دهد.

تابع وجود دقیقاً یک فهرست معتبر را الزامی می‌کند. اگر هیچ گزینه‌ای پیدا نشود یا چند ساختار مختلف با شرایط موردنظر مطابقت داشته باشند، اجرای برنامه با خطا متوقف می‌شود. این کنترل باعث می‌شود تزریق Adapter تنها زمانی انجام شود که محل لایه‌های متنی با اطمینان شناسایی شده باشد. پس از شناسایی نیز مسیر و تعداد لایه‌ها چاپ شده و فهرست بلوک‌های Transformer بازگردانده می‌شود.

پس از تعیین لایه‌های هدف، عملیات اصلی تزریق با تابع inject_adapters انجام می‌شود.

```python
def inject_adapters(model, adapter_dim):
    # Freeze all original model parameters
    for parameter in model.parameters():
        parameter.requires_grad = False

    # Find the text transformer layers dynamically
    layers = find_text_transformer_layers(model)

    # Read hidden size from the model configuration
    hidden_size = model.config.text_config.hidden_size

    injected_count = 0

    for layer in layers:
        # Prevent accidental double injection
        if isinstance(layer.mlp, MLPWithAdapter):
            raise RuntimeError("Adapters have already been injected.")

        original_mlp = layer.mlp

        # Use the same device and dtype as the original MLP
        reference_parameter = next(original_mlp.parameters())

        wrapped_mlp = MLPWithAdapter(
            original_mlp=original_mlp,
            hidden_size=hidden_size,
            adapter_dim=adapter_dim,
        )

        wrapped_mlp.adapter.to(
            device=reference_parameter.device,
            dtype=torch.float32,
        )

        layer.mlp = wrapped_mlp
        injected_count += 1

    return injected_count
```

*شکل ۳–۳: پیاده‌سازی تابع **inject_adapters** برای فریز مدل پایه و تزریق **Bottleneck Adapter** پس از **MLP** هر بلوک*

در ابتدای این تابع، ویژگی requires_grad تمام پارامترهای موجود در مدل برابر False قرار می‌گیرد. بنابراین، پارامترهای مدل پایه، شامل وزن‌های Attention، MLPهای اصلی و سایر اجزای پیش‌آموزش‌دیده، در جریان آموزش ثابت باقی می‌مانند.

پس از فریزکردن مدل، تابع find_text_transformer_layers برای دریافت بلوک‌های متنی فراخوانی می‌شود و بُعد پنهان نیز از text_config.hidden_size خوانده می‌شود. سپس MLP هر بلوک به‌صورت جداگانه بررسی می‌گردد. اگر MLP از قبل از نوع MLPWithAdapter باشد، اجرای برنامه متوقف می‌شود تا از تزریق چندباره Adapter در یک لایه جلوگیری شود.

برای هر بلوک، ابتدا یکی از پارامترهای MLP اصلی به‌عنوان مرجع دریافت می‌شود تا دستگاه محاسباتی مدل مشخص گردد. سپس یک نمونه از MLPWithAdapter ساخته می‌شود که MLP اصلی و Bottleneck Adapter جدید را در خود نگه می‌دارد. ازآنجاکه Adapter پس از فریزکردن مدل ساخته می‌شود، پارامترهای جدید آن به‌طور پیش‌فرض قابل‌آموزش باقی می‌مانند؛ درحالی‌که پارامترهای original_mlp همچنان فریز هستند.

Adapter جدید روی همان device پارامترهای MLP اصلی، اما با نوع داده float32 قرار گرفت. بنابراین، وزن‌های ثابت مدل پایه در float16 باقی ماندند، درحالی‌که پارامترهای قابل‌آموزش Adapter با دقت float32 نگهداری و به‌روزرسانی شدند. این

تفکیک باعث شد بدون تبدیل کل مدل به float32، پایداری عددی بیشتری در شاخه Adapter فراهم شود. پس از آن، MLP قبلی هر لایه با Wrapper جدید جایگزین شد:

$$layer.mlp \leftarrow MLPWithAdapter(original_mlp, adapter)$$

در پایان هر جایگزینی، شمارنده injected_count افزایش می‌یابد و تعداد کل Adapterهای تزریق‌شده بازگردانده می‌شود. این خروجی امکان کنترل تطابق تعداد Adapterها با تعداد بلوک‌های متنی مدل را فراهم می‌کند.

در اجرای نهایی، مقدار injected_count برابر ۳۰ بود و با تعداد بلوک‌های Transformer بخش متنی تطابق داشت. همچنین، ۱۲۰ Tensor قابل‌آموزش شناسایی شد که همگی به زیرماژول‌های Adapter تعلق داشتند. به این ترتیب، هم پوشش کامل لایه‌های هدف و هم فریزماندن اجزای اصلی مدل بررسی شد.

در مجموع، این پیاده‌سازی تضمین می‌کند که تزریق فقط در بخش متنی مدل انجام شود، MLPهای اصلی بدون تغییر باقی بمانند، از تزریق تکراری جلوگیری شود و تنها پارامترهای Bottleneck Adapterها قابل‌آموزش باشند. همچنین، اجرای شاخه Adapter در float32 و بازگرداندن خروجی آن به نوع داده MLP پیش از اتصال Residual، سازگاری نوع داده و پایداری عددی پیاده‌سازی را حفظ می‌کند. به این ترتیب، ساختار موردنیاز تمرین به‌صورت دستی و بدون وابستگی به کتابخانه‌های آماده Adapter در معماری مدل اعمال شده است.

۳-۵- جمع‌بندی فصل

در این فصل، روش Bottleneck Adapter به‌صورت کاملاً دستی در معماری مدل پیاده‌سازی شد و بدون استفاده از کتابخانه‌های آماده، ماژول‌های Down–GELU–Up پس از MLP تمام ۳۰ بلوک متنی قرار گرفتند. مدل پایه در طول آموزش ثابت باقی ماند و تنها ۲,۰۲۸,۰۰۰ پارامتر Adapter، معادل حدود ۰.۰۳۷۴۲۸ درصد از پارامترهای مدل پایه، به‌روزرسانی شدند. این نتیجه نشان داد که می‌توان با افزودن مجموعه کوچکی از پارامترهای تخصصی، رفتار یک مدل زبانی بزرگ را برای یک دامنه مشخص تغییر داد، بدون آنکه به تنظیم دقیق کامل وزن‌های آن نیاز باشد.

ارزیابی مدل روی داده‌های خبری دیده‌نشده نشان داد که Adapterها توانسته‌اند اطلاعات آموخته‌شده از مجموعه آموزش را به نمونه‌های جدید همان دامنه منتقل کنند. پرپلکسیتی اخبار از ۷.۷۰۹۴۳۹ در مدل پایه به ۵.۹۷۳۷۲۳ کاهش یافت که معادل بهبودی حدود ۲۲.۵۱ درصد است. همچنین، کاهش پیوسته زیان کامل Validation تا پایان Epoch سوم نشان داد که روند یادگیری Adapter در این بازه همچنان پایدار بوده است. این نتایج بیانگر آن‌اند که فضای میانی ۱۶بعدی Adapter، با وجود ظرفیت محدود، برای یادگیری اصلاحات مؤثر در بازنمایی‌های مدل کافی بوده است.

در مقابل، تخصصی‌شدن مدل برای اخبار با کاهش عملکرد آن روی متون عمومی ویکی‌پدیا همراه شد. پرپلکسیتی ویکی‌پدیا از ۷.۳۳۸۸۶۸ در مدل پایه به ۸.۷۹۹۱۸۷ افزایش یافت که نشان‌دهنده افتی حدود ۱۹.۹۰ درصدی در عملکرد برون‌دامنه‌ای است. این رفتار نشان می‌دهد که ثابت‌بودن وزن‌های مدل پایه لزوماً به معنای ثابت‌ماندن خروجی نهایی آن نیست. Adapterهای آموزش‌دیده در زمان استنتاج روی تمام ورودی‌ها فعال هستند و اصلاحات آموخته‌شده برای متون خبری ممکن است با توزیع

متون عمومی سازگار نباشند. بنابراین، اتصال Residual مسیر اصلی مدل را حفظ می‌کند، اما اثر تغییرات جدید را در دامنه‌های دیگر به‌طور کامل خنثی نمی‌سازد.

از نظر پیاده‌سازی، شناسایی پویا و کنترل‌شده لایه‌های متنی، جلوگیری از تزریق چندباره، قرار دادن Adapterها روی همان دستگاه مدل، اجرای شاخه Adapter در float32، بازگرداندن خروجی آن به نوع داده خروجی MLP و کنترل فریزماندن پارامترهای پایه، از مهم‌ترین عوامل صحت و پایداری اجرای این روش بودند. مدل پایه در float16 باقی ماند، درحالی‌که پارامترها و محاسبات داخلی Adapter با float32 انجام شدند تا احتمال Overflow و تولید مقادیر NaN کاهش یابد. این موارد نشان می‌دهند که در پیاده‌سازی دستی Adapter، تنها تعریف معماری Down–GELU–Up کافی نیست و محل دقیق تزریق، نوع داده محاسبات و مدیریت پارامترهای قابل‌آموزش نیز نقش تعیین‌کننده‌ای دارند.

در مجموع، Bottleneck Adapter با آموزش تنها ۲٬۰۲۸٬۰۰۰ پارامتر توانست پرپلکسیتی اخبار فارسی را حدود ۲۲.۵۱ درصد کاهش دهد، درحالی‌که پرپلکسیتی ویکی‌پدیا حدود ۱۹.۹۰ درصد افزایش یافت. بنابراین، نتایج یک مصالحه روشن میان بهبود عملکرد درون‌دامنه‌ای و حفظ عملکرد عمومی مدل را نشان دادند. برای قضاوت نهایی درباره کارآمدی این روش، باید میزان این بهبود و افت در کنار تعداد پارامترها، روند Validation، زمان آموزش، مصرف حافظه، کیفیت تولید متن و نتایج LoRA بررسی شود که تحلیل جامع آن در فصل چهارم ارائه خواهد شد.

فصل چهارم

ارزیابی نهایی و مقایسه روش‌های LoRA و Bottleneck Adapter

در فصل‌های پیشین، ابتدا عملکرد مدل پایه اندازه‌گیری شد و سپس دو روش LoRA و Bottleneck Adapter به‌صورت مستقل برای سازگارکردن مدل با اخبار فارسی پیاده‌سازی و ارزیابی شدند. نتایج هر روش نشان داد که تنظیم دقیق پارامترکارا می‌تواند عملکرد مدل را در دامنه هدف بهبود دهد، اما این تخصصی‌سازی ممکن است با کاهش عملکرد آن روی متون عمومی همراه باشد. ازاین‌رو، بررسی جداگانه هر روش برای قضاوت نهایی کافی نیست و لازم است همه نتایج در یک چارچوب مشترک مقایسه شوند.

در این فصل، مدل پایه، مدل تنظیم‌شده با LoRA و مدل دارای Bottleneck Adapter از چند جنبه بررسی می‌شوند. این مقایسه شامل تعداد پارامترهای قابل‌آموزش، پرپلکسیتی درون‌دامنه‌ای و برون‌دامنه‌ای، نرخ افت عملکرد روی ویکی‌پدیا، کیفیت تولید متن، زمان خالص آموزش، زمان اعتبارسنجی، بیشینه حافظه تخصیص‌یافته GPU و روند زیان آموزش و Validation است. هدف این تحلیل صرفاً انتخاب مدلی با کمترین پرپلکسیتی اخبار نیست؛ بلکه باید مشخص شود هر روش برای دستیابی به این بهبود چه هزینه‌ای از نظر حفظ عملکرد عمومی و منابع محاسباتی ایجاد کرده است.

در نهایت، نتایج کمی و کیفی در کنار یکدیگر قرار می‌گیرند تا روش دارای مصالحه مناسب‌تر میان یادگیری ویژگی‌های متون خبری، حفظ عملکرد پیشین مدل و پارامترکارایی تعیین شود. این فصل موارد تحلیلی و مقایسه‌ای خواسته‌شده در بخش چهارم تمرین را پوشش می‌دهد.

۴-۱- مقایسه پارامترهای قابل‌آموزش

یکی از اهداف اصلی روش‌های تنظیم دقیق پارامترکارا، کاهش تعداد وزن‌هایی است که در جریان آموزش باید به‌روزرسانی شوند. در تنظیم دقیق کامل، همه یا بخش بزرگی از پارامترهای مدل وارد بهینه‌ساز می‌شوند؛ اما در LoRA و Bottleneck Adapter، وزن‌های مدل پایه ثابت باقی می‌مانند و فقط مجموعه محدودی از پارامترهای افزوده‌شده آموزش داده می‌شوند.

برای انجام مقایسه‌ای دقیق، تعداد پارامترهای قابل‌آموزش مستقیماً از مدل‌های نهایی و بر اساس ویژگی requires_grad شمارش شد. درصد هر روش نیز نسبت به تعداد کل پارامترهای مدل پایه محاسبه گردید. مطابق صورت تمرین، این مقایسه شامل تعداد دقیق پارامترهای قابل‌آموزش LoRA و Adapter و سهم آن‌ها از کل پارامترهای مدل است.

تعداد کل پارامترهای مدل پایه بارگذاری‌شده برابر با ۵,۴۱۸,۴۶۴,۱۹۲ پارامتر بود. این مقدار با شمارش تمام عناصر Tensorهای پارامتری مدل به دست آمد و به‌عنوان مخرج مشترک برای محاسبه درصد پارامترهای قابل‌آموزش استفاده شد:

$N_{base} = 5{,}418{,}464{,}192$

در روش LoRA، تنها ماتریس‌های کم‌رتبه افزوده‌شده به نگاشت‌های q_proj، k_proj، v_proj و o_proj در بلوک‌های Attention قابل‌آموزش بودند. در اجرای نهایی، تعداد ۱۰۰ نگاشت خطی هدف شناسایی شد و تعداد کل پارامترهای قابل‌آموزش LoRA برابر بود با:

$$N_{LoRA} = 5{,}570{,}560$$

درصد پارامترهای قابل‌آموزش LoRA نسبت به مدل پایه از رابطه زیر محاسبه شد:

$$P_{LoRA} = \frac{N_{LoRA}}{N_{base}} \times 100 = \frac{5{,}570{,}560}{5{,}418{,}464{,}192} \times 100 \approx 0.102807\%$$

در روش Bottleneck Adapter، فقط وزن‌ها و بایاس‌های لایه‌های Down-projection و Up-projection در Adapterهای تزریق‌شده قابل‌آموزش بودند. در مجموع، ۳۰ Adapter پس از MLP بلوک‌های متنی تزریق شد و تعداد کل پارامترهای قابل‌آموزش این روش برابر بود با:

$$N_{Adapter} = 2{,}028{,}000$$

درصد آن نسبت به مدل پایه نیز به‌صورت زیر به دست آمد:

$$P_{Adapter} = \frac{N_{Adapter}}{N_{base}} \times 100 = \frac{2{,}028{,}000}{5{,}418{,}464{,}192} \times 100 \approx 0.037428\%$$

نتایج کامل این مقایسه در جدول ۴-۱ ارائه شده‌اند.

جدول ۴–۱: مقایسه تعداد و درصد پارامترهای قابل‌آموزش در روش‌های ***LoRA*** *و* ***Bottleneck Adapter***

روش	تعداد پارامترهای قابل‌آموزش	درصد نسبت به مدل پایه
LoRA	5,570,560	0.102807%
Bottleneck Adapter	2,028,000	0.037428%

هر دو روش تنها بخش بسیار کوچکی از ظرفیت پارامتری مدل را وارد فرایند آموزش کرده‌اند. در LoRA حدود ۰.۱۰۲۸۰۷ درصد و در Bottleneck Adapter حدود ۰.۰۳۷۴۲۸ درصد از پارامترهای مدل پایه قابل‌آموزش بوده‌اند. در نتیجه، حدود ۹۹.۸۹۷۱۹۳ درصد از پارامترهای مدل پایه در LoRA و حدود ۹۹.۹۶۲۵۷۲ درصد در Adapter ثابت باقی مانده‌اند.

۴-۱-۱- تحلیل پارامترکارایی دو روش

نتایج جدول نشان می‌دهند که Bottleneck Adapter در پیکربندی این آزمایش، تعداد پارامترهای قابل‌آموزش کمتری نسبت به LoRA داشته است. این روش ۳,۵۴۲,۵۶۰ پارامتر کمتر از LoRA دارد؛ به‌عبارت دیگر، تعداد پارامترهای قابل‌آموزش آن حدود ۶۳.۵۹ درصد کمتر از LoRA است. از دیدی دیگر، LoRA تقریباً ۲.۷۵ برابر پارامتر قابل‌آموزش بیشتری نسبت به Bottleneck Adapter دارد.

علت این تفاوت به محل و ساختار پارامترهای افزوده‌شده بازمی‌گردد. در LoRA، برای هر نگاشت خطی هدف، دو ماتریس کم‌رتبه ایجاد می‌شود. در اجرای حاضر، این تغییرات روی نگاشت‌های Query، Key، Value و Output در بلوک‌های Attention اعمال شدند. بنابراین، تعداد کل پارامترهای LoRA علاوه بر Rank، به تعداد نگاشت‌های هدف و ابعاد ورودی و خروجی هر نگاشت وابسته است:

$$N_{LoRA} = \sum_{i=1}^{M} r\left(d_{in,i} + d_{out,i}\right)$$

که در آن M تعداد نگاشت‌های خطی هدف و r رتبه LoRA است. در این پیاده‌سازی، ۱۰۰ نگاشت Attention هدف قرار گرفتند و Rank برابر ۱۶ بود. به همین دلیل، با وجود کم‌رتبه‌بودن هر تغییر، مجموع پارامترهای افزوده‌شده در تمام نگاشت‌ها به بیش از پنج میلیون پارامتر رسید.

در Bottleneck Adapter، پس از MLP هر یک از ۳۰ بلوک متنی، یک مسیر Down–GELU–Up با بُعد میانی ۱۶ قرار گرفت. تعداد پارامترهای هر Adapter با بُعد پنهان $d = 2048$ و بُعد میانی $m = 16$ برابر بود با:

$$N_{Adapter\ per\ layer} = dm + m + md + d = 67{,}600$$

بنابراین، برای ۳۰ بلوک متنی داریم:

$$N_{Adapter} = 30 \times 67{,}600 = 2{,}028{,}000$$

این اختلاف نشان می‌دهد که در پیکربندی استفاده‌شده در این آزمایش، Bottleneck Adapter برای ایجاد ظرفیت سازگارسازی جدید از تعداد پارامترهای قابل‌آموزش کمتری نسبت به LoRA استفاده کرده است. این نتیجه به انتخاب بُعد میانی ۱۶، تعداد ۳۰ Adapter، رتبه ۱۶ و تعداد نگاشت‌های هدف LoRA وابسته است و نباید به‌عنوان برتری ذاتی Adapter در همه معماری‌ها و پیکربندی‌ها تعمیم داده شود.

Bottleneck Adapter تغییرات را در یک فضای میانی ۱۶بعدی پس از MLP یاد می‌گیرد، درحالی‌که LoRA به‌طور مستقیم در تعداد بیشتری از نگاشت‌های Attention مداخله می‌کند. بااین‌حال، کمتر بودن تعداد پارامترهای قابل‌آموزش به‌تنهایی برای تعیین روش بهتر کافی نیست. پارامترکارایی باید در کنار میزان بهبود پرپلکسیتی اخبار، افت عملکرد ویکی‌پدیا، روند Validation، زمان آموزش و مصرف حافظه بررسی شود.

ممکن است روشی با پارامترهای بیشتر عملکرد بهتری ایجاد کند یا روشی با پارامترهای کمتر مصالحه مناسب‌تری میان یادگیری دامنه هدف و حفظ عملکرد عمومی ارائه دهد. بنابراین، نتیجه این بخش فقط نشان می‌دهد Bottleneck Adapter از نظر تعداد پارامترهای قابل‌آموزش اقتصادی‌تر بوده است و قضاوت نهایی درباره کارآمدی دو روش به تحلیل نتایج بخش‌های بعدی وابسته است.

۴-۲- مقایسه نتایج پرپلکسیتی سه مدل

برای مقایسه مستقیم عملکرد سه مدل، پرپلکسیتی آن‌ها روی دو مجموعه آزمون اخبار فارسی و ویکی‌پدیا در کنار یکدیگر قرار گرفت. آزمون اخبار، عملکرد درون‌دامنه‌ای مدل‌ها را پس از تنظیم دقیق روی داده‌های خبری نشان می‌دهد؛ درحالی‌که آزمون ویکی‌پدیا برای بررسی عملکرد برون‌دامنه‌ای و میزان حفظ عملکرد عمومی مدل استفاده می‌شود.

هر سه مدل با یک روش ارزیابیزیابی مشترک بررسی شدند. در این روش، Cross-Entropy در سطح تمام توکن‌های معتبر مجموعه محاسبه شد، توکن‌های Padding در میانگین زیان مشارکت نداشتند و پرپلکسیتی از اعمال تابع نمایی بر میانگین نهایی زیان به دست آمد. ثابت‌بودن مجموعه داده، توکن‌سازی و روش محاسبه باعث می‌شود تفاوت نتایج به اثر تنظیم دقیق با LoRA و Bottleneck Adapter نسبت داده شود، نه تفاوت در فرایند ارزیابی.

۴-۲-۱- جدول نهایی پرپلکسیتی

نتایج نهایی سه مدل در جدول ۴-۲ ارائه شده‌اند. در معیار پرپلکسیتی، مقدار کمتر نشان‌دهنده آن است که مدل به‌طور متوسط احتمال بیشتری به توکن صحیح اختصاص می‌دهد و در پیش‌بینی ادامه متن عدم‌اطمینان کمتری دارد.

*جدول ۴-۲: مقایسه پرپلکسیتی مدل پایه، **LoRA** و **Bottleneck Adapter** روی آزمون اخبار و ویکی‌پدیا*

مدل	پرپلکسیتی آزمون اخبار	پرپلکسیتی آزمون ویکی‌پدیا
مدل پایه	۷.۷۰۹۴۳۹	۷.۳۳۸۸۶۸
LoRA	6.059136	9.803662
Bottleneck Adapter	5.973723	8.799187

نتایج جدول دو الگوی مشخص را نشان می‌دهند. هر دو روش تنظیم دقیق، پرپلکسیتی اخبار را نسبت به مدل پایه کاهش داده‌اند؛ بنابراین، هر دو توانسته‌اند مدل را با دامنه هدف سازگار کنند. در مقابل، پرپلکسیتی ویکی‌پدیا در هر دو مدل تنظیم‌شده افزایش یافته است که نشان می‌دهد تخصصی‌شدن برای اخبار با کاهش قابل‌اندازه‌گیری عملکرد برون‌دامنه‌ای همراه بوده است.

۴-۲-۲- مقایسه عملکرد درون‌دامنه‌ای

مدل پایه پیش از هرگونه تنظیم دقیق روی داده‌های خبری، پرپلکسیتی بیشتری نسبت به دو مدل تنظیم‌شده ثبت کرد. پس از آموزش LoRA، پرپلکسیتی اخبار از ۷.۷۰۹۴۳۹ به ۶.۰۵۹۱۳۶ کاهش یافت. کاهش مطلق پرپلکسیتی برابر با ۱.۶۵۰۳۰۳ واحد بود. بهبود نسبی LoRA نسبت به مدل پایه نیز از رابطه زیر به دست می‌آید:

$$Improvement_{LoRA,News} = \frac{7.112165 - 6.031691}{7.112165} \times 100 \approx 15.19\%$$

Bottleneck Adapter نیز پرپلکسیتی اخبار را از ۷.۷۰۹۴۳۹ به ۵.۹۷۳۷۲۳ کاهش داد. کاهش مطلق این روش برابر با ۱.۷۳۵۷۱۶ واحد بود. بهبود نسبی Bottleneck Adapter نسبت به مدل پایه برابر است با:

$$Improvement_{Adapter,News} = \frac{7.112165 - 5.959396}{7.112165} \times 100 \approx 16.21\%$$

کاهش پرپلکسیتی هر دو روش روی مجموعه آزمون اهمیت دارد؛ زیرا متن‌های آزمون در فرایند بهینه‌سازی استفاده نشده بودند. بنابراین، این نتیجه نشان می‌دهد پارامترهای قابل‌آموزش صرفاً باعث کاهش زیان روی داده‌های Train نشده‌اند و تغییرات آموخته‌شده به متن‌های خبری دیده‌نشده نیز منتقل شده‌اند.

از میان سه مدل، Bottleneck Adapter کمترین پرپلکسیتی اخبار را به دست آورد و در نتیجه، بهترین عملکرد مستقیم درون‌دامنه‌ای را ثبت کرد. بااین‌حال، اختلاف آن با LoRA محدود بود:

$$6.059136 - 5.973723 = 0.085413$$

پرپلکسیتی Adapter حدود ۱.۴۱ درصد کمتر از پرپلکسیتی LoRA بود. بنابراین، هرچند Adapter از نظر عددی بهترین نتیجه خبری را ثبت کرده است، اختلاف عملکرد دو روش در دامنه هدف نسبتاً کوچک است و نمی‌توان تنها بر اساس این اختلاف، یکی از روش‌ها را به‌طور قطعی در تمام ابعاد برتر دانست.

این نتیجه از نظر پارامترکارایی نیز قابل‌توجه است؛ زیرا Bottleneck Adapter با تعداد پارامترهای قابل‌آموزش کمتر از LoRA، عملکرد درون‌دامنه‌ای اندکی بهتری ارائه داده است. این نتیجه نشان می‌دهد که در پیکربندی استفاده‌شده، فضای میانی ۱۶بعدی و مسیر Residual افزوده‌شده پس از MLPها ظرفیت کافی برای یادگیری اصلاحات مؤثر در دامنه خبری داشته‌اند.

۴-۲-۳- مقایسه عملکرد برون‌دامنه‌ای

در مجموعه آزمون ویکی‌پدیا، مدل پایه کمترین پرپلکسیتی را به دست آورد. ازآنجاکه این مدل تحت آموزش تخصصی روی اخبار قرار نگرفته بود، خروجی آن تحت تأثیر مؤلفه‌های دامنه‌محور LoRA یا Adapter قرار نداشت. نتایج نشان می‌دهند که هر دو روش تنظیم دقیق، در ازای بهبود اخبار، بخشی از عملکرد اولیه مدل روی ویکی‌پدیا را کاهش داده‌اند.

در مدل LoRA، پرپلکسیتی ویکی‌پدیا از ۷.۳۳۸۸۶۸ در مدل پایه به ۹.۸۰۳۶۶۲ افزایش یافت. افزایش مطلق آن برابر با ۲.۴۶۴۷۹۴ واحد بود. افزایش نسبی پرپلکسیتی ویکی‌پدیا در LoRA برابر است با:

$$Change_{LoRA,Wiki} = \frac{9.803662 - 7.338868}{7.338868} \times 100 \approx 33.59\%$$

در Bottleneck Adapter نیز پرپلکسیتی ویکی‌پدیا افزایش یافت، اما میزان این افزایش کمتر از LoRA بود. پرپلکسیتی این روش از ۷.۳۳۸۸۶۸ به ۸.۷۹۹۱۸۷ رسید و اختلاف مطلق آن با مدل پایه برابر با ۱.۴۶۰۳۱۹ واحد بود. افزایش نسبی پرپلکسیتی ویکی‌پدیا در Adapter برابر است با:

$$Change_{Adapter,Wiki} = \frac{8.799187 - 7.338868}{7.338868} \times 100 \approx 19.90\%$$

بنابراین، هر دو روش بخشی از عملکرد برون‌دامنه‌ای مدل را کاهش داده‌اند، اما Bottleneck Adapter توانسته است عملکرد عمومی مدل را بهتر از LoRA حفظ کند. پرپلکسیتی ویکی‌پدیای Adapter به‌اندازه ۱.۰۰۴۴۷۵ واحد کمتر از LoRA بود:

$$9.803662 - 8.799187 = 1.004475$$

این اختلاف نشان می‌دهد که در شرایط این آزمایش، تغییرات آموخته‌شده توسط LoRA اثر بیشتری بر پیش‌بینی متون خارج از دامنه آموزش گذاشته‌اند. در مقابل، Bottleneck Adapter افت محدودتری روی ویکی‌پدیا ایجاد کرده است. این تفاوت می‌تواند به محل اعمال پارامترها، تعداد پارامترهای قابل‌آموزش، نحوه مقداردهی اولیه و ساختار Residual Adapter مرتبط باشد؛ بااین‌حال، نتایج این آزمایش به‌تنهایی سهم دقیق هر عامل را مشخص نمی‌کنند.

ثابت‌بودن وزن‌های مدل پایه در هر دو روش به معنای ثابت‌ماندن خروجی مدل نیست. پارامترهای LoRA و Adapter پس از آموزش در تمام عبورهای پیش‌رو فعال هستند و نگاشت‌ها یا بازنمایی‌های میانی مدل را برای ورودی‌های خبری و غیرخبری تغییر می‌دهند. ازاین‌رو، افزایش پرپلکسیتی ویکی‌پدیا نشان می‌دهد اصلاحات آموخته‌شده برای اخبار با توزیع متون عمومی کاملاً سازگار نبوده‌اند.

در مجموع، نتایج پرپلکسیتی نشان می‌دهند که در این آزمایش، Bottleneck Adapter در هر دو سمت مقایسه وضعیت مطلوب‌تری داشته است. این روش پرپلکسیتی اخبار را به ۵.۹۷۳۷۲۳ رساند که حدود ۱.۴۱ درصد کمتر از LoRA است و در عین حال، پرپلکسیتی ویکی‌پدیای آن ۸.۷۹۹۱۸۷ بود که حدود ۱۰.۲۵ درصد کمتر از LoRA است. همچنین، بهبود نسبی اخبار در Adapter حدود ۲۲.۵۱ درصد و افت نسبی ویکی‌پدیا حدود ۱۹.۹۰ درصد بود؛ درحالی‌که این مقادیر برای LoRA به‌ترتیب ۲۱.۴۱ و ۳۳.۵۹ درصد بودند.

بنابراین، Bottleneck Adapter در این اجرا هم بهبود درون‌دامنه‌ای اندکی بیشتر و هم حفظ عملکرد برون‌دامنه‌ای بهتری ارائه کرده است. بااین‌حال، برای قضاوت دقیق درباره شدت کاهش عملکرد عمومی، لازم است Degradation Ratio به‌صورت رسمی محاسبه و مفهوم فراموشی فاجعه‌بار تحلیل شود که در بخش بعد انجام خواهد شد.

۴-۳- تحلیل فراموشی فاجعه‌بار و نرخ افت عملکرد

تنظیم دقیق مدل فقط با داده‌های اخبار فارسی انجام شد؛ بنابراین انتظار می‌رود مدل پس از آموزش، توزیع و الگوهای متون خبری را بهتر یاد بگیرد. بااین‌حال، تخصصی‌شدن مدل در یک دامنه ممکن است عملکرد آن را روی دامنه‌های عمومی‌تر کاهش دهد. برای بررسی این موضوع، عملکرد مدل‌های LoRA و Bottleneck Adapter روی مجموعه آزمون ویکی‌پدیا با عملکرد مدل پایه مقایسه شد.

در این بخش، ابتدا اثر تنظیم دقیق خبری بر عملکرد عمومی مدل بررسی می‌شود. سپس نرخ افت عملکرد یا Degradation Ratio برای هر روش محاسبه شده و شدت افت برون‌دامنه‌ای بر اساس دسته‌بندی عملیاتی تعریف‌شده در نوت‌بوک تحلیل می‌شود. در پایان نیز قابل‌توجیه‌بودن این افت در برابر بهبود عملکرد روی اخبار و یک راهکار نظری برای کاهش آن بررسی خواهد شد. محاسبه نرخ افت و تحلیل آن از الزامات مستقیم بخش چهارم تمرین است.

۴-۳-۱- اثر تنظیم دقیق خبری بر دانش عمومی مدل

مدل پایه پیش از تنظیم دقیق، کمترین پرپلکسیتی را روی مجموعه آزمون ویکی‌پدیا ثبت کرد. پس از آموزش مدل‌ها فقط با اخبار فارسی، پرپلکسیتی ویکی‌پدیا در هر دو روش افزایش یافت. این افزایش نشان می‌دهد مدل‌های تنظیم‌شده در پیش‌بینی توکن‌های متون عمومی نسبت به مدل پایه عدم‌اطمینان بیشتری پیدا کرده‌اند.

در روش LoRA، پرپلکسیتی ویکی‌پدیا از مقدار پایه ۷.۳۳۸۸۶۸ به ۹.۸۰۳۶۶۲ افزایش یافت. در Bottleneck Adapter نیز این مقدار به ۸.۷۹۹۱۸۷ رسید. بنابراین، هر دو روش با افت عملکرد برون‌دامنه‌ای همراه بودند، اما شدت این افت در LoRA بیشتر از Bottleneck Adapter بود.

ممکن است در نگاه نخست تصور شود که چون وزن‌های مدل پایه در هر دو روش فریز شده‌اند، عملکرد عمومی مدل نیز باید کاملاً ثابت باقی بماند. بااین‌حال، فریزشدن وزن‌های اصلی فقط مانع تغییر مستقیم آن‌ها می‌شود. ماژول‌های LoRA و Adapter پس از آموزش در زمان عبور پیش‌رو فعال هستند و نگاشت‌ها یا بازنمایی‌های میانی مدل را تغییر می‌دهند. در نتیجه، خروجی نهایی مدل می‌تواند حتی بدون تغییر وزن‌های پایه، روی داده‌های ویکی‌پدیا متفاوت شود.

در LoRA، ماتریس‌های کم‌رتبه مستقیماً به نگاشت‌های Query، Key، Value و Output در سازوکار Attention افزوده می‌شوند و می‌توانند نحوه تعامل اطلاعات میان توکن‌ها را تغییر دهند. در Bottleneck Adapter نیز یک مؤلفه قابل‌آموزش پس از خروجی MLP هر بلوک به مسیر اصلی افزوده می‌شود. بنابراین، هر دو روش می‌توانند رفتار مدل را علاوه بر دامنه آموزش، روی ورودی‌های خارج از آن نیز تغییر دهند.

ازاین‌رو، افت مشاهده‌شده را می‌توان نوعی تداخل دامنه‌ای یا فراموشی عملکردی دانست. دانش ذخیره‌شده در وزن‌های اصلی الزاماً حذف نشده است، اما مدل درحالی‌که ماژول تنظیم‌شده فعال است، عملکرد قبلی خود را روی متون عمومی با همان کیفیت نشان نمی‌دهد.

۴-۳-۲- محاسبه Degradation Ratio

برای کمی‌سازی افت عملکرد روی ویکی‌پدیا، از رابطه تعیین‌شده در صورت تمرین استفاده شد:

$$Degradation\ Ratio = \frac{PPL_{finetuned,Wiki} - PPL_{base,Wiki}}{PPL_{base,Wiki}}$$

مقدار مثبت این نسبت نشان می‌دهد پرپلکسیتی پس از تنظیم دقیق افزایش یافته و عملکرد برون‌دامنه‌ای کاهش پیدا کرده است. مقدار صفر به معنای نبود تغییر و مقدار منفی نشان‌دهنده بهبود عملکرد روی ویکی‌پدیا است. همین رابطه در نوت‌بوک برای محاسبه نرخ افت هر دو روش استفاده شده است.

برای LoRA داریم:

$$Ratio_{LoRA} = \frac{9.803662 - 7.338868}{7.338868} = 0.335855 \times 100 = 33.59\%$$

برای Bottleneck Adapter نیز داریم:

$$Ratio_{Adapter} = \frac{8.799187 - 7.338868}{7.338868} \times 100 = 19.90\%$$

نتایج کامل در جدول ۴-۳ ارائه شده‌اند.

جدول ۴-۳: مقایسه نرخ افت عملکرد ***LoRA*** *و* ***Bottleneck Adapter*** *روی آزمون ویکی‌پدیا*

روش	پرپلکسیتی پایه	پرپلکسیتی پس از تنظیم دقیق	تغییر مطلق	Degradation Ratio	درصد افت
LoRA	7.338868	9.803662	2.464794	0.335855	33.59%
Bottleneck Adapter	7.338868	8.799187	1.460319	0.198984	19.90%

نتایج نشان می‌دهند نرخ افت LoRA حدود ۱۳.۶۹ واحد درصد بیشتر از Bottleneck Adapter است. همچنین، افزایش مطلق پرپلکسیتی در LoRA به‌اندازه ۱.۰۰۴۴۷۵ واحد بیشتر از Adapter بوده است. بنابراین، Bottleneck Adapter توانسته است بخش بیشتری از عملکرد عمومی مدل را حفظ کند؛ هرچند عملکرد آن نیز نسبت به مدل پایه به‌طور قابل‌اندازه‌گیری کاهش یافته است.

۴-۳-۳- بررسی وقوع و شدت فراموشی فاجعه‌بار

اصطلاح «فراموشی فاجعه‌بار» آستانه عددی جهانی و ثابتی در معیار پرپلکسیتی ندارد. به همین دلیل، در نوت‌بوک برای تفسیر شفاف نتایج همین آزمایش، یک دسته‌بندی عملیاتی تعریف شده است:

- نسبت کمتر یا مساوی صفر: نبود افت برون‌دامنه‌ای؛
- افت کمتر از ۵ درصد: افت کم؛
- افت ۵ تا کمتر از ۲۰ درصد: افت متوسط؛
- افت ۲۰ درصد یا بیشتر: افت قابل‌توجه و سازگار با فراموشی شدید برون‌دامنه‌ای در این آزمایش.

بر اساس این معیار، نرخ افت Bottleneck Adapter برابر ۱۹.۹۰ درصد است و در محدوده افت متوسط قرار می‌گیرد. بااین‌حال، این مقدار فقط حدود ۰.۱۰ واحد درصد پایین‌تر از آستانه ۲۰ درصد است؛ بنابراین، Adapter با وجود قرارگرفتن در دسته افت متوسط، بسیار نزدیک به محدوده افت قابل‌توجه قرار دارد.

در مقابل، نرخ افت LoRA برابر ۳۳.۵۹ درصد است و به‌وضوح از آستانه عملیاتی ۲۰ درصد عبور می‌کند. بنابراین، طبق معیار تعریف‌شده در نوت‌بوک، این روش در محدوده افت قابل‌توجه و سازگار با فراموشی شدید برون‌دامنه‌ای قرار می‌گیرد.

در نتیجه، نمی‌توان گفت عملکرد عمومی مدل پس از تنظیم دقیق بدون تغییر باقی مانده است؛ زیرا در هر دو روش افزایش آشکار پرپلکسیتی ویکی‌پدیا مشاهده شد. بااین‌حال، بهتر است اصطلاح «فراموشی فاجعه‌بار» با احتیاط استفاده شود. وزن‌های مدل پایه حذف یا بازنویسی نشده‌اند و آستانه استفاده‌شده نیز یک معیار عملیاتی برای همین آزمایش است، نه یک مرز علمی و جهانی.

بیان دقیق‌تر آن است که Bottleneck Adapter موجب تداخل برون‌دامنه‌ای متوسط اما نزدیک به محدوده قابل‌توجه شده است، درحالی‌که LoRA موجب افت شدیدتر و قابل‌توجه عملکرد برون‌دامنه‌ای شده است. بنابراین، شواهد این آزمایش درباره فراموشی عملکردی در LoRA قوی‌تر از Adapter هستند.

۴-۳-۴- قابل‌توجیه‌بودن افت و راهکار نظری کاهش آن

قابل‌توجیه‌بودن افت عملکرد باید در کنار میزان بهبود اخبار بررسی شود. LoRA پرپلکسیتی اخبار را از ۷.۷۰۹۴۳۹ به ۶.۰۵۹۱۳۶ کاهش داد که معادل بهبود نسبی حدود ۲۱.۴۱ درصد بود. بااین‌حال، این روش هم‌زمان افزایش ۳۳.۵۹ درصدی پرپلکسیتی ویکی‌پدیا را ایجاد کرد.

بنابراین، در یک مقایسه توصیفی، هزینه برون‌دامنه‌ای LoRA از میزان بهبود نسبی آن در دامنه اخبار بیشتر بوده است. در کاربردی که علاوه بر سازگاری با اخبار، حفظ عملکرد عمومی نیز اهمیت دارد، این افت نسبتاً زیاد است و به‌سختی می‌توان آن را فقط بر اساس بهبود اخبار کاملاً توجیه کرد.

Bottleneck Adapter پرپلکسیتی اخبار را از ۷.۷۰۹۴۳۹ به ۵.۹۷۳۷۲۳ کاهش داد که معادل بهبود نسبی حدود ۲۲.۵۱ درصد بود. درعین‌حال، افت نسبی ویکی‌پدیا در این روش ۱۹.۹۰ درصد باقی ماند. بنابراین، Adapter نه‌تنها بهبود اندکی بیشتری روی اخبار ایجاد کرد، بلکه هزینه برون‌دامنه‌ای کمتری نیز نسبت به LoRA داشت.

بر این اساس، افت Bottleneck Adapter در برابر سازگاری ایجادشده با اخبار قابل‌دفاع‌تر است. بااین‌حال، افت نزدیک به ۲۰ درصد همچنان برای مدلی که قرار است توانایی عمومی خود را حفظ کند، ناچیز نیست و بهتر است در طراحی فرایند تنظیم دقیق کاهش یابد. همچنین، مقایسه درصدهای دو مجموعه آزمون فقط یک تحلیل توصیفی از مصالحه است و به‌تنهایی جایگزین تعیین اهمیت واقعی هر دامنه در کاربرد نهایی نمی‌شود.

یک راهکار نظری مناسب برای کاهش افت، استفاده از بازپخش داده‌های عمومی یا Mixed-domain Rehearsal است. در این روش، هنگام تنظیم دقیق روی اخبار، درصد کنترل‌شده‌ای از نمونه‌های عمومی مانند متون ویکی‌پدیا نیز در Batchهای آموزشی قرار می‌گیرند. تابع هدف را می‌توان به‌صورت ترکیبی تعریف کرد:

$$\mathcal{L}_{total} = \mathcal{L}_{News} + \lambda \mathcal{L}_{General}$$

در این رابطه، $\mathcal{L}_{News}$ مدل را به سمت یادگیری دامنه خبری هدایت می‌کند و $\mathcal{L}_{General}$ مانع دورشدن بیش‌ازحد رفتار مدل از متون عمومی می‌شود. ضریب λ نیز میزان اهمیت حفظ توانایی عمومی را کنترل می‌کند.

این روش باعث می‌شود گرادیان‌های آموزش فقط از توزیع اخبار به دست نیایند و مدل در طول تخصصی‌شدن، به‌طور هم‌زمان بخشی از الگوهای عمومی را نیز مرور کند. نسبت نمونه‌های خبری و عمومی و مقدار λ باید بر اساس عملکرد مدل روی مجموعه اعتبارسنجی اخبار و یک مجموعه اعتبارسنجی عمومی تنظیم شوند.

انتظار می‌رود با این روش، بهبود اخبار تا حدی محدودتر شود، اما افزایش پرپلکسیتی ویکی‌پدیا نیز کاهش یابد و مصالحه متعادل‌تری میان تخصصی‌سازی خبری و حفظ عملکرد عمومی ایجاد شود. در مجموع، نتایج این بخش نشان می‌دهند Bottleneck Adapter در آزمایش حاضر نسبت به LoRA تعادل مناسب‌تری میان یادگیری اخبار و حفظ عملکرد برون‌دامنه‌ای ارائه کرده است.

۴-۴- مقایسه مصالحه میان یادگیری اخبار و حفظ دانش قبلی

ارزیابی یک روش تنظیم دقیق نباید فقط بر اساس بهترین عملکرد آن در دامنه هدف انجام شود. ممکن است یک روش پرپلکسیتی اخبار را کاهش دهد، اما هم‌زمان موجب افت قابل‌توجه عملکرد مدل روی متون عمومی شود. ازاین‌رو، برای بررسی کارآمدی واقعی LoRA و Bottleneck Adapter، باید میزان بهبود آن‌ها روی اخبار در کنار میزان افت عملکرد روی ویکی‌پدیا تحلیل شود.

در این مقایسه، کاهش پرپلکسیتی اخبار به‌عنوان سود حاصل از تخصصی‌سازی و افزایش پرپلکسیتی ویکی‌پدیا به‌عنوان هزینه برون‌دامنه‌ای در نظر گرفته می‌شود. روش مطلوب روشی است که با کمترین اختلال در عملکرد عمومی مدل، بیشترین بهبود را در دامنه هدف ایجاد کند. تعداد پارامترهای قابل‌آموزش نیز به‌عنوان عامل مکمل در تحلیل پارامترکارایی دو روش در نظر گرفته می‌شود.

۴-۴-۱- مقایسه بهبود اخبار در برابر افت ویکی‌پدیا

برای مقایسه مستقیم دو روش، تغییر پرپلکسیتی آن‌ها نسبت به مدل پایه در جدول ۴-۴ ارائه شده است.

جدول ۴-۴: مقایسه بهبود درون‌دامنه‌ای و افت برون‌دامنه‌ای روش‌های ***LoRA*** *و* ***Bottleneck Adapter***

روش	کاهش پرپلکسیتی اخبار	بهبود نسبی اخبار	افزایش پرپلکسیتی ویکی‌پدیا	افت نسبی ویکی‌پدیا	پارامترهای قابل‌آموزش
LoRA	1.650303	21.41%	2.464794	33.59%	5,570,560
Bottleneck Adapter	1.735716	22.51%	1.460319	19.90%	2,028,000

هر دو روش توانستند عملکرد مدل را روی اخبار فارسی بهبود دهند، اما هزینه‌ای که برای این تخصصی‌سازی در دامنه عمومی ایجاد کردند یکسان نبود. در LoRA، پرپلکسیتی اخبار از مقدار پایه ۷.۷۰۹۴۳۹ به ۶.۰۵۹۱۳۶ کاهش یافت. بنابراین، کاهش مطلق پرپلکسیتی برابر با ۱.۶۵۰۳۰۳ و بهبود نسبی برابر با ۲۱.۴۱ درصد بود. بااین‌حال، پرپلکسیتی ویکی‌پدیا در این روش از ۷.۳۳۸۸۶۸ به ۹.۸۰۳۶۶۲ افزایش یافت. این افزایش برابر با ۲.۴۶۴۷۹۴ واحد و معادل افت نسبی ۳۳.۵۹ درصد بود. بنابراین، در LoRA میزان افت نسبی عملکرد ویکی‌پدیا از میزان بهبود نسبی اخبار بیشتر بود. به بیان دیگر، بهبود درون‌دامنه‌ای این روش با هزینه برون‌دامنه‌ای قابل‌توجهی همراه شد.

در Bottleneck Adapter، پرپلکسیتی اخبار از ۷.۷۰۹۴۳۹ به ۵.۹۷۳۷۲۳ کاهش یافت. کاهش مطلق آن برابر با ۱.۷۳۵۷۱۶ و بهبود نسبی آن حدود ۲۲.۵۱ درصد بود. در مقابل، پرپلکسیتی ویکی‌پدیا در این روش از ۷.۳۳۸۸۶۸ به ۸.۷۹۹۱۸۷ افزایش یافت.

این افزایش برابر با ۱.۴۶۰۳۱۹ واحد و معادل افت نسبی ۱۹.۹۰ درصد بود. بنابراین، Bottleneck Adapter در کنار بهبود اندکی بیشتر روی اخبار، افت کمتری نیز روی ویکی‌پدیا ایجاد کرد. بهبود نسبی اخبار در این روش حدود ۱.۱۱ واحد درصد بیشتر از LoRA بود. همچنین، نرخ افت ویکی‌پدیا در Adapter حدود ۱۳.۶۹ واحد درصد کمتر از LoRA بود. از نظر تغییر مطلق نیز Adapter پرپلکسیتی اخبار را به‌اندازه ۰.۰۸۵۴۱۳ واحد بیشتر از LoRA کاهش داد. درعین‌حال، افزایش پرپلکسیتی ویکی‌پدیا در Adapter به‌اندازه ۱.۰۰۴۴۷۵ واحد کمتر از LoRA بود.

از نظر تعداد پارامترها نیز Bottleneck Adapter با تعداد بسیار کمتری پارامتر قابل‌آموزش به این نتایج دست یافت. تعداد پارامترهای قابل‌آموزش Adapter برابر با ۲,۰۲۸,۰۰۰ بود، درحالی‌که LoRA دارای ۵,۵۷۰,۵۶۰ پارامتر قابل‌آموزش بود. بنابراین، Adapter حدود ۶۳.۵۹ درصد پارامتر قابل‌آموزش کمتری نسبت به LoRA داشت.

در نتیجه، عملکرد بهتر Adapter صرفاً حاصل استفاده از ظرفیت پارامتری بیشتر نبوده است. این روش با تعداد پارامترهای قابل‌آموزش کمتر، هم سازگاری درون‌دامنه‌ای اندکی بیشتری ایجاد کرده و هم عملکرد برون‌دامنه‌ای را بهتر حفظ کرده است.

این نتایج نشان می‌دهند که در آزمایش حاضر میان دو روش مبادله‌ای وجود نداشت که یکی از آن‌ها روی اخبار بهتر و دیگری روی ویکی‌پدیا بهتر باشد. Bottleneck Adapter در هر دو معیار پرپلکسیتی و همچنین در تعداد پارامترهای قابل‌آموزش وضعیت مطلوب‌تری داشت. به بیان دیگر، بر اساس معیارهای بررسی‌شده در جدول ۴-۴، Adapter نسبت به LoRA برتری هم‌زمان نشان داد.

بااین‌حال، هر دو مدل تنظیم‌شده نسبت به مدل پایه روی ویکی‌پدیا افت کردند. بنابراین، حتی Bottleneck Adapter نیز نتوانست تخصصی‌سازی خبری را بدون هزینه برون‌دامنه‌ای انجام دهد و افت ۱۹.۹۰ درصدی آن همچنان قابل‌توجه و نزدیک به آستانه عملیاتی ۲۰ درصد است.

۴-۴-۲- تعیین روش دارای مصالحه بهتر

بر اساس نتایج کمی، Bottleneck Adapter مصالحه مناسب‌تری میان یادگیری اخبار فارسی و حفظ عملکرد عمومی مدل ارائه کرده است. این روش نه‌تنها کمترین پرپلکسیتی اخبار را با مقدار ۵.۹۷۳۷۲۳ ثبت کرد، بلکه پرپلکسیتی ویکی‌پدیای آن نیز با مقدار ۸.۷۹۹۱۸۷ کمتر از مقدار ۹.۸۰۳۶۶۲ به‌دست‌آمده برای LoRA بود. همچنین، این نتایج با تعداد پارامترهای قابل‌آموزش کمتری به دست آمدند.

تفاوت عملکرد دو روش می‌تواند تا حدی به محل اعمال تغییرات در معماری مربوط باشد. در LoRA، اصلاحات کم‌رتبه با Rank برابر ۱۶ به نگاشت‌های Query، Key، Value و Output در Attention افزوده شدند. در این پیاده‌سازی، مجموعاً ۱۰۰ نگاشت خطی هدف قرار گرفتند. سازوکار Attention تعیین می‌کند هر توکن چگونه اطلاعات سایر توکن‌ها را ترکیب کند؛ بنابراین، تغییر در این نگاشت‌ها می‌تواند بر روابط توکنی و بازنمایی‌های ایجادشده در دامنه‌های مختلف اثر گسترده‌ای داشته باشد.

در مقابل، در روش Bottleneck Adapter پس از خروجی MLP هر یک از ۳۰ بلوک متنی، یک مسیر Down–GELU–Up با بُعد میانی ۱۶ افزوده شد. خروجی این مسیر به‌صورت Residual به مسیر اصلی اضافه می‌شود:

$$y = h + Adapter(h)$$

در این ساختار، مسیر اصلی MLP بدون تغییر حفظ می‌شود و Adapter یک مؤلفه اصلاحی به آن می‌افزاید. همچنین، وزن‌های لایه Up-projection در آغاز برابر صفر مقداردهی شدند. در نتیجه، خروجی Adapter در شروع آموزش صفر بود و مدل پیش از یادگیری پارامترهای جدید، رفتاری معادل مدل پایه داشت. سپس سهم مسیر اصلاحی به‌تدریج در جریان آموزش شکل گرفت.

این تفاوت معماری می‌تواند یکی از دلایل احتمالی افت کمتر Bottleneck Adapter روی ویکی‌پدیا باشد. مسیر Residual ممکن است امکان اعمال اصلاحات محدودتر و کنترل‌شده‌تری را فراهم کرده باشد، درحالی‌که تغییر مستقیم نگاشت‌های Attention در LoRA می‌تواند تأثیر گسترده‌تری بر پردازش توالی‌ها و عملکرد خارج از دامنه داشته باشد.

بااین‌حال، این توضیح یک تفسیر مهندسی مبتنی بر نتایج همین آزمایش است و از مقایسه حاضر نمی‌توان رابطه علّی قطعی میان محل تزریق ماژول‌ها و میزان افت ویکی‌پدیا استخراج کرد. برای تبدیل این تفسیر به نتیجه‌ای عمومی، لازم است آزمایش‌ها با Seedهای مختلف، مجموعه‌داده‌های متنوع، Rankها و ابعاد میانی متفاوت و تنظیمات آموزشی گوناگون تکرار شوند.

پارامترهای اصلی آموزش دو روش نیز یکسان بودند. هر دو روش با مجموعه آموزشی خبری یکسان، نرخ یادگیری 2×10^{-4}، سه دوره آموزش، اندازه مؤثر Batch برابر ۸ و Rank یا بُعد میانی برابر ۱۶ اجرا شدند. بنابراین، تفاوت مشاهده‌شده را می‌توان عمدتاً به نوع پارامترسازی، تعداد پارامترهای قابل‌آموزش و محل تزریق ماژول‌ها مرتبط دانست، هرچند سایر تفاوت‌های پیاده‌سازی نیز ممکن است نقش داشته باشند.

بر این اساس، Bottleneck Adapter در آزمایش حاضر روش موفق‌تری برای ایجاد تعادل میان تخصصی‌سازی خبری و حفظ عملکرد عمومی مدل بوده است. این روش با ۶۳.۵۹ درصد پارامتر قابل‌آموزش کمتر، بهبود نسبی ۲۲.۵۱ درصدی روی اخبار و افت نسبی ۱۹.۹۰ درصدی روی ویکی‌پدیا ایجاد کرد. در مقابل، LoRA بهبود ۲۱.۴۱ درصدی اخبار را با افت ۳۳.۵۹ درصدی ویکی‌پدیا به دست آورد.

بنابراین، از دید معیارهای پرپلکسیتی و پارامترکارایی، Bottleneck Adapter نسبت به LoRA انتخاب مناسب‌تری محسوب می‌شود. بااین‌حال، نتیجه‌گیری نهایی درباره برتری عملی یک روش باید علاوه بر این معیارها، کیفیت تولید متن، خطر تولید جزئیات پشتیبانی‌نشده، مصرف حافظه، زمان آموزش و روند همگرایی را نیز در نظر بگیرد. این جنبه‌ها در بخش‌های بعدی بررسی می‌شوند تا مشخص شود آیا برتری کمی Bottleneck Adapter در سایر ابعاد ارزیابی نیز حفظ می‌شود یا خیر.

۴-۵- ارزیابی کیفی تولید متن

پرپلکسیتی نشان می‌دهد هر مدل تا چه اندازه در پیش‌بینی توکن بعدی موفق بوده است، اما به‌تنهایی نمی‌تواند ویژگی‌هایی مانند روانی متن، شباهت به سبک خبری، انسجام معنایی، تکرار، خطاهای زبانی یا تولید جزئیات تأییدنشده را مشخص کند. به همین دلیل، در کنار ارزیابی کمی، خروجی مدل پایه، LoRA و Bottleneck Adapter برای سه آغازگر خبری یکسان به‌صورت کیفی بررسی شد.

مطابق خواسته تمرسته تمرین، خروجی هر سه مدل برای تمام پرامپت‌ها در گزارش ارائه شده و تفاوت آن‌ها از نظر لحن خبری، ارتباط با پرامپت، انسجام معنایی، میزان تکرار و خطر Hallucination تحلیل می‌شود.

در این تحلیل، هر ادعا یا جزئیاتی که در خود پرامپت وجود ندارد و مدل بدون دسترسی به منبع بیرونی تولید می‌کند، «جزئیات پشتیبانی‌نشده» در نظر گرفته می‌شود. این عنوان لزوماً به معنای اثبات نادرستی ادعا نیست، اما نشان می‌دهد صحت آن از اطلاعات موجود در ورودی قابل تأیید نیست. همچنین، علاوه بر Hallucination، پدیده‌هایی مانند تکرار شدید، بی‌انسجامی معنایی و فروپاشی تولید نیز بررسی شده‌اند؛ زیرا ممکن است مدلی بدون تولید اطلاعات دقیق ساختگی، در تولید یک ادامه معنادار شکست بخورد.

۴-۵-۱- پرامپت‌ها و شرایط تولید

سه پرامپت انتخاب‌شده، سه حوزه رایج در اخبار فارسی را پوشش می‌دهند:

۱. خبرهای پارلمانی و سیاسی؛
۲. گزارش‌های هواشناسی؛
۳. اطلاعیه‌های حوزه سلامت.

متن پرامپت‌ها به‌صورت زیر بود:

- «دیروز در مجلس شورای اسلامی، نمایندگان درباره»
- «بر اساس اعلام سازمان هواشناسی، در روزهای آینده»
- «وزارت بهداشت امروز در بیانیه‌ای اعلام کرد»

برای رعایت عدالت در مقایسه، هر سه مدل با پرامپت‌ها و تنظیمات تولید یکسان اجرا شدند. حداکثر تعداد توکن جدید برابر ۱۲۰ بود و تولید به‌صورت قطعی با do_sample=False انجام شد. بنابراین، نمونه‌برداری تصادفی، Temperature و Top-p در فرایند تولید دخالت نداشتند. مقدار repetition_penalty نیز برابر ۱.۰۵ قرار گرفت.

ازآنجاکه تولید به‌صورت قطعی انجام شد، تفاوت خروجی‌ها ناشی از تصادفی‌بودن نمونه‌برداری نیست و به تفاوت رفتار مدل پایه و ماژول‌های آموزش‌دیده مربوط می‌شود. با وجود استفاده از جریمه تکرار، در تعدادی از خروجی‌ها تکرار شدید عبارت‌ها مشاهده شد. همچنین، بیشتر خروجی‌ها با رسیدن به سقف ۱۲۰ توکن جدید در میانه جمله یا کلمه پایان یافتند.

۴-۵-۲- مقایسه خروجی‌ها برای پرامپت پارلمانی

پرامپت اول:

دیروز در مجلس شورای اسلامی، نمایندگان درباره

خروجی مدل پایه:

دیروز در مجلس شورای اسلامی، نمایندگان درباره طرحی به نام «اصلاح قانون انتخابات» بحث و تبادل نظر کردند.

در این جلسه، نمایندگان با ۱۸۴ رای موافق، ۷۲ رای مخالف و ۱۰ رای ممتنع از مجموع ۲۶۳ نماینده حاضر در صحن علنی، طرح اصلاح قانون انتخابات را تصویب کردند.

این طرح که توسط کمیسیون اصل نود (۹۰) قانون اساسی تهیه شده بود، شامل ۱۵ ماده است که مهمترین آنها عبارتند از:

- "حذف شرط سنی برای کاندیداتوری در انتخابات ریاست"

خروجی مدل پایه از نظر جمله‌بندی روان و از نظر قالب به خبرهای رسمی پارلمانی شبیه است. مدل موضوعی مرتبط با ادامه پرامپت انتخاب کرده و متن را با گزارش جلسه، رأی‌گیری و معرفی مفاد طرح ادامه داده است. بااین‌حال، نام طرح، تعداد دقیق آرا، نام کمیسیون و تعداد مواد همگی بدون پشتیبانی پرامپت تولید شده‌اند. بنابراین، روانی و ظاهر معتبر متن نباید با صحت محتوایی آن یکسان تلقی شود.

خروجی مدل LoRA:

دیروز در مجلس شورای اسلامی، نمایندگان درباره لایحه بودجه سال ۹۹ اظهار نظر کردند.

در این جلسه، نمایندگان به بررسی بخش‌های مختلف لایحه بودجه پرداختند و برخی از آنها خواستار اصلاح برخی از بندها شدند. [n] در ادامه این جلسه، نمایندگان با طرح سوالی از وزیر اقتصاد، به عملکرد وزارت امور اقتصادی و دارایی در حوزه مالیاتی و همچنین نحوه اجرای قانون مالیات بر ارزش افزوده پرداختند. [n] در پایان این جلسه، نمایندگان با رای گیری، موافقت خود را با لایحه بودجه سال ۹۹ اعلام کردند.

LoRA یک ساختار خبری روشن ایجاد کرده و موضوع را در فضای فعالیت‌های مجلس حفظ کرده است. توالی معرفی موضوع، بیان دیدگاه نمایندگان، طرح سؤال از وزیر و اعلام نتیجه رأی‌گیری به الگوی گزارش‌های خبری شباهت دارد. بااین‌حال، مدل موضوع مشخص «بودجه سال ۹۹» را انتخاب کرده و رویدادهایی درباره وزیر اقتصاد و مالیات بر ارزش افزوده ساخته است که در پرامپت وجود ندارند.

با توجه به وجود نشانه‌های [n]، احتمالاً مدل الگوی جداکننده یا نشانه خط جدید موجود در داده‌های آموزشی را به‌صورت متن خام بازتولید کرده است. بنابراین، LoRA لحن خبری مناسبی ایجاد کرده، اما از نظر پاکیزگی خروجی و جزئیات پشتیبانی‌نشده ضعف دارد.

خروجی Bottleneck Adapter:

دیروز در مجلس شورای اسلامی، نمایندگان درباره طرحی که به نام «طرح اصلاح قانون انتخابات» شناخته می‌شود، رای گیری کردند. [n] این طرح با ۱۸۶ رای موافق، ۲۱ رای مخالف و ۳۰ رای ممتنع از مجموع ۲۳۷ نماینده حاضر در صحن علنی مجلس تصویب شد. [n] بر اساس این طرح، هر یک از اعضای هیات رئیسه مجلس شورای اسلامی می‌تواند تا سه نفر را برای حضور در جلسات کمیسیون‌های تخصصی انتخاب کند. [n] همچنین در این طرح آمده است: هر یک از اعضای هیات رئیسه مجلس

خروجی Adapter نیز لحن رسمی و ساختار قابل‌قبولی دارد و مانند مدل پایه، موضوع اصلاح قانون انتخابات را انتخاب کرده است. بااین‌حال، تعداد دقیق آرا و قاعده مطرح‌شده درباره اعضای هیئت‌رئیسه بدون هیچ پشتوانه‌ای از پرامپت تولید شده‌اند. متن همچنین حاوی [n] است و جمله پایانی آن کامل نشده است.

از میان سه خروجی، Adapter بیشترین جزئیات عددی پشتیبانی‌نشده را در این مثال تولید کرده است. در مقابل، LoRA موضوع خبر را به بودجه تغییر داده و مدل پایه نیز اعداد و اطلاعات قانونی دقیقی ساخته است. بنابراین، هر سه مدل از نظر لحن موفق بوده‌اند، اما هیچ‌کدام از تولید جزئیات ساختگی مصون نبوده‌اند.

۴-۵-۳- مقایسه خروجی‌ها برای پرامپت هواشناسی

پرامپت دوم:

بر اساس اعلام سازمان هواشناسی، در روزهای آینده

خروجی مدل پایه:

بر اساس اعلام سازمان هواشناسی، در روزهای آینده آسمان استان صاف تا قسمتی ابری و گاهی وزش باد پیش بینی می شود.

- به گزارش سایت قطره و به نقل ازخبرگزاری فارس از اصفهان، بر اساس اعلام سازمان هواشناسی، در روزهای آینده آسمان استان صاف تا قسمتی ابری و گاهی وزش باد پیش بینی می شود.

در این مدت بیشینه دمای هوای شهرها بین ۲۰ تا ۳۰ درجه سانتیگراد خواهد بود.

پایان پیام/۶۳۱۰۴/

مدل پایه قالب آشنای خبر هواشناسی را تولید کرده است، اما جمله نخست تقریباً دوباره تکرار می‌شود. عبارت‌هایی مانند نام وب‌سایت، خبرگزاری، محل انتشار و «پایان پیام» ظاهری شبیه متن استخراج‌شده از یک خبر واقعی ایجاد می‌کنند، ولی ارتباط آن‌ها با پرامپت مشخص نیست.

عبارت «آسمان استان» نیز بدون تعیین استان آمده است، درحالی‌که بعداً اصفهان ذکر می‌شود. بازه دقیق دما و کد پیام نیز جزئیات پشتیبانی‌نشده‌اند. بنابراین، مهم‌ترین ضعف مدل پایه در این مثال، تکرار، متن‌های زائد شبیه Boilerplate خبرگزاری و انسجام ناقص مکانی است.

خروجی مدل LoRA:

بر اساس اعلام سازمان هواشناسی، در روزهای آینده جوی پایدار در کشور حاکم است و آسمان اکثر مناطق صاف تا کمی ابری خواهد بود. [n] همچنین از بعدازظهر امروز (شنبه) و فردا (یکشنبه) در برخی مناطق استان‌های آذربایجان شرقی، اردبیل، گیلان، مازندران، گلستان، خراسان شمالی، شمال خلیج فارس و سواحل دریای خزر وزش باد نسبتا شدید پیش‌بینی می‌شود. [n] در این مدت در ارتفاعات البرز مرکزی و دره‌های زاگرس مرکزی بارش پراکنده رخ می‌دهد

LoRA نسبت به مدل پایه، متنی منسجم‌تر و حرفه‌ای‌تر تولید کرده است. ساختار کلی شامل توصیف وضعیت عمومی کشور و سپس ارائه جزئیات منطقه‌ای است و به شیوه گزارش‌های هواشناسی شباهت زیادی دارد.

بااین‌حال، فهرست استان‌ها، روزهای هفته، وزش باد و بارش در ارتفاعات همگی بدون اطلاعات زمینه‌ای موجود در پرامپت ساخته شده‌اند. علاوه بر این، نشانه‌های [n] همچنان در متن دیده می‌شوند. بنابراین، LoRA از نظر سبک و انسجام بهتر عمل کرده، اما افزایش جزئیات باعث افزایش خطر Hallucination نیز شده است.

خروجی Bottleneck Adapter:

بر اساس اعلام سازمان هواشناسی، در روزهای آینده بارش‌های پراکنده در برخی مناطق کشور

به گزارش ایمنا و به نقل از روابط عمومی سازمان هواشناسی، بر اساس تحلیل آخرین نقشه‌های پیش‌یابی، امروز (یکشنبه) در استان‌های آذربایجان شرقی، آذربایجان غربی، اردبیل، زنجان، قزوین، کردستان، کرمانشاه، ایلام، لرستان، چهارمحال و بختیاری، کهگیلویه و بویراحمد، شمال خوزستان، غرب اصفهان، جنوب تهران، البرز، مرکزی، همدان، خراسان رض

Adapter نیز الگوی حرفه‌ای خبر هواشناسی را بازتولید کرده و از عباراتی مانند «تحلیل آخرین نقشه‌های پیش‌یابی» و «روابط عمومی سازمان هواشناسی» استفاده کرده است. بااین‌حال، نام منبع خبری و فهرست بسیار طولانی استان‌ها بدون پشتیبانی پرامپت ایجاد شده‌اند.

در این نمونه، Adapter بیشترین مقدار جزئیات جغرافیایی پشتیبانی‌نشده را تولید کرده است. جمله پایانی نیز در میانه نام یک استان قطع شده که خوانایی خروجی را کاهش می‌دهد. در مقابل، LoRA انسجام بهتری دارد و مدل پایه بیشترین تکرار و متن زائد خبرگزاری را نشان می‌دهد.

۴-۵-۴- مقایسه خروجی‌ها برای پرامپت وزارت بهداشت

پرامپت سوم:

وزارت بهداشت امروز در بیانیه‌ای اعلام کرد

خروجی مدل پایه:

وزارت بهداشت امروز در بیانیه‌ای اعلام کرد: «با توجه به اینکه برخی از افراد با هدف ایجاد ناامنی و یا سوءاستفاده از شرایط موجود، اقدام به انتشار اخبار کذب و شایعاتی مبنی بر افزایش تعداد مبتلایان به کرونا ویروس در کشور کرده‌اند، لازم است مردم عزیز بدانند که این شایعات صحت ندارد.»

در ادامه این بیانیه آمده است: «بر اساس آخرین آمار رسمی وزارت بهداشت، درمان و آموزش پزشکی، تاکنون ۱۰ هزار و ۲۳۶ نفر در کشور به بیماری کووید-۱۹ (کرونا) مبتلا شده‌اند

مدل پایه قالب یک بیانیه رسمی را به‌صورت روان و باورپذیر تولید کرده است. استفاده از نقل‌قول مستقیم و عبارت «بر اساس آخرین آمار رسمی» باعث می‌شود متن ظاهری معتبر داشته باشد. بااین‌حال، موضوع شایعات مربوط به کرونا، متن نقل‌قول و آمار دقیق مبتلایان کاملاً توسط مدل ساخته شده‌اند.

در اینجا، ظاهر رسمی متن خطر Hallucination را افزایش می‌دهد؛ زیرا خواننده ممکن است عدد و نقل‌قول تولیدشده را به اشتباه یک بیانیه واقعی تلقی کند. جمله نهایی نیز ناتمام باقی مانده است.

خروجی مدل LoRA:

> وزارت بهداشت امروز در بیانیه‌ای اعلام کرد: با توجه به شیوع ویروس کرونا و برای جلوگیری از انتقال آن، کلیه مراکز درمانی دولتی و خصوصی کشور از امروز (شنبه) تعطیل شد. [n] این وزارتخانه همچنین اعلام کرد که تمامی بیمارستان‌های دولتی و خصوصی کشور از امروز (شنبه) تعطیل شده‌اند و تنها خدمات اورژانس و بخش‌های ویژه مانند ICU و CCU را ارائه خواهند کرد. [n] بر اساس این گزارش، تمام مراکز درمانی دولتی و خصوصی کشور از امروز (شنبه) تعطیل شدند و تنها خدمات اورژانس و بخش‌های ویژه مانند ICU

خروجی LoRA از لحن رسمی اطلاعیه‌های وزارت بهداشت استفاده می‌کند، اما ادعای بسیار شدیدی مبنی بر تعطیلی همه مراکز درمانی و بیمارستان‌های دولتی و خصوصی مطرح می‌کند. این ادعا چند بار با عبارت‌های تقریباً مشابه تکرار شده است و هیچ پشتیبانی‌ای در پرامپت ندارد.

وجود تکرار، [n] و ناتمام‌ماندن جمله پایانی نیز کیفیت خروجی را کاهش داده است. از آنجا که متن درباره دسترسی به خدمات درمانی تصمیم‌گیری می‌کند، این Hallucination فقط یک جزئیات کم‌اهمیت نیست و در صورت برداشت واقعی می‌تواند گمراه‌کننده و خطرناک باشد. در میان خروجی‌های پرامپت سوم، LoRA بیشترین خطر محتوایی را نشان می‌دهد.

خروجی Bottleneck Adapter:

> وزارت بهداشت امروز در بیانیه‌ای اعلام کرد: با توجه به شیوع ویروس کرونا و افزایش تعداد مبتلایان، از مردم می‌خواهیم که برای حفظ سلامت خود و دیگران، از رفتن به اماکن عمومی خودداری کنند. [n] وزارت بهداشت همچنین از مردم خواست تا در صورت داشتن علائم بیماری مانند سرفه، تب و تنگی نفس، به مراکز درمانی مراجعه نکنند و با شماره‌های ۱۱۱ و ۱۲۳ تماس بگیرند. [n] این وزارتخانه همچنین از مردم خواست تا در صورت داشتن علائم بیماری، از حضور در اجتماعات و مراسم مذهبی و غیر

خروجی Adapter نسبت به LoRA پیوستگی بیشتری دارد و به‌جای تعطیلی کامل مراکز درمانی، توصیه‌های عمومی سلامت را مطرح می‌کند. بااین‌حال، علائم، شماره‌های تماس و دستور مراجعه‌نکردن به مراکز درمانی همگی بدون منبع تولید شده‌اند.

به‌ویژه ارائه شماره تلفن و دستورالعمل پزشکی پشتیبانی‌نشده می‌تواند گمراه‌کننده باشد. بنابراین، هرچند شدت ادعای Adapter کمتر از LoRA است، خروجی آن نیز از نظر کاربرد واقعی قابل‌اعتماد نیست. متن همچنین با نشانه [n] همراه است و پیش از کامل‌شدن جمله آخر پایان می‌یابد.

۴-۵-۵- جمع‌بندی ارزیابی کیفی

هر سه مدل توانستند از آغازگرهای کوتاه، متن‌هایی با ساختار قابل‌تشخیص خبری تولید کنند. مدل پایه در بسیاری از موارد از لحن رسمی و قالب‌های رایج خبرگزاری استفاده کرد، اما با تکرار، عبارت‌های زائد و جزئیات دقیق پشتیبانی‌نشده همراه بود. این مدل در برخی خروجی‌ها متن‌هایی بسیار باورپذیر تولید کرد که می‌توانند به اشتباه واقعی تلقی شوند.

LoRA عموماً ساختار خبر را منظم‌تر و تخصصی‌تر کرد. در خروجی‌های پارلمانی و هواشناسی، متن آن از نظر توالی اطلاعات و لحن گزارش‌گونه مناسب بود. بااین‌حال، این روش جزئیات زمانی، جغرافیایی و سازمانی متعددی به متن افزود و نشانه [n] در هر سه خروجی آن مشاهده شد. همچنین در پرامپت سلامت، LoRA ادعایی شدید، تکراری و بالقوه خطرناک درباره تعطیلی مراکز درمانی تولید کرد.

Bottleneck Adapter نیز لحن رسمی و الگوی خبرنویسی را به‌خوبی بازتولید کرد و در پرامپت سلامت از نظر انسجام بهتر از LoRA بود. بااین‌حال، در پرامپت‌های مجلس و هواشناسی مقدار زیادی جزئیات عددی و جغرافیایی بدون پشتوانه تولید کرد. خروجی‌های آن نیز دارای [n] و پایان‌های ناتمام بودند.

از نظر Hallucination نمی‌توان یک مدل را در همه مثال‌ها به‌طور مطلق بهترین دانست. در پرامپت پارلمانی و هواشناسی، Adapter بیشترین جزئیات عددی یا جغرافیایی پشتیبانی‌نشده را ایجاد کرد. در پرامپت وزارت بهداشت، LoRA به‌وضوح بیشترین خطر را داشت؛ زیرا یک ادعای گسترده و تکرارشونده درباره تعطیلی مراکز درمانی تولید کرد. مدل پایه نیز در هر سه نمونه اعداد، نقل‌قول‌ها و انتساب‌های مشخصی را بدون شاهد ایجاد کرد.

در نتیجه، بهبود پرپلکسیتی اخبار لزوماً به معنای کاهش Hallucination یا تولید متن کاملاً قابل‌اعتماد نبود. LoRA و Bottleneck Adapter هر دو توانستند لحن خبری را تقویت کنند، اما تخصص‌یافتگی آن‌ها در برخی موارد با تولید جزئیات بیشتر و در نتیجه افزایش خطر اطلاعات تأییدنشده همراه شد. برای استفاده عملی از این مدل‌ها در تولید خبر، اتصال به یک منبع بازیابی معتبر، راستی‌آزمایی ادعاها و کنترل انسانی پیش از انتشار ضروری خواهد بود.

۴-۶- مقایسه منابع محاسباتی و زمان آموزش

علاوه بر کیفیت مدل نهایی، هزینه محاسباتی یکی از معیارهای مهم در انتخاب روش تنظیم دقیق است. کاهش تعداد پارامترهای قابل‌آموزش لزوماً به معنای کاهش یکسان زمان آموزش یا مصرف حافظه نیست؛ زیرا بخش عمده محاسبات همچنان به عبور پیش‌رو و پس‌انتشار از مدل پایه مربوط می‌شود و معماری ماژول‌های افزوده‌شده نیز بر فعال‌سازی‌های میانی و تعداد عملیات ماتریسی اثر می‌گذارد.

برای مقایسه منصفانه، LoRA و Bottleneck Adapter تحت شرایط آموزشی یکسان اجرا شدند. هر دو مدل سه دور کامل روی داده‌های اخبار، با ریزBatch برابر ۲، تجمیع گرادیان در چهار مرحله، محاسبات bfloat16 و Gradient Checkpointing آموزش دیدند. زمان‌سنجی فقط حلقه آموزش سه‌دوره‌ای را پوشش می‌دهد و ارزیابی پرپلکسیتی و تولید متن پس از آموزش در آن محاسبه نشده‌اند.

پیش از شروع هر آموزش، آمار بیشینه حافظه CUDA بازنشانی شد و همگام‌سازی GPU پیش و پس از زمان‌سنجی انجام گرفت. بنابراین، مقادیر ثبت‌شده برای دو روش تحت یک رویه اندازه‌گیری مشترک به دست آمده‌اند. گزارش Peak VRAM و زمان کل آموزش سه‌دوره‌ای و تحلیل اختلاف میان پیاده‌سازی PEFT و Adapter دستی از خواسته‌های مستقیم بخش چهارم تمرین است.

۴-۶-۱- مقایسه بیشینه حافظه GPU

علاوه بر کیفیت مدل نهایی، هزینه محاسباتی یکی از معیارهای مهم در انتخاب روش تنظیم دقیق است. کاهش تعداد پارامترهای قابل‌آموزش لزوماً به کاهش متناسب زمان آموزش یا مصرف حافظه منجر نمی‌شود؛ زیرا بخش عمده محاسبات همچنان به عبور پیش‌رو و پس‌انتشار از مدل پایه مربوط است. علاوه بر این، محل تزریق ماژول‌های جدید، نوع داده پارامترهای قابل‌آموزش، فعال‌سازی‌های میانی و تعداد عملیات ماتریسی نیز بر مصرف منابع اثر می‌گذارند.

برای مقایسه منصفانه، LoRA و Bottleneck Adapter با تنظیمات اصلی آموزشی یکسان اجرا شدند. هر دو روش سه دور کامل روی ۱۰,۰۰۰ متن آموزشی اخبار فارسی، با حداکثر طول توالی ۲۵۶ توکن، اندازه ریزBatch برابر ۴، تجمیع گرادیان در دو مرحله و اندازه مؤثر Batch برابر ۸ آموزش دیدند. نوع داده مدل پایه float16 بود، پارامترهای قابل‌آموزش در هر دو روش در float32 نگهداری شدند و Gradient Checkpointing نیز فعال بود.

با توجه به اندازه ریزBatch برابر ۴، در هر دور ۲,۵۰۰ ریزBatch پردازش شد. در نتیجه، طی سه دور آموزش مجموعاً ۷,۵۰۰ ریزBatch پردازش شد. ازآنجاکه گرادیان‌ها در دو مرحله تجمیع می‌شدند، بهینه‌ساز در هر دور ۱,۲۵۰ بار و در کل آموزش ۳,۷۵۰ بار به‌روزرسانی شد.

در نوت‌بوک، زمان آموزش و زمان اعتبارسنجی به‌صورت جداگانه ثبت شدند. اعتبارسنجی شامل دو بخش بود: یک Mini-validation پس از هر ۲۰ ریزBatch آموزشی و ارزیابی کامل مجموعه Dev در پایان هر دور. زمان صرف‌شده برای Mini-validation از زمان خالص آموزش کسر شد و همراه با زمان ارزیابی کامل Dev در متغیر جداگانه زمان Validation قرار گرفت. بنابراین، زمان آموزش گزارش‌شده در این بخش فقط عملیات بهینه‌سازی مدل را شامل می‌شود و تحت تأثیر اجرای دوره‌ای Validation قرار نگرفته است.

آمار بیشینه حافظه CUDA نیز پیش از اندازه‌گیری بازنشانی شد و Peak VRAM آموزش و Validation به‌صورت جداگانه ثبت گردید. برای این منظور از torch.cuda.max_memory_allocated() استفاده شد که بیشترین حافظه اختصاص‌یافته به Tensorهای PyTorch را گزارش می‌کند.

نتایج حافظه آموزش و اعتبارسنجی در جدول ۴-۵ ارائه شده‌اند.

جدول ۴-۵: مقایسه بیشینه حافظه تخصیص‌یافته ***GPU*** *در آموزش* ***LoRA*** *و* ***Bottleneck Adapter***

روش	***Peak VRAM*** **آموزش**	***Peak VRAM*** **اعتبارسنجی**
LoRA	15.360205 GB	11.440088 GB
Bottleneck Adapter	15.308016 GB	11.390708 GB

در مرحله آموزش، Bottleneck Adapter حدود ۰.۰۵۲۱۸۹ گیگابایت حافظه کمتر از LoRA مصرف کرد:

$$\Delta VRAM_{Train} = 15.360205 - 15.308016 = 0.052189\ GB$$

این اختلاف نسبت به Peak VRAM آموزش LoRA برابر با حدود ۰.۳۴ درصد است. بنابراین، مصرف حافظه آموزش دو روش تقریباً یکسان بود و اختلاف حدود ۵۲ مگابایتی آن‌ها از نظر عملی بسیار محدود محسوب می‌شود. بااین‌حال، برخلاف اجرای قبلی، Bottleneck Adapter در اجرای نهایی اندکی حافظه کمتری از LoRA ثبت کرد.

در مرحله اعتبارسنجی نیز Peak VRAM مربوط به Adapter اندکی پایین‌تر بود. این مقدار معادل کاهش حدود ۰.۴۳ درصدی نسبت به LoRA است.

$$\Delta VRAM_{Validation} = 11.440088 - 11.390708 = 0.049380\ GB$$

نزدیک‌بودن مصرف حافظه دو روش نشان می‌دهد که در این آزمایش، حافظه موردنیاز بیشتر تحت تأثیر مدل پایه، فعال‌سازی‌های بلوک‌های Transformer، طول توالی، اندازه ریزBatch و Gradient Checkpointing قرار داشته است تا صرفاً تعداد پارامترهای قابل‌آموزش.

در LoRA، پارامترهای کم‌رتبه به ۱۰۰ نگاشت خطی در بخش Attention افزوده شدند. در Bottleneck Adapter نیز ۳۰ مسیر Down–GELU–Up پس از MLPها اجرا شدند. مسیر Adapter در این پیاده‌سازی برای جلوگیری از سرریز عددی در float32 اجرا شد و سپس خروجی آن به نوع داده مسیر اصلی بازگردانده شد. با وجود این محاسبات اضافی و فعال‌سازی میانی، Peak VRAM مربوط به Adapter از LoRA بیشتر نشد و حتی مقدار بسیار اندکی پایین‌تر باقی ماند.

این نتیجه نشان می‌دهد کاهش تعداد پارامترهای قابل‌آموزش Adapter به صرفه‌جویی چشمگیری در حافظه منجر نشده است؛ زیرا سهم عمده حافظه همچنان به عبور از مدل پایه و محاسبه گرادیان از میان بلوک‌های آن اختصاص دارد.

همچنین باید توجه داشت که max_memory_allocated() فقط حافظه اختصاص‌یافته به Tensorهای PyTorch را اندازه‌گیری می‌کند. حافظه رزروشده توسط CUDA Caching Allocator، CUDA Context، کتابخانه‌ها، Kernelها، Fragmentation و سایر فرایندهای سیستم به‌طور کامل در این مقدار منعکس نمی‌شود. بنابراین، اعداد جدول نباید با کل حافظه اشغال‌شده‌ای که ابزارهایی مانند nvidia-smi نمایش می‌دهند یکسان در نظر گرفته شوند.

۴-۶-۲- مقایسه زمان آموزش و اعتبارسنجی

زمان خالص آموزش از زمان صرف‌شده برای Validation جدا شد. زمان آموزش شامل عبور پیش‌رو، محاسبه زیان، پس‌انتشار، تجمیع گرادیان، Gradient Clipping و به‌روزرسانی بهینه‌ساز است. زمان Validation نیز مجموع Mini-validationهای دوره‌ای و سه ارزیابی کامل مجموعه Dev در پایان دورها را شامل می‌شود.

جدول ۴-۶: مقایسه زمان آموزش و اعتبارسنجی ***LoRA*** *و* ***Bottleneck Adapter***

روش	زمان خالص آموزش	زمان تقریبی آموزش	زمان اعتبارسنجی	زمان تقریبی اعتبارسنجی
LoRA	14,418.930916ثانیه	۴ ساعت و ۱۹ ثانیه	۵۸۵.۳۳۲۸۵۶ ثانیه	۹ دقیقه و ۴۵ ثانیه
Bottleneck Adapter	11,224.265415ثانیه	۳ ساعت و ۷ دقیقه و ۴ ثانیه	۷۶۵.۸۶۸۲۶۷ ثانیه	۱۲ دقیقه و ۴۶ ثانیه

آموزش Bottleneck Adapter حدود ۳,۱۹۴.۶۶۵۵۰۱ ثانیه زودتر از LoRA به پایان رسید:

$$\Delta t = 14{,}418.93 - 11{,}224.26 = 3{,}194.66s$$

این اختلاف تقریباً برابر با ۵۳ دقیقه و ۱۵ ثانیه است. در نتیجه، زمان خالص آموزش Bottleneck Adapter نسبت به LoRA حدود ۲۲.۱۶ درصد کمتر بود:

$$Training\ Time\ Reduction = \frac{14{,}418.93 - 11{,}224.26}{14{,}418.93} \times 100 \approx 22.16\%$$

بنابراین، اختلاف زمان آموزش در اجرای نهایی دیگر محدود به چند درصد نیست و Bottleneck Adapter به‌طور محسوسی سریع‌تر از LoRA آموزش دیده است.

در مقابل، زمان اعتبارسنجی Adapter حدود ۱۸۰.۵۳۵۴۱۱ ثانیه بیشتر از LoRA بود. این اختلاف تقریباً برابر با ۳ دقیقه و یک ثانیه و معادل افزایش حدود ۳۰.۸۴ درصدی زمان Validation است.

بااین‌حال، زمان‌های Validation دو روش مستقیماً برای سنجش سرعت معماری قابل‌مقایسه نیستند. Mini-validation هر دو روش با Batch برابر ۲ اجرا شد، اما ارزیابی کامل مجموعه Dev برای LoRA با Batch برابر ۴ و برای Bottleneck Adapter با Batch برابر ۲ انجام شد. انتخاب Batch محافظه‌کارانه‌تر برای Adapter تعداد Batchهای ارزیابی کامل را افزایش داده و می‌تواند بخش مهمی از زمان Validation بیشتر آن را توضیح دهد. بنابراین، از این اختلاف نمی‌توان نتیجه گرفت که استنتاج یا ارزیابی Adapter ذاتاً کندتر از LoRA است.

مجموع زمان خالص آموزش و Validation در LoRA برابر بود با:

$$14{,}418.93 + 585.33 = 15{,}004.26s$$

این مقدار تقریباً معادل ۴ ساعت و ۱۰ دقیقه و ۴ ثانیه است. برای Bottleneck Adapter نیز داریم:

$$11{,}224.26 + 765.86 = 11{,}990.13s$$

این مقدار تقریباً برابر با ۳ ساعت و ۱۹ دقیقه و ۵۰ ثانیه است. بنابراین، حتی با احتساب زمان Validation، اجرای Adapter حدود ۳,۰۱۴.۱۳۰۰۹۰ ثانیه، معادل تقریباً ۵۰ دقیقه و ۱۴ ثانیه، کوتاه‌تر بوده است. بااین‌حال، به دلیل تفاوت Batch ارزیابی

کامل، این مقایسه مجموع زمان باید به‌عنوان توصیف زمان اجرای ثبت‌شده در همین آزمایش تفسیر شود، نه یک مقایسه کاملاً کنترل‌شده از سرعت Validation.

مدت‌زمان آموزش دو روش تا حد زیادی به اندازه مدل پایه، پردازش توالی‌های حداکثر ۲۵۶توکنی، اجرای ۷,۵۰۰ ریزBatch و فعال‌بودن Gradient Checkpointing مربوط است. Gradient Checkpointing بخشی از فعال‌سازی‌ها را در حافظه نگهداری نمی‌کند و آن‌ها را هنگام پس‌انتشار دوباره محاسبه می‌کند. این روش مصرف حافظه را کاهش می‌دهد، اما زمان آموزش را افزایش می‌دهد.

۴-۶-۳- تحلیل فنی اختلاف مصرف منابع و سرعت آموزش

تفاوت منابع دو روش را نمی‌توان فقط با تعداد پارامترهای قابل‌آموزش توضیح داد. LoRA دارای ۵,۵۷۰,۵۶۰ پارامتر قابل‌آموزش است و تغییرات کم‌رتبه را به ۱۰۰ نگاشت Query، Key، Value و Output در بلوک‌های Attention اضافه می‌کند. در نتیجه، در تعداد زیادی از نقاط معماری، عملیات ماتریسی اضافی در هر عبور پیش‌رو انجام می‌شود.

در مقابل، Bottleneck Adapter دارای ۲,۰۲۸,۰۰۰ پارامتر قابل‌آموزش و ۳۰ نقطه تزریق است. هر Adapter پس از خروجی MLP یک مسیر شامل Down-projection، GELU و Up-projection اجرا می‌کند. بنابراین، تعداد نقاط تزریق آن کمتر از LoRA است، هرچند هر نقطه شامل یک مسیر پردازشی کامل و یک فعال‌سازی میانی جدید است.

تعداد کمتر پارامترهای قابل‌آموزش و نقاط تزریق محدودتر می‌تواند یکی از دلایل کاهش زمان آموزش Bottleneck Adapter باشد. بااین‌حال، از نتایج یک اجرا نمی‌توان سهم دقیق هر عامل را جدا کرد. تفاوت محل تزریق، شکل ماتریس‌ها، هزینه Kernelها، پیاده‌سازی کتابخانه PEFT و اجرای دستی Adapter همگی می‌توانند بر سرعت اثر بگذارند.

کتابخانه PEFT تزریق و مدیریت ماژول‌های LoRA، فریز وزن‌های اصلی و ذخیره پارامترهای جدید را ساده می‌کند؛ اما عملیات کم‌رتبه افزوده‌شده همچنان باید در هر Batch محاسبه شوند. از طرف دیگر، پیاده‌سازی Adapter به‌صورت دستی انجام شد و فاقد Kernel اختصاصی یا Fused برای ترکیب MLP و مسیر Adapter بود. لایه‌های Down-projection، GELU و Up-projection به‌صورت عملیات جداگانه اجرا شدند و شاخه Adapter نیز برای پایداری عددی در float32 محاسبه شد. با وجود این سربار، زمان خالص آموزش Adapter حدود ۲۲.۱۶ درصد کمتر از LoRA ثبت شد.

استفاده از کتابخانه PEFT لزوماً به این معنا نیست که تمام عملیات LoRA در قالب Kernelهای ادغام‌شده و سریع‌تر اجرا می‌شوند. در این پیاده‌سازی، PEFT ماژول‌های خطی هدف را با ساختاری شامل مسیر اصلی و شاخه کم‌رتبه LoRA مدیریت می‌کند و در هر عبور پیش‌رو، خروجی شاخه LoRA پس از اعمال مقیاس به خروجی نگاشت اصلی افزوده می‌شود. بنابراین، برای هر یک از ۱۰۰ نگاشت هدف، عملیات ماتریسی اضافی و فراخوانی ماژول مربوط به LoRA همچنان وجود دارد. مزیت اصلی PEFT در این آزمایش، استانداردسازی فرایند تزریق ماژول‌ها، تعیین دقیق پارامترهای قابل‌آموزش و سازگاری مطمئن با معماری مدل بود، نه حذف هزینه محاسباتی شاخه‌های کم‌رتبه.

در مقابل، Adapter دستی پس از MLP هر بلوک، ورودی را از float16 به float32 تبدیل می‌کند، مسیر Down–GELU–Up را خارج از Autocast اجرا می‌کند و سپس خروجی را دوباره به نوع داده مسیر اصلی بازمی‌گرداند. این تبدیل نوع داده، اجرای سه

عملیات جداگانه و ایجاد فعال‌سازی میانی می‌توانند تعداد Kernel launchها، انتقال داده و سربار حافظه را افزایش دهند. بااین‌حال، Adapter فقط در ۳۰ نقطه تزریق شده و ۲,۰۲۸,۰۰۰ پارامتر قابل‌آموزش دارد؛ درحالی‌که LoRA در ۱۰۰ نگاشت فعال است و ۵,۵۷۰,۵۶۰ پارامتر قابل‌آموزش دارد. در اجرای حاضر، کاهش تعداد نقاط تزریق و پارامترهای قابل‌آموزش Adapter ظاهراً بر سربار ناشی از اجرای دستی و محاسبات float32 غلبه کرده و زمان آموزش کمتری ایجاد کرده است.

تعداد بیشتر پارامترهای قابل‌آموزش LoRA علاوه بر هزینه محاسباتی Forward و Backward، موجب افزایش حافظه موردنیاز برای گرادیان‌ها و حالت‌های بهینه‌ساز می‌شود. برای نمونه، بهینه‌ساز AdamW برای هر پارامتر قابل‌آموزش معمولاً دو حالت ممان اول و دوم نگهداری می‌کند. بااین‌حال، چون حافظه مدل پایه و فعال‌سازی‌های آن سهم غالب را تشکیل می‌دهند، اختلاف ۶۳.۵۹ درصدی تعداد پارامترهای قابل‌آموزش فقط به اختلاف ۰.۳۴ درصدی در Peak VRAM منجر شده است.

از نظر حافظه، اختلاف دو روش بسیار کوچک‌تر از اختلاف زمان بود. Adapter تنها حدود ۰.۰۵۲ گیگابایت حافظه آموزشی کمتر مصرف کرد که معادل حدود ۰.۳۴ درصد است. این نتیجه تأیید می‌کند که تعداد کمتر پارامترهای قابل‌آموزش الزاماً به کاهش مشابه Peak VRAM منجر نمی‌شود؛ زیرا حافظه فعال‌سازی‌های مدل پایه، حالت‌های بهینه‌ساز، گرادیان‌ها و محاسبات میانی سهم مهمی از مصرف حافظه را تشکیل می‌دهند.

همچنین، این نتایج مربوط به یک اجرای کامل از هر روش هستند. عواملی مانند وضعیت اولیه GPU، گرم‌شدن CUDA، زمان‌بندی Kernelها، Fragmentation حافظه، عملکرد DataLoader و بار سیستم می‌توانند زمان و Peak VRAM ثبت‌شده را تا حدی تغییر دهند. برای نتیجه‌گیری عمومی درباره سرعت دو روش، تکرار اجراها و گزارش میانگین و انحراف معیار لازم است.

بنابراین، نتیجه دقیق این آزمایش آن است که Bottleneck Adapter از نظر زمان خالص آموزش به‌طور محسوسی سریع‌تر از LoRA بود و در عین حال Peak VRAM آن نیز مقدار بسیار اندکی پایین‌تر ثبت شد. برتری اصلی Adapter در منابع محاسباتی مربوط به کاهش حدود ۲۲.۱۶ درصدی زمان آموزش است؛ درحالی‌که اختلاف حافظه دو روش کمتر از نیم درصد بود و از نظر عملی ناچیز محسوب می‌شود.

در مجموع، Bottleneck Adapter با پارامترهای قابل‌آموزش کمتر، زمان آموزشی کوتاه‌تر و Peak VRAM تقریباً مشابه LoRA، در اجرای حاضر وضعیت محاسباتی مطلوب‌تری داشت. بااین‌حال، زمان Validation بیشتر آن را نباید بدون توجه به Batch کوچک‌تر ارزیابی کامل، به ساختار Adapter نسبت داد. این نتایج در کنار پرپلکسیتی، کیفیت تولید متن و روند همگرایی برای قضاوت نهایی درباره دو روش استفاده خواهند شد.

۴-۷- تحلیل منحنی‌های زیان آموزش

برای بررسی روند یادگیری، همگرایی و احتمال بیش‌برازش در روش‌های LoRA و Bottleneck Adapter، زیان Cross-Entropy آموزش در تمام ریزBatchها ثبت شد. علاوه بر آن، دو نوع ارزیابی اعتبارسنجی در طول آموزش انجام گرفت:

۱. Mini-validation دوره‌ای: پس از هر ۲۰ گام آموزشی، زیان روی یک ریزBatch کوچک از مجموعه Dev محاسبه شد تا تغییرات میان‌دوره‌ای مدل قابل مشاهده باشد.

۲. Full-dev validation: در پایان هر دوره، مدل روی کل مجموعه Dev ارزیابی شد و زیان و پرپلکسیتی رسمی اعتبارسنجی به دست آمد.

مجموعه آموزش شامل ۱۰٬۰۰۰ نمونه و اندازه ریزBatch برابر ۴ بود. بنابراین، هر دوره شامل ۲۵۰۰ ریزگام آموزشی و کل آموزش سه‌دوره‌ای شامل ۷۵۰۰ ریزگام بود. با توجه به تجمیع گرادیان در دو مرحله، بهینه‌ساز در هر دوره ۱۲۵۰ بار و در کل آموزش ۳۷۵۰ بار پارامترها را به‌روزرسانی کرد.

بنابراین، منظور از Training step روی محور افقی نمودارها، شماره ریزBatch آموزشی پردازش‌شده است، نه تعداد به‌روزرسانی‌های بهینه‌ساز.

در هر نمودار، نقاط Full-dev دقیقاً در پایان دوره‌های آموزشی قرار گرفته‌اند:

- گام ۲۵۰۰: پایان Epoch 1؛
- گام ۵۰۰۰: پایان Epoch 2؛
- گام ۷۵۰۰: پایان Epoch 3.

در نمودار LoRA این نقاط با دایره‌های سبز و در نمودار Bottleneck Adapter با مربع‌های بنفش نشان داده شده‌اند. این نقاط نمایانگر زیان محاسبه‌شده روی کل مجموعه Dev هستند و برخلاف منحنی Mini-validation، تنها به یک ریزBatch کوچک وابسته نیستند.

زیان خام آموزش به دلیل تفاوت طول و دشواری متن‌های موجود در ریزBatchها دارای نوسان است. برای آشکارشدن روند اصلی، منحنی آموزش با میانگین متحرک ۱۰۰مرحله‌ای هموار شد:

$$\bar{\mathcal{L}}^{Train} = \frac{1}{100} \sum_{i=t=99}^{t} \mathcal{L}_i^{Train}$$

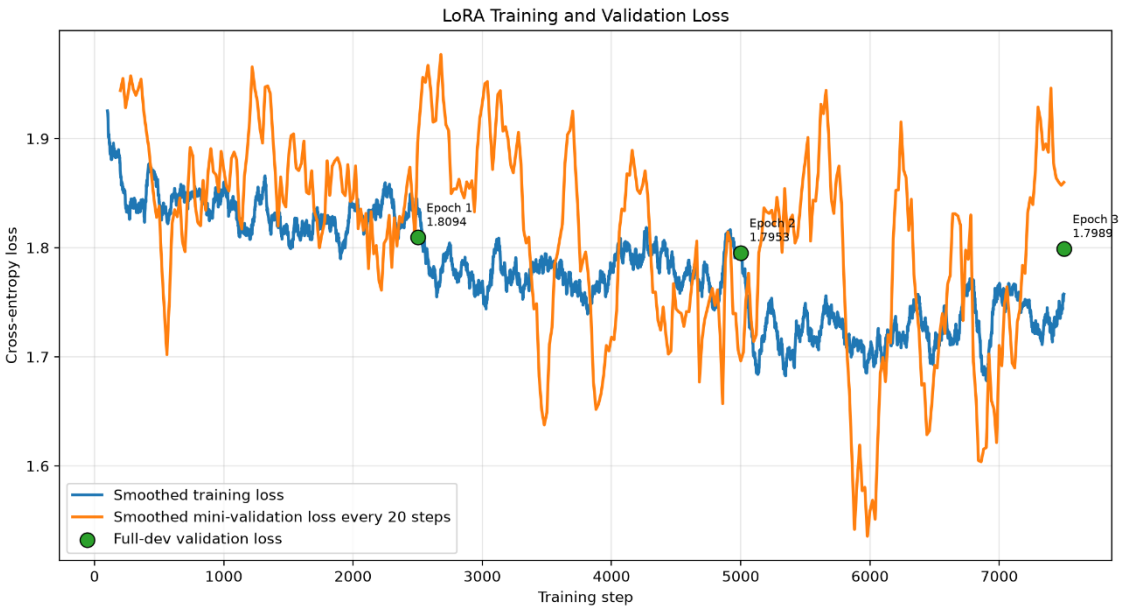
Mini-validation پس از هر ۲۰ گام آموزشی اجرا شد. در نتیجه، در هر دوره ۱۲۵ نقطه و طی سه دوره ۳۷۵ نقطه Mini-validation ثبت شد. این مقادیر برای نمایش نمودار با پنجره‌ای شامل ۱۰ مشاهده اعتبارسنجی هموار شدند:

$$\bar{\mathcal{L}}_j^{mini-val} = \frac{1}{10} \sum_{i=j-9}^{j} \mathcal{L}_i^{mini-val}$$

این هموارسازی‌ها فقط برای نمایش نمودار انجام شده‌اند و در محاسبه گرادیان، به‌روزرسانی مدل یا مقادیر رسمی Full-dev نقشی ندارند. همچنین، به دلیل آنکه هر نقطه Mini-validation فقط بر اساس یک ریزBatch با اندازه ۲ محاسبه شده است، نوسانات خط نارنجی نباید به‌اندازه نقاط Full-dev قابل‌اعتماد تلقی شوند.

۴-۷-۱- تحلیل منحنی آموزش و اعتبارسنجی LoRA

در شکل ۴-۱، خط آبی زیان هموارشده آموزش، خط نارنجی زیان هموارشده Mini-validation و دایره‌های سبز زیان Full-dev در پایان سه دوره را نشان می‌دهند.



*شکل ۴–۱: منحنی زیان آموزش، **Mini-validation** و نقاط **Full-dev** روش **LoRA** طی سه دوره*

در ابتدای آموزش، زیان هموارشده LoRA در محدوده‌ای بالاتر از ۱/۹ قرار دارد و در چندصد گام نخست با سرعت کاهش پیدا می‌کند. پس از این افت اولیه، منحنی وارد روندی تدریجی‌تر می‌شود و با وجود نوسانات موضعی، سطح کلی آن در دوره‌های دوم و سوم پایین‌تر از دوره اول قرار می‌گیرد.

میانگین زیان آموزش LoRA در سه دوره به‌ترتیب برابر بود با:

1.837921 → 1.778811 → 1.726314

کاهش زیان آموزش از دوره اول به دوم برابر با حدود ۳/۲۲ درصد بود و کاهش دوره دوم به سوم نیز حدود ۲/۹۵ درصد بود و در مجموع، میانگین زیان آموزش LoRA از دوره اول تا سوم حدود ۶/۰۷ درصد کاهش یافت. این کاهش پیوسته نشان می‌دهد فرایند بهینه‌سازی پایدار بوده و LoRA در تمام سه دوره همچنان داده‌های آموزش را بهتر برازش کرده است.

بااین‌حال، نقاط سبز Full-dev رفتار متفاوتی را نشان می‌دهند. نتایج کامل اعتبارسنجی LoRA عبارت بودند از:

1.809363 → 1.795328 → 1.798904

پرپلکسیتی‌های متناظر نیز برابر بودند با:

6.106555 → 6.021451 → 6.043023

نقطه اول در گام ۲۵۰۰، نقطه دوم در گام ۵۰۰۰ و نقطه سوم در گام ۷۵۰۰ قرار دارد و به‌ترتیب پایان Epochهای اول، دوم و سوم را نشان می‌دهد.

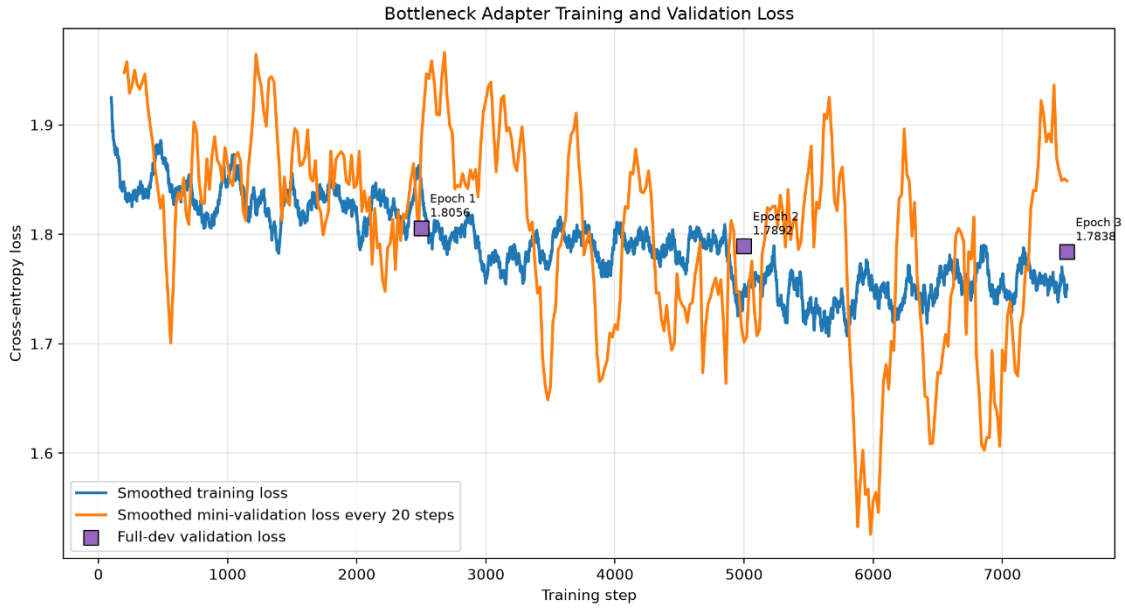
زیان Full-dev از دوره اول به دوم کاهش یافت اما در دوره سوم اندکی افزایش پیدا کرد. این افزایش معادل حدود ۰/۲۰ درصد نسبت به دوره دوم است. بنابراین، بهترین نتیجه اعتبارسنجی LoRA در پایان Epoch 2 به دست آمده است. در Epoch 3، با وجود ادامه کاهش زیان آموزش، زیان و پرپلکسیتی Full-dev اندکی افزایش یافته‌اند.

این واگرایی کوچک میان روند آموزش و اعتبارسنجی، یک نشانه خفیف از شروع بیش‌برازش یا توقف بهبود تعمیم است. شدت این تغییر کم است و نمی‌توان آن را بیش‌برازش شدید دانست، اما نشان می‌دهد ادامه آموزش پس از Epoch 2 برای LoRA سودی در عملکرد Dev ایجاد نکرده است.

نوسانات زیاد خط نارنجی نیز به دلیل استفاده از ریزBatch کوچک اعتبارسنجی طبیعی است. برخی نقاط این منحنی به‌طور موقت بسیار کاهش یا افزایش یافته‌اند، اما این تغییرات محلی به‌تنهایی نشان‌دهنده بهبود یا افت کلی مدل نیستند. برای انتخاب بهترین دوره، نقاط Full-dev معیار معتبرتری هستند.

۴-۷-۲- تحلیل منحنی آموزش و اعتبارسنجی Bottleneck Adapter

در شکل ۴-۲، خط آبی زیان هموارشده آموزش، خط نارنجی زیان هموارشده Mini-validation و مربع‌های بنفش نتایج Full-dev را در پایان دوره‌های اول، دوم و سوم نشان می‌دهند.



*شکل ۴–۲: منحنی زیان آموزش، **Mini-validation** و نقاط **Full-dev** روش **Bottleneck Adapter** طی سه دوره*

مانند LoRA، زیان Bottleneck Adapter نیز در ابتدای آموزش با سرعت کاهش پیدا می‌کند و پس از آن وارد روندی آرام‌تر می‌شود. با عبور از مرز دوره اول در گام ۲۵۰۰ و دوره دوم در گام ۵۰۰۰، سطح کلی منحنی آموزش به‌تدریج پایین‌تر می‌آید.

میانگین زیان آموزش Adapter در سه دوره برابر بود با:

$$1.835557 \rightarrow 1.786484 \rightarrow 1.750007$$

کاهش زیان آموزش از دوره اول به دوم حدود ۲/۶۷ درصد و کاهش دوره دوم به سوم نیز حدود ۲/۰۴ درصد بود. در مجموع، زیان آموزش Adapter از دوره اول تا سوم حدود ۴/۶۶ درصد کاهش یافت. کاهش دوره دوم به سوم از کاهش دوره اول به دوم کوچک‌تر است؛ بنابراین، سرعت کاهش زیان در مراحل پایانی کمتر شده و مدل به ناحیه باثبات‌تری نزدیک شده است. بااین‌حال، روند آموزش هنوز کاملاً افقی نیست.

زیان‌های Full-dev مربوط به مربع‌های بنفش در پایان سه دوره برابر بودند با:

$$1.805581 \rightarrow 1.789198 \rightarrow 1.783826$$

پرپلکسیتی‌های متناظر نیز به‌صورت زیر به دست آمدند:

$$6.083507 \rightarrow 5.984649 \rightarrow 5.952586$$

زیان Full-dev از دوره اول به دوم حدود ۰/۰۱۶۳۸۳ واحد کاهش و از دوره دوم به سوم نیز کاهش ادامه یافت.

اگرچه مقدار بهبود Epoch 3 کمتر از Epoch 2 است، هیچ بازگشت صعودی در زیان اعتبارسنجی مشاهده نمی‌شود. بنابراین، بهترین نتیجه Full-dev Adapter در پایان Epoch 3 به دست آمده است.

ادامه کاهش هم‌زمان زیان آموزش و Full-dev نشان می‌دهد Bottleneck Adapter تا پایان دوره سوم نشانه آشکاری از بیش‌برازش درون‌دامنه‌ای ندارد. بااین‌حال، کاهش کوچک‌تر در دوره سوم نشان می‌دهد مدل به مرحله بازده نزولی نزدیک شده است؛ یعنی ادامه آموزش ممکن است همچنان بهبود ایجاد کند، اما احتمالاً این بهبود از دوره‌های نخست کمتر خواهد بود.

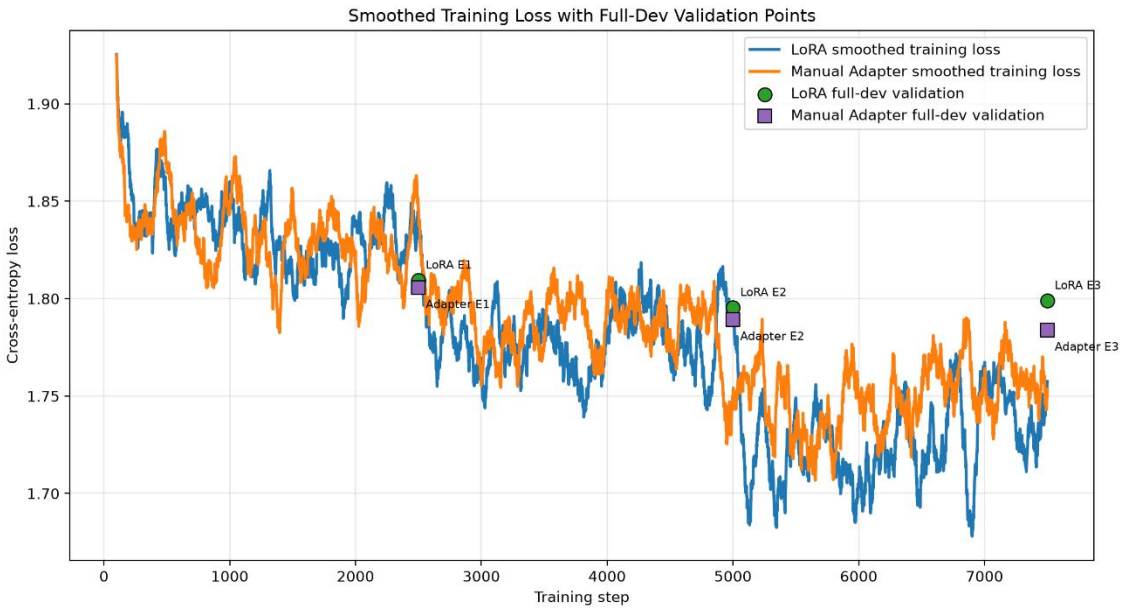
مانند نمودار LoRA، خط نارنجی Mini-validation دارای نوسانات زیادی است. افت‌های عمیق یا قله‌های کوتاه این خط به دلیل کوچک‌بودن Batch اعتبارسنجی هستند و نباید به‌عنوان تغییر پایدار در کیفیت مدل تفسیر شوند.

۴-۷-۳- مقایسه مستقیم روند آموزش دو روش

برای مقایسه مستقیم، منحنی‌های هموارشده آموزش LoRA و Bottleneck Adapter همراه با نقاط Full-dev هر دو روش در شکل ۴-۳ قرار گرفته‌اند.

در این شکل:

- خط آبی: زیان آموزش هموارشده LoRA؛
- خط نارنجی: زیان آموزش هموارشده Bottleneck Adapter؛
- دایره‌های سبز: Full-dev روش LoRA در پایان Epochهای اول، دوم و سوم؛
- مربع‌های بنفش: Full-dev روش Adapter در پایان Epochهای اول، دوم و سوم.



شکل ۴–۳: مقایسه منحنی زیان آموزش و نقاط ***Full-dev*** *روش‌های* ***LoRA*** *و* ***Bottleneck Adapter***

نقاط اعتبارسنجی روی این نمودار دقیقاً در گام‌های ۲۵۰۰، ۵۰۰۰ و ۷۵۰۰ قرار دارند؛ بنابراین، این نقاط به‌ترتیب نتایج ارزیابی کل Dev در پایان Epochهای اول، دوم و سوم هستند، نه نقاط Mini-validation دوره‌ای.

نتایج عددی آموزش و Full-dev در جدول ۴-۷ خلاصه شده‌اند.

جدول ۴–۷: مقایسه زیان آموزش و اعتبارسنجی ***LoRA*** *و* ***Bottleneck Adapter*** *در پایان هر دوره*

روش	دوره	میانگین زیان آموزش	زیان Full-dev	پرپلکسیتی Full-dev
LoRA	1	1.837921	1.809363	6.106555
LoRA	2	1.778811	1.795328	6.021451
LoRA	3	1.726314	1.798904	6.043023
Bottleneck Adapter	1	1.835557	1.805581	6.083507
Bottleneck Adapter	2	1.786484	1.789198	5.984649
Bottleneck Adapter	3	1.750007	1.783826	5.952586

در دوره اول، میانگین زیان آموزش دو روش بسیار نزدیک بود و Adapter مقدار اندکی کمتر ثبت کرد. از دوره دوم به بعد، LoRA زیان آموزش کمتری داشت. اختلاف در دوره دوم برابر با ۰/۰۰۷۶۷۳ و در دوره سوم برابر با ۰/۰۲۳۶۹۳ بود. بنابراین، LoRA با ادامه آموزش داده‌های Train را قوی‌تر از Adapter برازش کرده است.

بااین‌حال، نتایج Full-dev در هر سه دوره به نفع Bottleneck Adapter بودند. اختلاف زیان Full-dev میان دو روش در دوره‌های اول تا سوم به‌ترتیب برابر بود با:

$$1.809363 - 1.805581 = 0.003782$$

$1.795328 - 1.789198 = 0.006130$

$1.798904 - 1.783826 = 0.015078$

این اختلاف از دوره اول تا سوم افزایش یافته است. بنابراین، با وجود آنکه LoRA به زیان آموزشی پایین‌تری رسیده، Adapter در هر سه دوره زیان اعتبارسنجی پایین‌تری ثبت کرده است.

این الگو نشان می‌دهد LoRA داده‌های آموزش را تهاجمی‌تر برازش کرده است، اما کاهش بیشتر زیان Train آن به تعمیم بهتر روی Dev منجر نشده است. در مقابل، Bottleneck Adapter با زیان آموزشی بالاتر، عملکرد اعتبارسنجی بهتری ارائه کرده است. این نتیجه با پرپلکسیتی بهتر Adapter روی آزمون اخبار دیده‌نشده نیز سازگار است.

البته، میانگین زیان آموزش و Full-dev دقیقاً با یک روش تجمیع محاسبه نشده‌اند؛ زیان دوره آموزشی از میانگین زیان ریزBatchها به دست آمده، درحالی‌که Full-dev بر اساس تمام توکن‌های معتبر مجموعه Dev محاسبه شده است. بنابراین، اختلاف مطلق میان این دو مقدار نباید به‌تنهایی به‌عنوان شکاف تعمیم تفسیر شود. جهت تغییر آن‌ها در طول دوره‌ها برای تحلیل همگرایی و بیش‌برازش اهمیت بیشتری دارد.

۴-۷-۴- بررسی همگرایی دو روش

در هر دو روش، زیان آموزش از دوره اول تا سوم کاهش یافته و هیچ نشانه‌ای از واگرایی، انفجار زیان یا ناپایداری پایدار دیده نمی‌شود. بنابراین، فرایند بهینه‌سازی هر دو روش موفق و باثبات بوده است.

بااین‌حال، رفتار اعتبارسنجی آن‌ها متفاوت است.

در LoRA، زیان آموزش در دوره سوم همچنان کاهش می‌یابد، اما بهترین زیان Full-dev در پایان دوره دوم ثبت شده و در دوره سوم اندکی افزایش یافته است. بنابراین، از دید تابع هدف آموزش، LoRA هنوز به Plateau کامل نرسیده؛ اما از دید تعمیم روی Dev، مدل در حدود Epoch 2 به بهترین نقطه خود رسیده و ادامه آموزش سود بیشتری ایجاد نکرده است.

در Bottleneck Adapter، هم زیان آموزش و هم زیان Full-dev در هر سه دوره کاهش یافته‌اند. بنابراین، این روش تا پایان دوره سوم هنوز بهبود اعتبارسنجی نشان می‌دهد. بااین‌حال، مقدار بهبود دوره سوم کمتر از دوره دوم است و مدل به ناحیه همگرایی نسبی نزدیک شده است.

در نتیجه:

- LoRA از نظر زیان آموزش هنوز در حال یادگیری است، اما عملکرد Dev آن در Epoch 3 متوقف شده و اندکی بازگشته است.
- Bottleneck Adapter هنوز در Epoch 3 بهبود Dev دارد، ولی سرعت این بهبود کاهش یافته است.

بنابراین، اصطلاح «همگرایی کامل» برای هیچ‌یک از دو روش قابل اثبات نیست، اما Adapter به روند باثبات‌تری از نظر تعمیم رسیده است.

۴-۷-۵- کفایت سه دور آموزش

برای هدف اصلی تمرین، سه دوره آموزش کافی بوده است؛ زیرا هر دو روش توانستند پرپلکسیتی آزمون اخبار را نسبت به مدل پایه کاهش دهند و روند یادگیری پایدار ایجاد کنند.

بااین‌حال، کفایت سه دوره برای دو روش معنای یکسانی ندارد.

برای LoRA، بهترین Full-dev loss در پایان Epoch 2 ثبت شد. Epoch 3 زیان آموزش را بیشتر کاهش داد، اما زیان و پرپلکسیتی اعتبارسنجی را اندکی افزایش داد. بنابراین، بر اساس معیار Dev، آموزش سه‌دوره‌ای برای LoRA اندکی بیشتر از نقطه بهینه اعتبارسنجی ادامه یافته است. در یک پیاده‌سازی مبتنی بر Early Stopping، checkpoint پایان Epoch 2 انتخاب مناسب‌تری بود.

برای Bottleneck Adapter، بهترین زیان و پرپلکسیتی Full-dev در پایان Epoch 3 ثبت شدند. بنابراین، دوره سوم برای این روش همچنان مفید بود و حذف آن باعث از دست رفتن بخشی از بهبود اعتبارسنجی می‌شد. بااین‌حال، کوچک‌شدن میزان بهبود نشان می‌دهد افزایش زیاد تعداد دوره‌ها بدون پایش Dev احتمالاً سود محدودی خواهد داشت.

در نتیجه، سه دوره برای مقایسه عملی دو روش کافی بوده است، اما برای LoRA انتخاب بهترین checkpoint بر اساس Validation اهمیت بیشتری دارد. برای Adapter نیز ادامه آموزش فقط در صورتی مناسب است که کاهش زیان Full-dev در دوره‌های بعدی ادامه پیدا کند..

۴-۷-۶- بررسی خطر Overfitting

نوسان خط آموزش یا Mini-validation به‌تنهایی نشانه Overfitting نیست. ریزBatchهای کوچک، تفاوت طول توالی‌ها و ناهمگونی محتوای اخبار می‌توانند نوسان‌های زیادی ایجاد کنند. معیار معتبرتر برای این تحلیل، مقایسه روند زیان آموزش با Full-dev در پایان هر دوره است.

در LoRA، از دوره دوم به سوم:

- زیان آموزش از ۱/۷۷۸۸۱۱ به ۱/۷۲۶۳۱۴ کاهش یافت؛
- زیان Full-dev از ۱/۷۹۵۳۲۸ به ۱/۷۹۸۹۰۴ افزایش یافت؛
- پرپلکسیتی Full-dev از ۶/۰۲۱۴۵۱ به ۶/۰۴۳۰۲۳ افزایش یافت.

این الگو یک هشدار خفیف بیش‌برازش محسوب می‌شود؛ زیرا مدل Train را بهتر برازش کرده، اما تعمیم آن روی Dev اندکی ضعیف‌تر شده است. با توجه به کوچک‌بودن افزایش Validation، نمی‌توان از بیش‌برازش شدید سخن گفت، ولی Epoch 2 نقطه مناسب‌تری برای توقف LoRA بوده است.

در Bottleneck Adapter، زیان آموزش و Full-dev در هر سه دوره کاهش یافته‌اند. بنابراین، تا پایان Epoch 3 نشانه‌ای از بازگشت زیان اعتبارسنجی یا بیش‌برازش درون‌دامنه‌ای مشاهده نمی‌شود. بااین‌حال، کاهش کوچک‌تر Validation در دوره سوم نشان می‌دهد باید در دوره‌های بیشتر احتمال ایجاد Plateau یا افزایش زیان Dev کنترل شود.

افزایش پرپلکسیتی ویکی‌پدیا را نیز نباید مستقیماً Overfitting نامید. Dev و آزمون اخبار از همان دامنه آموزش هستند و برای تشخیص بیش‌برازش درون‌دامنه‌ای استفاده می‌شوند؛ درحالی‌که افت ویکی‌پدیا بیشتر نشان‌دهنده تخصصی‌شدن مدل، تداخل دامنه‌ای و فراموشی عملکردی است.

در مجموع، نتایج نشان می‌دهند:

- هر دو روش به‌صورت پایدار آموزش دیده‌اند.
- LoRA زیان Train را بیشتر کاهش داده، اما بهترین تعمیم آن در Epoch 2 رخ داده و Epoch 3 نشانه خفیفی از بیش‌برازش دارد.
- Bottleneck Adapter تا پایان Epoch 3 بهبود پیوسته Full-dev نشان داده و نشانه آشکاری از بیش‌برازش ندارد.
- سه دوره برای انجام مقایسه و دستیابی به بهبود عملی کافی بوده است، اما انتخاب checkpoint بر اساس Full-dev برای LoRA ضروری‌تر است.
- زیان آموزشی پایین‌تر LoRA به عملکرد بهتر روی Dev یا آزمون اخبار منجر نشده و Bottleneck Adapter مصالحه مناسب‌تری میان برازش داده‌های آموزش و تعمیم درون‌دامنه‌ای ارائه کرده است.

۸-۴- جمع‌بندی فصل

مقایسه نهایی مدل پایه، LoRA و Bottleneck Adapter نشان داد که هر دو روش تنظیم دقیق پارامترکارا توانسته‌اند مدل را با دامنه اخبار فارسی سازگار کنند. پرپلکسیتی مدل پایه روی آزمون اخبار برابر با ۷.۷۰۹۴۳۹ بود که پس از آموزش LoRA به ۶.۰۵۹۱۳۶ و پس از آموزش Bottleneck Adapter به ۵.۹۷۳۷۲۳ کاهش یافت. بنابراین، بهبود نسبی عملکرد درون‌دامنه‌ای برای LoRA حدود ۲۱.۴۱ درصد و برای Adapter حدود ۲۲.۵۱ درصد بود. این نتیجه نشان می‌دهد تغییرات آموخته‌شده فقط موجب کاهش زیان داده‌های آموزش نشده‌اند، بلکه به نمونه‌های خبری دیده‌نشده نیز منتقل شده‌اند.

بااین‌حال، تخصصی‌شدن مدل برای اخبار در هر دو روش با کاهش عملکرد روی متون عمومی ویکی‌پدیا همراه بود. پرپلکسیتی ویکی‌پدیا در مدل پایه برابر با ۷.۳۳۸۸۶۸ بود که پس از آموزش LoRA به ۹.۸۰۳۶۶۲ و در Bottleneck Adapter به ۸.۷۹۹۱۸۷ افزایش یافت. نرخ افت نسبی LoRA برابر با ۳۳.۵۹ درصد و نرخ افت Adapter برابر با ۱۹.۹۰ درصد بود. بر اساس دسته‌بندی عملیاتی تعریف‌شده در این آزمایش، LoRA در محدوده افت قابل‌توجه و سازگار با فراموشی شدیدتر برون‌دامنه‌ای قرار می‌گیرد، درحالی‌که Adapter در محدوده افت متوسط، اما بسیار نزدیک به آستانه ۲۰ درصد، قرار دارد. بنابراین، شواهد مربوط به تداخل دامنه‌ای و فراموشی عملکردی در LoRA به‌مراتب قوی‌تر از Adapter است.

فریزشدن وزن‌های مدل پایه مانع تغییر عملکرد عمومی مدل نشد. وزن‌های اصلی در هر دو روش ثابت باقی ماندند، اما مؤلفه‌های LoRA و Adapter پس از آموزش در تمام عبورهای پیش‌رو فعال هستند و نگاشت‌ها یا بازنمایی‌های میانی مدل را تغییر می‌دهند. در نتیجه، دانش ذخیره‌شده در پارامترهای اصلی الزاماً حذف نشده است، اما مدل در حضور ماژول‌های تنظیم‌شده، عملکرد پیشین خود را روی متون خارج از دامنه با همان کیفیت نشان نمی‌دهد. ازاین‌رو، افت مشاهده‌شده را می‌توان نوعی تداخل دامنه‌ای و فراموشی عملکردی دانست.

از نظر پارامترکارایی، Bottleneck Adapter وضعیت مطلوب‌تری داشت. تعداد پارامترهای قابل‌آموزش LoRA برابر با ۵,۵۷۰,۵۶۰ و تعداد پارامترهای Adapter برابر با ۲,۰۲۸,۰۰۰ بود. بنابراین، Adapter حدود ۶۳.۵۹ درصد پارامتر قابل‌آموزش کمتری نسبت به LoRA داشت. با وجود این ظرفیت پارامتری کوچک‌تر، Adapter هم پرپلکسیتی اخبار کمتری ثبت کرد و هم افت ویکی‌پدیای محدودتری داشت. در نتیجه، عملکرد بهتر این روش ناشی از استفاده از تعداد بیشتر پارامترهای قابل‌آموزش نبوده است.

مقایسه مصالحه میان یادگیری اخبار و حفظ عملکرد عمومی نیز به نفع Bottleneck Adapter بود. LoRA بهبود ۲۱.۴۱ درصدی اخبار را با افت ۳۳.۵۹ درصدی ویکی‌پدیا به دست آورد؛ درحالی‌که Adapter بهبود ۲۲.۵۱ درصدی اخبار را با افت ۱۹.۹۰ درصدی ویکی‌پدیا همراه کرد. بنابراین، Adapter علاوه بر ایجاد حدود ۱.۱۰ واحد درصد بهبود بیشتر روی اخبار، حدود ۱۳.۶۹ واحد درصد افت کمتری روی ویکی‌پدیا ایجاد کرد. این نتیجه نشان می‌دهد در شرایط این آزمایش، Adapter مصالحه مناسب‌تری میان تخصصی‌سازی خبری و حفظ عملکرد برون‌دامنه‌ای ارائه کرده است.

تحلیل منحنی‌های آموزش و اعتبارسنجی نشان داد که هر دو روش از نظر بهینه‌سازی روندی پایدار داشته‌اند. زیان آموزش LoRA در سه دوره از ۱.۸۳۷۹۲۱ به ۱.۷۷۸۸۱۱ و سپس به ۱.۷۲۶۳۱۴ کاهش یافت. در Bottleneck Adapter نیز این مقدار از ۱.۸۳۵۵۵۷ به ۱.۷۸۶۴۸۴ و سپس به ۱.۷۵۰۰۰۷ رسید. بنابراین، LoRA در پایان آموزش زیان Train کمتری داشت و داده‌های آموزشی را قوی‌تر برازش کرد.

بااین‌حال، نقاط Full-dev روی نمودارها رفتار متفاوتی را نشان دادند. این نقاط در گام‌های ۲۵۰۰، ۵۰۰۰ و ۷۵۰۰ قرار دارند و به‌ترتیب نتایج ارزیابی کامل مجموعه Dev در پایان Epochهای اول، دوم و سوم هستند. در نمودار LoRA این نقاط با دایره‌های سبز و در نمودار Adapter با مربع‌های بنفش نمایش داده شده‌اند.

زیان Full-dev در LoRA به‌صورت زیر تغییر کرد:

$$1.809363 \rightarrow 1.795328 \rightarrow 1.798904$$

بنابراین، بهترین نتیجه اعتبارسنجی LoRA در پایان Epoch 2 ثبت شد. در Epoch 3، با وجود ادامه کاهش زیان آموزش، زیان و پرپلکسیتی Dev اندکی افزایش یافتند. این واگرایی کوچک یک هشدار خفیف از شروع بیش‌برازش یا توقف بهبود تعمیم است و نشان می‌دهد انتخاب checkpoint پایان Epoch 2 برای LoRA مناسب‌تر بود.

در Bottleneck Adapter، زیان Full-dev در هر سه دوره کاهش یافت:

$$1.805581 \rightarrow 1.789198 \rightarrow 1.783826$$

بنابراین، بهترین نتیجه اعتبارسنجی Adapter در پایان Epoch سوم به دست آمد. اگرچه میزان بهبود در دوره سوم کمتر از دوره‌های پیشین بود، هیچ افزایش مجددی در زیان Dev مشاهده نشد و تا پایان آموزش نشانه آشکاری از بیش‌برازش درون‌دامنه‌ای وجود نداشت. این نتایج نشان می‌دهند پایین‌تر بودن زیان Train در LoRA به تعمیم بهتر منجر نشده است و Adapter با وجود زیان آموزشی بالاتر، عملکرد بهتری روی Dev و آزمون اخبار دیده‌نشده ارائه کرده است.

سه دور آموزش برای دستیابی به بهبود عملی روی اخبار کافی بود، اما کفایت آن برای دو روش یکسان نبود. برای LoRA، دوره سوم از نظر کاهش زیان آموزش مفید بود، اما عملکرد اعتبارسنجی را بهبود نداد؛ بنابراین، Early Stopping بر اساس Full-dev می‌توانست از ادامه غیرضروری آموزش جلوگیری کند. در Adapter، دوره سوم همچنان موجب کاهش زیان و پرپلکسیتی Dev شد، هرچند کوچک‌شدن مقدار بهبود نشان می‌دهد این روش نیز به ناحیه بازده نزولی نزدیک شده است.

ارزیابی کیفی تولید متن نشان داد که بهبود پرپلکسیتی اخبار الزاماً به معنای تولید متن صحیح‌تر یا قابل‌اعتمادتر نیست. هر سه مدل توانستند برای آغازگرهای کوتاه، خروجی‌هایی با ساختار قابل‌تشخیص خبری ایجاد کنند؛ بااین‌حال، هر سه در نمونه‌های بررسی‌شده اطلاعاتی را تولید کردند که در پرامپت وجود نداشت و با منبع بیرونی نیز پشتیبانی نشده بود.

مدل پایه در پرامپت پارلمانی، متنی روان و شبیه گزارش رسمی مجلس تولید کرد و موضوع «اصلاح قانون انتخابات»، تعداد دقیق آرا، نام کمیسیون و تعداد مواد طرح را به متن افزود. اگرچه این خروجی از نظر جمله‌بندی و قالب خبری مناسب بود، جزئیات دقیق آن بدون شاهد تولید شده بودند. در پرامپت هواشناسی نیز مدل پایه قالب یک خبر خبرگزاری را بازتولید کرد، اما جمله نخست را تقریباً تکرار کرد و نام رسانه، خبرگزاری، اصفهان، بازه دمایی و کد پایان پیام را بدون پشتوانه وارد متن کرد. در پرامپت سلامت نیز یک بیانیه رسمی و باورپذیر درباره شایعات کرونا و آمار دقیق مبتلایان ساخت. بنابراین، مهم‌ترین خطر مدل پایه تولید متن‌هایی با ظاهر معتبر و اطلاعات دقیق پشتیبانی‌نشده بود که ممکن است از سوی خواننده واقعی تلقی شوند.

LoRA در پرامپت پارلمانی، ساختاری روشن و منظم درباره بررسی لایحه بودجه سال ۹۹، سؤال از وزیر اقتصاد و رأی‌گیری مجلس ایجاد کرد. در پرامپت هواشناسی نیز خروجی آن از نظر ترتیب اطلاعات و سبک گزارش‌نویسی نسبتاً منسجم بود و ابتدا وضعیت کلی کشور و سپس جزئیات منطقه‌ای را مطرح کرد. بااین‌حال، موضوع بودجه، رویدادهای مربوط به وزیر اقتصاد، روزهای هفته، فهرست استان‌ها و وضعیت‌های جوی همگی بدون اطلاعات زمینه‌ای موجود در پرامپت ساخته شدند. نشانه [n] در برخی خروجی‌های مدل‌های تنظیم‌شده مشاهده شد. ازآنجاکه این نشانه در داده‌های آموزشی به‌عنوان جداکننده بخش‌ها یا نمایش خط جدید وجود داشت، بازتولید آن را نمی‌توان نقص مدل دانست؛ بلکه این رفتار نشان‌دهنده یادگیری الگو نشان‌دهنده یادگیری الگوهای قالب‌بندی موجود در مجموعه آموزش است. بااین‌حال، برای ارائه خروجی نهایی به کاربر می‌توان این نشانه را در مرحله پس‌پردازش به خط جدید تبدیل کرد.

خطرناک‌ترین خروجی LoRA مربوط به پرامپت وزارت بهداشت بود. مدل ادعا کرد تمام مراکز درمانی و بیمارستان‌های دولتی و خصوصی تعطیل شده‌اند و این ادعا را چند بار با عبارات مشابه تکرار کرد. این اطلاعات نه‌تنها در پرامپت وجود نداشتند، بلکه به دلیل ارتباط با دسترسی مردم به خدمات درمانی، در صورت برداشت واقعی می‌توانستند گمراه‌کننده و بالقوه خطرناک باشند. بنابراین، در میان خروجی‌های پرامپت سلامت، LoRA بیشترین خطر محتوایی را نشان داد.

Bottleneck Adapter در پرامپت پارلمانی لحن رسمی و ساختار قابل‌قبولی ایجاد کرد، اما تعداد دقیق آرا و قاعده‌ای درباره حضور افراد منتخب اعضای هیئت‌رئیسه در جلسات کمیسیون‌ها را بدون پشتوانه تولید کرد. در پرامپت هواشناسی نیز الگوی حرفه‌ای گزارش سازمان هواشناسی را بازتولید کرد، اما نام خبرگزاری ایمنا، روابط عمومی سازمان هواشناسی و فهرست طولانی استان‌ها را به متن افزود. در این دو پرامپت، Adapter بیشترین میزان جزئیات عددی، نهادی یا جغرافیایی پشتیبانی‌نشده را ایجاد کرد.

در پرامپت وزارت بهداشت، خروجی Adapter نسبت به LoRA از نظر پیوستگی مناسب‌تر بود و به‌جای اعلام تعطیلی همه بیمارستان‌ها، توصیه‌های عمومی سلامت مطرح کرد. بااین‌حال، علائم بیماری، دستور مراجعه‌نکردن به مراکز درمانی و شماره‌های ۱۱۱ و ۱۲۳ بدون هیچ منبعی تولید شدند. ارائه شماره تماس و توصیه پزشکی تأییدنشده نیز می‌تواند در کاربرد واقعی گمراه‌کننده باشد. بنابراین، هرچند شدت ادعای Adapter در این مثال کمتر از LoRA بود، خروجی آن نیز قابل‌اعتماد تلقی نمی‌شود.

از نظر کیفیت تولید و Hallucination نمی‌توان یکی از سه مدل را در تمام نمونه‌ها به‌طور مطلق برتر دانست. مدل پایه در موارد مختلف اعداد، نقل‌قول‌ها و انتساب‌هایی با ظاهر بسیار معتبر ساخت. LoRA در پرامپت‌های مجلس و هواشناسی ساختار منسجم‌تری ارائه کرد، اما جزئیات تأییدنشده و نشانه‌های قالب‌بندی داشت و در پرامپت سلامت خطرناک‌ترین ادعا را تولید کرد. Adapter نیز در پرامپت‌های مجلس و هواشناسی بیشترین جزئیات عددی و جغرافیایی بدون پشتوانه را وارد متن کرد و در پرامپت سلامت دستورالعمل و شماره‌های تماس تأییدنشده ساخت.

در نتیجه، کاهش پرپلکسیتی در LoRA و Bottleneck Adapter به حذف Hallucination منجر نشد. حتی در مواردی که خروجی از نظر سبک، روانی و ساختار خبر بهتر بود، احتمال تولید ادعاهای دقیق اما بدون شاهد افزایش یافته بود. بنابراین، معیار پرپلکسیتی باید در کنار ارزیابی مستقیم انسجام، صحت محتوایی، انتساب‌ها، اعداد و خطر ادعاهای حساس تفسیر شود.

از نظر منابع محاسباتی، Bottleneck Adapter در اجرای نهایی وضعیت مطلوب‌تری داشت. زمان خالص آموزش LoRA برابر با ۱۴,۴۱۸.۹۳۰۹۱۶ ثانیه و زمان آموزش Adapter برابر با ۱۱,۲۲۴.۲۶۵۴۱۵ ثانیه بود. بنابراین، Adapter حدود ۳,۱۹۴.۶۷ ثانیه، معادل تقریباً ۵۳ دقیقه و ۱۵ ثانیه، سریع‌تر آموزش دید و زمان آموزش آن حدود ۲۲.۱۶ درصد کمتر از LoRA بود.

Peak VRAM آموزش LoRA برابر با ۱۵.۳۶۰۲۰۵ گیگابایت و Peak VRAM Adapter برابر با ۱۵.۳۰۸۰۱۶ گیگابایت بود. در نتیجه، Adapter حدود ۰.۰۵۲۱۸۹ گیگابایت، معادل تقریباً ۰.۳۴ درصد، حافظه کمتری مصرف کرد. این اختلاف از نظر عملی بسیار کوچک است و نشان می‌دهد کاهش ۶۳.۵۹ درصدی پارامترهای قابل‌آموزش به کاهش مشابه Peak VRAM منجر نشده است؛ زیرا بخش عمده حافظه به مدل پایه، فعال‌سازی‌ها، Gradient Checkpointing و محاسبه گرادیان اختصاص دارد.

زمان Validation در Adapter بیشتر از LoRA ثبت شد، اما ارزیابی کامل Dev برای LoRA با Batch برابر ۴ و برای Adapter با Batch برابر ۲ انجام شد. بنابراین، این اختلاف نباید مستقیماً به کندتر بودن ذاتی Adapter در زمان استنتاج یا اعتبارسنجی نسبت داده شود. مقایسه قابل‌اتکاتر منابع نشان می‌دهد Adapter از نظر زمان خالص آموزش به‌طور محسوسی سریع‌تر بوده، درحالی‌که مصرف حافظه دو روش تقریباً برابر و اندکی به نفع Adapter بوده است.

در مجموع، Bottleneck Adapter از دید معیارهای کمی و محاسباتی در شرایط این آزمایش انتخاب مناسب‌تری بود. این روش با ۶۳.۵۹ درصد پارامتر قابل‌آموزش کمتر، پرپلکسیتی بهتر روی اخبار، افت محدودتر روی ویکی‌پدیا، روند اعتبارسنجی پایدارتر، زمان آموزش کوتاه‌تر و Peak VRAM اندکی کمتر از LoRA همراه بود. بااین‌حال، ارزیابی کیفی نشان داد این برتری عددی به معنای تولید محتوای صحیح‌تر و قابل‌اعتمادتر در تمام پرامپت‌ها نیست.

هیچ‌یک از دو روش نتوانستند تخصصی‌سازی خبری را بدون هزینه برون‌دامنه‌ای و بدون خطر تولید جزئیات پشتیبانی‌نشده انجام دهند. برای کاربرد عملی در تولید خبر، استفاده از بازیابی اطلاعات معتبر، محدودکردن تولید به شواهد بازیابی‌شده،

راستی‌آزمایی اعداد و انتساب‌ها، تبدیل نشانه‌های قالب‌بندی آموخته‌شده مانند [n] به خط جدید در مرحله پس‌پردازش و نظارت انسانی پیش از انتشار ضروری است.

نتیجه اصلی این فصل آن است که ارزیابی روش‌های تنظیم دقیق باید به‌صورت چندمعیاره انجام شود. زیان آموزش یا پرپلکسیتی اخبار به‌تنهایی برای انتخاب مدل کافی نیستند و باید در کنار حفظ عملکرد عمومی، روند Full-dev، خطر بیش‌برازش، Hallucination، کیفیت و پاکیزگی متن تولیدشده و هزینه محاسباتی تحلیل شوند. بر اساس مجموعه این معیارها، Bottleneck Adapter مصالحه کمی و محاسباتی مناسب‌تری ارائه کرد، اما کیفیت محتوایی خروجی هر سه مدل همچنان به سازوکارهای کنترلی و راستی‌آزمایی نیاز دارد.